

K4510

USER'S AND PROGRAMMER'S GUIDE

a fantasy 8/16-bit computer

Michael Longval
project orchestrator

Draft version: alpha-0.9+3
CPU 45GS10 · VICKY · MELODY · 256 MB RAM

Date: 07.10.2026

This is alpha documentation. It describes a machine that is still being built, and it is written alongside it. Everything in it — commands, keys, registers, the layout of files, the names of things — is subject to change, and some of it will have changed between this printing and the machine you are running. Where the two disagree, the machine is right and this book is behind it; Appendix B says how to tell us. The version on the cover is the build this book was made from.

Contents

I	User's Guide	1
1	The Machine	2
1.1	What is the K4510?	2
1.2	One machine, three ways to run it	4
1.3	Building the machine	5
1.4	First boot	6
1.5	The keyboard	7
1.6	The F12 menu	8
1.7	The menu file: what the menu shows	11
1.8	Placement, the register panel, and F8	16
2	The Shell	18
2.1	This book, on the machine	18
2.2	The disk	19
2.3	Running things	21
2.4	The file managers	22
2.5	SWAP: running one thing from inside another	23
2.6	At the prompt	23
2.7	Aliases	24
2.8	The status bands	24
2.9	Two screens	25
2.10	The network	26
2.11	CAPSLOCK	28
2.12	Scripts and the boot file	29
2.13	The monitor	29
2.14	Colours and palettes	30
2.15	Housekeeping	31
2.16	When STARTUP.BAT is the problem	33
2.17	When something goes wrong	33

3	Every Command	34
4	EhBASIC	42
4.1	Ten minutes of it	42
4.2	The machine's own statements	42
4.3	Hardware sprites	43
4.4	The * escape	45
4.5	Editing the program in VI	45
5	The Tube	46
5.1	Graphics and sound	46
5.2	Sprites	47
5.3	The terminal: JIM	48
5.4	Star commands	50
6	Forth	52
6.1	The machine at your fingertips	52
6.2	The assembler	52
6.3	Manners	53
7	LOGO	54
7.1	The words	56
7.2	Procedures	56
7.3	Files	57
8	Pascal	58
8.1	Turbo Pascal 3, on CP/M	58
8.2	Mad Pascal, the cross-compiler	59
8.3	The library reference	66
9	The Editors	75
9.1	EDIT	75
9.2	VI	77
9.3	PROG	82
9.4	WORD	85
9.5	Editing from inside a BASIC	86

10 RX: the Machine's REXX	88
10.1 Why REXX, and what ARexx was	88
10.2 The three environments	89
10.3 Ports: the file is the mailbox	89
10.4 What is in /LANG/RX	89
11 The Linux Underneath	91
11.1 ! — the Linux prompt, from the machine's prompt . . .	91
11.2 NVIM — Neovim, dressed for the machine	92
11.3 PAS and CC: compiling on the machine	93
11.4 What is on the other side	94
11.5 mosh: another computer, over a poor network	95
11.6 On a stick	96
11.7 Beside another system	97
11.8 From another computer	98
 II Programmer's Guide	 100
12 Memory	101
12.1 The 64 KB view and the 256 MB behind it	101
12.2 The CPU view, unmapped	101
12.2.1 Low memory	102
12.2.2 Who uses the 256 MB	103
12.3 Programs	104
12.4 System calls	104
12.5 RAM under the ROM	106
12.6 Sideways ROM	106
12.7 RAM under the I/O page	107
12.8 Programs bigger than the window	107
13 The I/O Page	108
13.1 A split screen: SHEILA	108
 A The Registers	 111
A.1 The I/O page	111
A.2 VICKY, SHEILA and the sprites	118

A.3	The network	123
A.4	JIM, the terminal	124
A.5	Memory and the far view	126
B	Filing an Issue	127
B.1	Before you try again	127
B.2	BUG writes the report	127
B.3	Where it goes	128
C	The Sound, and What It Took	130
C.1	Four chips, because it is a fantasy	130
C.2	A SID is not a CPU	131
C.3	“Choppy choppy choppy” — which was not the sound	131
C.4	Silence is not free	132
C.5	The ring that started empty	132
C.6	Chips that play on without the machine	132
C.7	The meter that read zero	133
C.8	A machine keeping somebody else’s time	134
C.9	Where it stood, when it was decided	135
C.10	What was decided	135
C.11	And the moral, once more	136
D	The Code Page	138
D.1	The K4510 page	138
D.2	The languages each writes	140
D.3	The first 32, and \$7F	141
D.4	Typing them	141
D.5	A BBS	141
	Disclaimer	142
	Thank You	144
	The heart of the machine	144
	The tongues	144
	Letters on the screen	145
	The bare metal, which the machine no longer stands on	146
	The workshop	147

Heritage	147
And whoever is missing	148
Licences	149
The project itself	149
Code inside the machine	149
Fonts	150
The Linux the machine boots on	151
Build tools	151
What is deliberately not shipped	152
If you redistribute the machine	152

Part I

User's Guide

Chapter 1

The Machine

The K4510 is a fantasy computer: a machine that never existed, built the way 1985 might have built it with no budget committee in the room. It is not an emulation of any real computer. Its CPU, video chip, sound, and operating software are its own; the parts it borrows — a 6502-family instruction set, the AdLib's FM chip — it borrows openly and then outgrows.

1.1 What is the K4510?

- **CPU: the 45GS10** — the MEGA65's 45GS02 instruction set (a 4510 with the Q pseudo-register and 32-bit flat addressing) plus this machine's own memory management: bank registers, a far-call gate, and RAM under the ROM.

What the MHz means. The clock is how many CPU cycles the machine runs in a second of its own time, and it decides one thing: how soon a long calculation finishes. It does not decide how quickly the machine *answers*. The console, the files, the network, DMA and the MATH unit work at the host's full speed whatever the clock says, so typing, DIR and loading a program feel the same at 15 as at 40.5 — measured, DIR of a full directory takes a twentieth of a second longer at 15, while a Mandelbrot takes 2.7 times as long. Nor is it another computer's megahertz: the cycles are counted with the MEGA65's timings, and the machine is a fantasy whose timings are suggestions. For a figure that compares, MARK prints beside each result the clock a real 65C02 would need to match it — at 40.5 the K4510 is a 65C02 at 43 to 51, and nearly three times that with the MATH unit.

The clock is whatever the host it runs on can hold at sixty frames a second with no gaps in the sound. It is a setting (F12, under Machine — six steps from 60 MHz down to 10; the ladder goes on to 202.5, and those faster steps are held back for now). A host nobody

has measured runs at 40.5, the MEGA65's number; SETUP measures it properly, with the sound and the picture really running, and the machine keeps the answer for that host so later boots pay nothing. It never second-guesses a clock you chose yourself. A desktop of the last ten years holds a hundred megahertz or more. INFO reports the clock in force, read live, and BENCH is there when you want the whole picture.

- **Memory: 256 MB**, flat, 28-bit. The CPU sees 64 KB at a time; everything else is one instruction away.
- **Video: VICKY** — 640×480, 256 colours from a 24-bit palette, four layers, 128 sprites, a blitter that draws lines and filled triangles, and a display-list coprocessor named SHEILA. Smaller modes (640×240, 320×240, 320×200, 160×200) are drawn into the same glass, doubled and centred, so the raster is always 480 lines however few of them the picture uses. The 640×480 screen comes two ways in the menu: 640x480, eighty columns by thirty in tall 8×16 letters, and 640x480x60, eighty by sixty in 8×8 — the screen the machine had before it settled on one font. `MODE 0 -d` and `MODE 0 -n` choose them at the prompt, and the machine remembers which. The HD screens are the panel's own: the machine finds the display's native size, takes the largest 4:3 in it (or all of it, `F12` → Video → Canvas), and divides that by a whole number — the *integer display resolutions*, every machine pixel the same square of panel pixels. A 1920x1080 panel offers 1440x1080 (`MODE -s 1`), 720x540 (`-s 2`), 480x360 (`-s 3`) and 360x270 (`-s 4`); a 1366x768 one 1024x768, 512x384 and 341x256; a 4K panel starts at 1440x1080 shown twice over, since a larger picture would cost more than a frame to draw. `MODE -1` lists what this panel offers; `MODE 720x540` picks one by its size, and `MODE 5, 6` and `7` are scales 1, 2 and 4. Each comes in larger cells and, with `-d` (`--double: MODE -s 2 -d`), smaller ones: at 1440x1080 sixteen wide, 90 columns by 33 or 67 rows; at 720x540 eight wide, drawn with the panel's pixels (below), 90 by 33 or 67. K/OS keeps to 132 columns and 25 rows at least. How large a picture is too large depends on the computer: !k4510-vidcap times this one and writes its *pixel cap* into `k4510.cfg`, so a slow laptop offers fewer and smaller resolutions rather than a

machine that cannot keep up.

- **Sound: MELODY** — an OPL2, the Yamaha YM3812, at \$D480: nine FM voices wired the AdLib's way, an address port, a data port and a status register you poll. Any AdLib register list or instrument patch therefore means what it says on this machine. A four-channel sound sequencer at \$D5E0 plays through it, in the BBC Micro's idiom, which is what BBC BASIC's SOUND and Mad Pascal's Sound use. **FRED**, the floating-point MATH unit, sits beside it, which the BASICs and LOGO lean on. (The machine had four SID chips until September 2026. Appendix C is the whole story.)
- **K/OS**: a shell with directories, in ROM — and on the disk, one word away each, a shelf of languages: EhBASIC with graphics and sprites, BBC BASIC on the Tube, Forth, LOGO, CP/M on a Z80, and RX, the machine's REXX. Two compilers are one word away as well: PAS and CC turn a Pascal or C source in the directory you are standing in into a program beside it (Chapter 11).

1.2 One machine, three ways to run it

The computer is the **K4510**: the 45GS10, VICKY, SHEILA, MELODY, FRED, JIM and K/OS. There is one of it. What changes is what it is standing on.

As a whole computer, from a stick. Write the image (`linux/build-live.sh`) to a USB stick, boot a spare laptop from it, and the machine is the machine: a minimal Debian that exists only to hold it up, loaded into RAM, keeping its settings and your files on the stick and never touching the laptop's own drive.

As a whole computer, installed beside another system. `linux/install-k4510.sh` copies the same system from the stick onto a partition of a computer's internal disk and adds it to that computer's boot menu, so the machine is one choice of two when it starts. The other system is not changed, and the computer boots whichever of the two was used last (Chapter 11).

Either way, the Linux underneath is not hidden. The cross-compilers, git, an editor and a second terminal are on it, and the machine can reach

them (Chapter 11); but it is furniture. Switch on and you are at the `/HOME]` prompt.

As a window. Run the same program on a Linux desktop and you get the same machine in a window, which is how it is developed and tested, and how most of the figures in this book were taken. There is also a sandboxed container flavour (`linux/podman.sh`) that is shown nothing of your computer but the display, the sound, the game controllers and one shared folder.

Same ROM bytes, same software, every way. Where the desktop window and the whole computer differ, the book says “on a desktop” or “on the K4510’s own Linux”. (Until 2026-09-07 there was also a bare-metal edition for the Raspberry Pi 3B+; it was retired so that every feature is built once. A Pi still runs the machine under Linux like any other computer; the tag `alpha-0.5` is the last tree with the bare-metal edition in it.)

1.3 Building the machine

On a **Linux desktop**, three lines build the whole machine:

```
git clone https://github.com/mlongval/k4510
cd k4510 && ./setup.sh
./k4510
```

`setup.sh` installs the compilers and SDL2, builds everything, and runs the machine’s test suites.

Or: the whole computer

```
sudo ./linux/build-live.sh
```

makes the bootable image: a small Debian, the machine and its compilers, about half an hour the first time. Write it to a USB stick with `dd` and boot from it; to put it on a computer’s own disk instead, boot that computer’s usual Linux with the stick in and run `sudo ./install-k4510.sh` from it. Chapter 11 has both.

What it needs

The machine asks little of a computer, and most of what it asks is for the Linux under it:

Processor	64-bit x86, two cores; a Core 2 is enough to run it, and what the processor decides is only the clock
Memory	2 GB; 4 is comfortable. The whole system is loaded into RAM at boot (about 0.9 GB), and a running machine uses 1.7 GB in all, the emulator itself 90 MB of it
Graphics	any Intel or AMD chip Linux drives natively (KMS); no desktop is used
Screen	640×480 or more
Firmware	UEFI or BIOS, either
Storage	a 2 GB USB stick, or a 2 GB partition

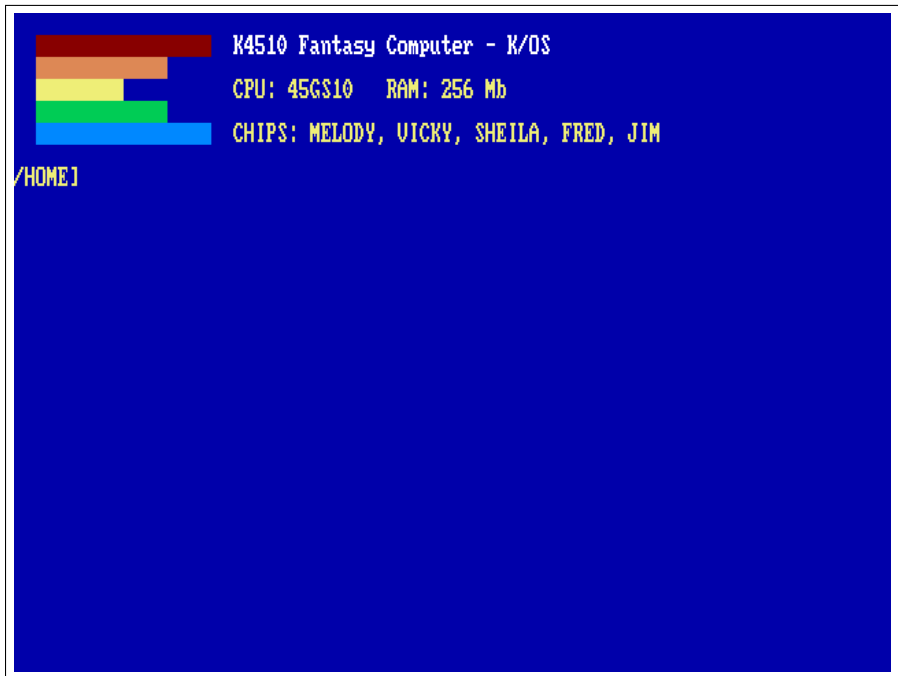
In a window on a desktop it needs only what the desktop has already: SDL2 and a few hundred megabytes of memory.

1.4 First boot

Switch it on (on a desktop: `./k4510`; otherwise, power up). The machine's logo is on the screen while Linux starts, and then the machine greets you in yellow on blue:

The `/HOME]` at the bottom is the shell prompt — the part before the `]` is the directory you are in, and the machine starts you in `/HOME`, which is yours. Type `HELP` for the commands, `INFO` for the machine's full self-description, and `DIR` to see your files. The first time on a new computer, type `SETUP` and let the machine measure it.

Keys: Escape is RUN/STOP (stops a BASIC program), Ctrl-C is STOP, Shift+Escape quits the emulator. Reset is a chord, the way it was Commodore+Restore on the C64: Super+PageUp (the Windows or Command key and PageUp) — one key cannot do it by accident. F12 opens the settings menu; F8 pauses the machine and turns the keyboard into a debugger's. PrtSc takes a screenshot — the



The boot screen: five stepped colour bars, then the machine, its CPU, its memory and its chips — and no clock, because the machine no longer has one speed. Everything else, the clock in force included, is one INFO away, and BANNER prints this again.

screen flashes, and the picture is in shots/ beside the emulator: the machine's own, and beside it a -full one of the whole display, border and all.

1.5 The keyboard

The keyboard is the one in front of you, and F12 → Input → *Keyboard layout* says what its keys mean: US, US-International, Canadian French, French, German, Spanish, UK or Italian — or *Host*, the first choice, which leaves the question to the desktop or Linux the machine is running on. A layout chosen here takes effect at once, with its AltGr and its dead keys:

on US-intl, ' then e is é, the dead key twice or before a space gives the mark itself, and a ' or " before a letter that takes no accent types itself. On the K4510's own Linux the Linux consoles follow the same choice. A letter the machine's character set (the IBM PC's) cannot draw is not offered. For programming, plain US has no dead keys to step round.

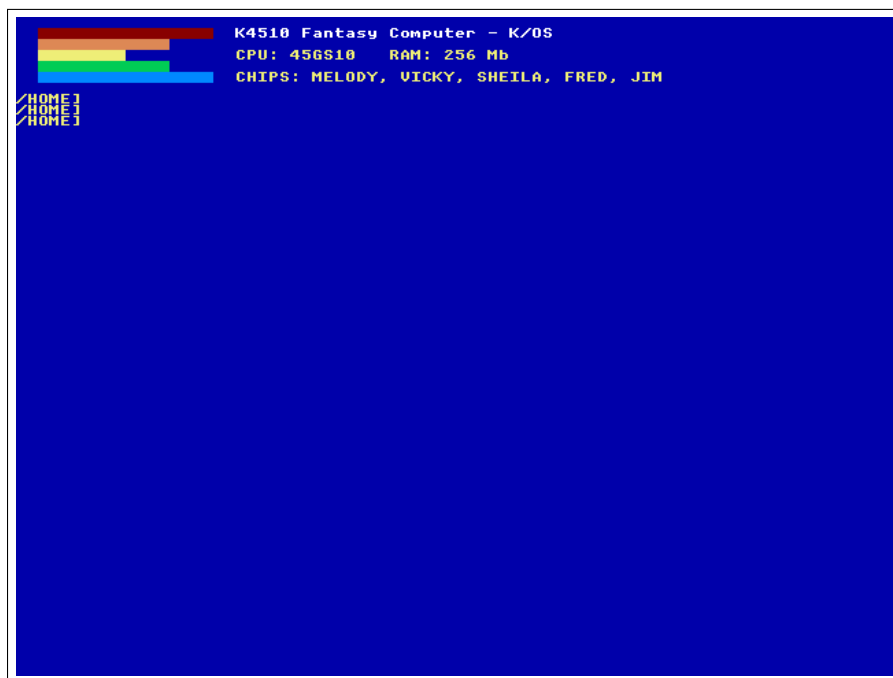
Caps Lock is Ctrl, beside it, makes the Caps Lock key a second Ctrl, where the old Unix keyboards had it; the shell's own CAPSLOCK command (Chapter 2) is how you get capitals without it.

1.6 The F12 menu

F12 freezes the machine and takes the whole screen, in the manner of the C64 Ultimate's: the categories down the left, the settings of the chosen one on the right, and a line at the foot saying what the keys do in whichever pane holds the cursor. Sound stops, and closing the menu resumes exactly where it froze. The menu draws with its own copy of the font and never touches the machine's memory, so it opens even when a program has wrecked the screen.

Up and Down move; Enter or Right crosses from the categories to the settings; Left and Right step a value; Escape goes back a pane, and closes at the categories; F12 closes from anywhere. The mouse works too. Shift+F12 pauses the machine instead, the picture held; Shift+F12 again goes on. The menu was on F7 until the autumn of 2026, a habit from the Commodore's keyboard; most emulators keep theirs on F12, and F7 and F8 are ordinary keys now, for the programs that want all ten. Input → *Menu key* puts it back on F7 for fingers that remember. A setting with a list of choices opens that list over the panes, and *applies as the cursor passes each one* — the font changes under the menu so you can see it — with Escape putting back whatever was there when the list opened.

Video the border's width, and the *frame colour* — the border's and both status bands', one colour, dark grey to begin with. *Frame follows palette*, on to begin with, makes it that colour's entry in the palette in use (amber under AMBER); off, the frame keeps the colour itself whatever the palette, and the bands' text is black or white,



The F12 menu: categories on the left, the Video page on the right.

whichever reads better on it. *Resolution*, which is the MODE command's knob: this panel's integer display resolutions, each with its text grid ("720x540 90x33"), kept by scale so a machine moved to another panel comes up at the same /n (a program may still put the machine in 640x480 and the others, and the row says so); *canvas*, 4:3 or the whole panel; *font*, the text's face at the panel's own pixels. At scale 1 it is the machine's font, 16 wide; at an even scale each 8x16 cell is drawn as 16x32 (an 8x8 one as 16x16), and at scale 3 — 480x360 on a 1080 panel — as 24x48 (24x24), while the graphics stay the machine's, and a character a program has redefined keeps its own shape. The faces: *Zhekov Bold* (to begin with) and *Zhekov* — Dimitar Zhekov's Terminus Font, renamed as its licence asks of a changed copy — *Spleen*, *IBM VGA*, *Atkinson Mono* (the Braille Institute's face for low vision), *Go Mono*, *Fira Mono*, *Proggy Clean*, *Tamzen Bold*, or *unscii*, the machine's own

drawn doubled; and *placement*, which is the section after next. The picture is always full screen and scaled by a whole number, every pixel the same size. The figures in this book are taken with the effects off.

Terminal *status bands*, on or off (a row at the top, a row at the bottom) (Section 2.8), and the clock and date format they print.

Audio volume.

Input the reset chord; which key opens the menu; whether a click captures the mouse pointer, and whether the host's pointer shows over the picture (full screen, the pointer stays on the machine's picture, and goes into the side panel only when there is one); the keyboard, above; and the *key pipe*, typing from another computer (Chapter 11): *off*, *on*, or *on, shown* — the one it starts at — where every key typed that way is echoed in a bar at the foot of the window for a few seconds, so nobody types into the machine unseen.

Machine *save state* and *load state*, four slots each (the whole machine — CPU, every used page of the 256 MB, VICKY, the devices, JIM — to k4510-slotN.k4s beside the settings file; the Tube co-processor is not in the file and is stopped by a load); reset; power cycle; stop the Tube; quit; *CPU clock* — the steps, live, 60 MHz at the most for now; choosing one switches *Auto clock* off, because a clock chosen by hand is not to be second-guessed. *Auto clock* on uses what SETUP measured on this host. On the K4510's own Linux there are two last rows: *Save and power off*, which keeps the machine as it is — the program, the screen, the memory — and brings it back at the next power-on, and *Shut down the computer*.

Shell whether an unknown word at the prompt may run a CP/M .COM — off to begin with, on purpose: D typed for DIR should not start a Z80 program; and whether /STARTUP.BAT runs at power-on, which is the way out of a bad one (Section 2.16).

Info the version and the exact build, the ROM, the files, the host — the K4510's own Linux, or a window on a desktop — and the battery, where there is one.

Host on the K4510's own Linux only: the computer's name, its network address and Tailscale's; *Lid closed*, which is *keep running* to begin with — the machine goes on with the lid down — or *suspend*; *Charge to 100% once*, for a trip; a *Wi-Fi / network setup* row that opens the network settings on a spare console, and a row that telnets into the Linux for you.

Leaving the menu writes the settings to `k4510.cfg` beside `fs/` — a plain `key = value` file you may edit; comments and keys it does not know are kept, which is how a settings file survives the machine changing under it.

Vertical sync is off, and that is deliberate. With it off the machine keeps its own 60 Hz and presents a frame when it is ready; the cost is tearing. With it on, the pacing belongs to your display, and on a host whose refresh is not exactly 60 that costs frames — and frames are sound here (Appendix C). It is no longer a row in the menu; `video.vsync` in `k4510.cfg` still turns it on.

1.7 The menu file: what the menu shows

Beside `k4510.cfg` is a second file, `k4510-menu.cfg`, and it decides not what the settings *are* but which of them the menu *offers*. The machine writes it out in full the first time it starts — every category and every row, each marked `show` — so the file is itself the list of what can be changed:

```
[K4510]
Audio           = show
...
[Video]
Border width    = hide
...
[Locks]
linux          = open
consoles       = open
```

- `hide` on a row takes it out of the menu, and a category with nothing left in it goes too; `hide` under `[K4510]` takes a whole category away. A

hidden setting keeps the value it has in `k4510.cfg`: it is not reset, only out of reach.

- `linux = locked` shuts every door into the Linux underneath: `!`, SSH, and the telnet row of the Host menu. `PAS` and `CC` still compile. `consoles = locked` makes `Ctrl+Alt+F2` to `F6` do nothing.
- Names are the menu's own, in either case; `#` starts a comment; a row the file does not name is shown. The file is read when the machine starts, so a change takes effect at the next start.

The file lives outside the machine's disk, so nothing running on the machine can undo it. On the K4510's own Linux it is in `/home/k4510/k4510/`; once Linux is locked, edit it over ssh from another computer.

Every row

The list below is generated from the menu's own source, with the name each setting has in `k4510.cfg`, its choices and where it starts.

Video

Border width	<i>video.border</i>
0 to 64; to begin with, 0	
Frame colour	<i>video.border_colour</i>
Resolution	<i>video.mode</i>
Canvas	<i>video.base</i>
4:3, full; to begin with, 4:3	
Font	<i>video.hdfont</i>
unscii, Zhekov Bold, Zhekov, Spleen, IBM VGA, Atkinson Mono, Go Mono, Fira Mono, Proggy Clean, Tamzen Bold; to begin with, Zhekov Bold	
Placement	<i>video.placement</i>
centre, left, right; to begin with, centre	
Frame follows palette	<i>video.frame_follow</i>
on, off; to begin with, on	
Palette	
the VIC-II sixteen, or any .PAL in <code>/SYSTEM/ETC/PALETTES</code> : <code>PALETTE LOAD</code> , typed at the prompt	

Terminal

Status bands on, off; to begin with, off	<i>term.bands</i>
Battery time on, off; to begin with, off	<i>term.battime</i>
24-hour clock on, off; to begin with, on	<i>term.clock24</i>
Date format DD.MM.YYYY, YYYY-MM-DD, MM/DD/YYYY; to begin with, DD.MM.YYYY	<i>term.datefmt</i>
Code page CP437, K4510; to begin with, CP437	<i>text.codepage</i>
K/OS screen (Alt+1) does it	
Terminal screen (Alt+2) does it	

Audio

Volume 0 to 100; to begin with, 80	<i>audio.volume</i>
--	---------------------

Input

Reset chord Super+PageUp, Ctrl+PageUp, Alt+PageUp, Ctrl+Alt+Del; to begin with, Super+PageUp	<i>input.reset_chord</i>
Menu key F7, F8, F11, Pause, F12; to begin with, F12	<i>input.menu_key</i>
Mouse capture on, off; to begin with, off	<i>input.mouse_grab</i>
Mouse pointer on, off; to begin with, on	<i>input.mouse_pointer</i>
Caps Lock is Ctrl on, off; to begin with, off	<i>input.caps_ctrl</i>

Keyboard layout*input.kbd_layout*

Host, US, US-intl, Canada-FR, France, Germany, Spain, UK, Italy; to begin with, Host

Key pipe*input.keypipe*

off, on, on, shown; to begin with, on, shown

Machine

Save state

4 slots, each showing when it was written

Load state

4 slots, each showing when it was written

Reset

does it

Power cycle

does it

Stop the Tube

does it

Quit the emulator

does it

CPU clock*cpu.clock*

60 MHz, 40.5 MHz, 30 MHz, 20 MHz, 15 MHz, 10 MHz; to begin with, 40.5 MHz

Auto clock*cpu.auto*

on, off; to begin with, on

Save and power off

does it

Shut down the computer

does it; only on the K4510's own Linux

Shell

CP/M .COM by name*shell.cpm_com*

on, off; to begin with, off

Run STARTUP.BAT*shell.startup*

on, off; to begin with, on

Info

Version

shows

Build

shows

ROM

shows

Files

shows

Host

shows; only on the K4510's own Linux

Battery

shows

Host

Only on the K4510's own Linux.

Name

shows

Address

shows

Tailscale

shows

Lid closed

keep running, suspend; to begin with, keep running

host.lid

Charge to 100% once

on, off; to begin with, off

host.charge_once

Wi-Fi / network setup

does it

Telnet into the host

does it

1.8 Placement, the register panel, and F8

A 4:3 picture on a 16:9 screen leaves a third of the glass empty. *Placement* (centre, left, right) puts the picture against one edge instead of in the middle; what is left is the frame colour, dark grey like the status bands unless you choose another. (The sidebars — scenes, a gradient, the register panel beside the picture — are switched off for now, and their rows are out of the menu.)

Last, *Palette* lists the VIC-II sixteen and every .PAL in /SYSTEM/ETC/PALETTES (one you made with PALETTE SAVE shows up the next time the menu opens). Choosing one types PALETTE LOAD at the prompt for you (Chapter 2); with a program running, it waits until the program has ended. As with the command, it is kept: the machine comes back in it at the next power-on and after every reset, until another is loaded or PALETTE RESET brings back the VIC-II sixteen.

The panel is a strip twenty-six columns wide taking the whole height of the window, one fact to a line: the program counter, A X Y Z, the stack pointer, the B register, the flags, the next few instructions disassembled, VICKY's mode and the raster line it is on, the eight bank registers with the engaged ones lit, the audio gaps, the frame counter and the frame rate. It never reads through the I/O page — a read of \$D100 would pop a key off the keyboard, and an instrument that changes what it measures is not one.

F8 pauses the machine, and then the keyboard is the debugger's:

Key	Does
F8	pause, and again to run on
Space	one instruction
L	the rest of this scanline
F	the rest of this frame
D	write a dump — dumps/dump-NNN.txt, the same file DUMP writes, and arm the instruction recorder so the next one has history
T	start or stop an instruction trace to /SYSTEM/LOG/TRACE.TXT: one line per instruction with the registers after it, stopping itself at 200,000 lines

Every other key is swallowed while the machine is paused. The panel shows the legend and the state: which scanline is next, whether the trace is running and how long it is, and the number of the last dump.

Chapter 2

The Shell

The shell is what the machine boots into. It is a file manager, a program launcher and the front door to everything else, and every command here also works from a BASIC with a * in front. Type HELP for a summary on the screen — the text it prints is itself a file on disk, /SYSTEM/ETC/HELP — and see Chapter 3 for every command with a line about each. This chapter is how the shell behaves.

2.1 This book, on the machine

BOOK is this handbook, on the machine it describes — no browser, no printer:

BOOK	the contents
BOOK 3	chapter 3, every command
BOOK SHELL	the chapter whose title says SHELL

The arrows, PgUp and PgDn (or Space) and Home and End move through a page. Tab chooses the next link and Shift+Tab the one before; Enter follows it and Backspace comes back, as far as eight pages. / finds a word further down, and n finds it again. Q or Escape leaves. A link marked *[picture]* is one of this book's screenshots, and the picture is there in the page beneath it, at half size; Enter on the link shows it full screen, drawn by the machine as the machine drew it, and any key comes back to the page.

The pages are made from the same source as this book and its web edition, at the same time, so the three never disagree. They live in /SYSTEM/DOC, one to a chapter, in Gemini text — a small, open format where a line's first characters say what it is: # a heading, => a link, * an item in a list. So TYPE reads them too, and so does any Gemini reader on another computer.

2.2 The disk

Files live on the host: the `fs/` directory beside the emulator. The machine sees that directory as `/` and nothing above it. The root has seven folders and one file in it, and that is fixed — everything else lives one level down:

<code>/SYSTEM</code>	the machine's own, in three parts: BIN the tools (VI, TYPE, MONITOR, RANGER, SETUP, BUG...), ETC what it reads (HELP, VI.SAMPLE, STARTUP.SAMPLE, PALETTES/, a <code>chargen.bin</code> if you add one), LOG what it writes (BENCH-*, TRACE.TXT, BUGREPORTS/)
<code>/LANG/name</code>	one folder per language: its runtime, a README and an EX/ of examples. PASCAL and C keep the sources <i>beside</i> the programs they compile to
<code>/APPS/name</code>	one folder per program, with its data beside it — LOD's levels, CHESS's openings, OPLPLAY's tunes
<code>/HOME</code>	yours. The prompt starts here
<code>/CPM</code>	the Z80's drives, A to P, one folder each
<code>/MNT</code>	things from outside: SHARE, the folder the container flavour is given, and anything you MOUNT
<code>/DOCUMENTS</code>	papers about the machine, kept where the machine can read them. SHIPPING.CFG is the list of everything built, each thing marked essential, maybe or nope: edit it here with VI, and the build reads it
<code>/STARTUP.BAT</code>	yours; run at power-on (Section 2.16)

The search path, which is why you never type a path

A bare name not found where you are is looked for, in this order, in `/SYSTEM/BIN`, then in `/APPS/NAME/`, `/LANG/NAME/` and `/HOME/PROJECTS/NAME/` — where *NAME* is the name you typed without its extension, upper-cased, because on this disk **the name is the folder** — and then by extension: a `.BAS` in `/LANG/EHBASIC/EX`, a `.BBC` in `/LANG/BBCBASIC/EX`, a `.RX` in `/LANG/RX`, a `.PAS`, `.C` or `.prg` in the two compiled languages' folders. So SKYFIRE, EHBASIC, RANGER and HELLO run from wherever you are standing, and RUN "INVADERS.BAS" inside EhBASIC still finds it. A name is matched regardless of case when the exact one is absent, and `..` never climbs above `/`.

```

DIR -a *.BAS      LS is the same; -a shows the hidden names
CD /LANG/LOGO    the rest of the line is the name, spaces and all
TYPE README.TXT  a screenful at a time; Esc or Q stops
CP README.TXT /HOME

```

Names that begin with a dot are hidden; DIR -a shows them, and DIR -l lists one name to a line. DIR also takes a directory to look in, or a pattern to match: DIR APPS lists that folder without going there, DIR *.PAS and DIR ?.TXT match here (* any run of characters, ? any one).

```

K4510 Fantasy Computer - K/OS
CPU: 45GS10  RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME] cd /
/l dir
directory of /
APPS                <DIR>          CPM                <DIR>
DOCUMENTS           <DIR>          HOME               <DIR>
LANG                <DIR>          MNT                <DIR>
STARTUP.BAT         529          SYSTEM             <DIR>
1 file(s), 529 bytes
/l

```

DIR at the root: the disk's whole shape in seven folders and one file. Directories come first, in white; files carry their sizes; two columns.

Nothing is deleted in a hurry

RM moves a file to /.TRASH rather than destroying it. RM -f is the one that really removes. DELETE -l lists the trash, DELETE -r name puts one back, and DELETE -e empties it; the file managers' own delete uses the same

trash. RENAME and CP refuse to overwrite an existing file, and take `-f` when you mean it — the host's own habit is to overwrite without a word, and that habit has cost this project a file.

2.3 Running things

`RUN cube.prg` runs a program; so does `RUN cube`, and so does plain `CUBE` — an unknown word is tried as a program on disk before the shell gives up, *with its arguments*, the way REXX did it on the mainframes. A program reads its arguments through the `ARGS` system call; `SAY` is the demonstration:

```
SAY HELLO FROM THE DISK
```

prints `HELLO FROM THE DISK` — there is no `SAY` command, only a `say.prg` in `/SYSTEM/BIN`. The same rule takes an `.RX` script (Chapter 10), so a REXX program is a command too. Only a program is run this way: typing the name of a text file at the prompt does not load it over the machine.

Several of the machine's commands are programs in exactly this sense — `TYPE`, `VI`, `SETUP` and the monitor among them — and you cannot tell from the prompt which is which, which is the point. The disk is the host's (and on the K4510's own Linux, in RAM), so a program starts as quickly as a word in the ROM.

These words open whole languages: `EHBASIC` (Chapter 4), `BBC` (Chapter 5), `FORTH` (Chapter 6), `LOGO` (Chapter 7), `RX` (Chapter 10) and `CPM`.

What is on the disk

`/APPS` holds the programs the machine ships with, one folder each, and any of them runs by name from anywhere:

LODE	a lode-runner: dig, climb, collect the gold. The levels are text files in its folder and E on the title card is a level editor
SKYFIRE	a formation shooter, in the Galaxian rules
FLUFFY	a platformer: variable jump height, stomping, spikes
BOMBER	a bomb-party arena
CHES	a chess set with an engine; it can also be driven from a script (Chapter 10)
OPLPLAY	plays .OPL streams from its TUNES/
MANDEL	the set, drawn by the machine
CUBE	a rotating solid

2.4 The file managers

Two of them, because they are two different ideas about what a file manager is for, and neither is a compromise.

KOMMANDER is the one with two panels and function keys along the bottom — Tab moves between the panels, and the keys copy, move, make a directory and delete. RANGER is the other tradition: three miller columns, what is under the cursor previewed in the right-hand one, and vi's fingers (hjk_l, gg, G, / to search).

Both share the rules that matter:

- **Enter on a directory** goes into it.
- **Enter on a program** — a .prg, or a CP/M .com — leaves the file manager and runs it, because the output of a program belongs on the machine's screen and not inside a browser's frame. A .com starts CP/M to do it.
- **Enter or F4 on a text file** opens it in VI or EDIT, and you come back to where you were standing. In RANGER the program is chosen by the file's extension, from /SYSTEM/ETC/RANGER.RC: one rule a line, the extension and then the command (PAS PROG, TXT EDIT -s), the first that matches winning. As shipped, Pascal, C and BASIC files open in PROG and everything else in VI; edit the file to change it.

- **DD deletes** to /.TRASH, the same trash the shell's RM uses, so **DELETE -r** brings it back.

2.5 SWAP: running one thing from inside another

A program is loaded where the last one was, so starting one from inside a BASIC would ordinarily flatten the program you were writing. SWAP command puts the caller away first — the whole of the CPU's memory, and the screen — runs the command on a clean machine, and gives the caller back untouched:

```
*SWAP EDIT MYPROG.BAS
```

from EhBASIC edits a file and returns to your program, variables and screen as they were. One level deep; the thing being run must be an ordinary program, and a BASIC's far memory is not disturbed because nothing else touches it. BBC BASIC needs none of this: it lives on the co-processor, out of reach of anything running here.

SWAP -k is the same but *keeps* the screen the program left, instead of restoring the caller's. A file manager wants its display back; a script that ran a program to read what it printed wants the other thing, which is why RX uses this form.

2.6 At the prompt

The line you are typing is editable: Left and Right move through it, Home and End go to its ends, Delete takes the character under the cursor and Backspace the one before it, characters insert where the cursor is, and Escape clears the line. It works across a line that has wrapped, and it takes accented letters.

Up and Down walk the last eight lines you typed, and Down past the newest gives you the empty line again. A line that repeats the one before it is not kept. The history lives in the ROM bank the line editor has to itself, so it survives a reset — but not a power cycle, which is what /STARTUP.BAT is for.

Options are written as on Unix. A dash and a letter (DIR -l), several letters behind one dash (DIR -al), or a word behind two (DIR --long); an option that takes a value has it next (MODE -s 2, MODE -s2) or after an equals sign (MODE --scale=2), and -- ends the options. MODE, DIR, INFO, SWAP and the programs that take options — EDIT, PROG, WORD, DELETE, FONTED — all read them the same way, through one parser.

2.7 Aliases

ALIAS gives a name to a line.

```
ALIAS                list them
ALIAS LL DIR -l      define one
ALIAS LL             nothing after the name: remove it
```

Whatever you type after the alias is added to the end, so a definition takes arguments. Aliases are tried *last*, after every other rule, so one can never hide a real command: ALIAS DIR ECHO no is accepted and DIR still lists the directory. They live in memory — in a sideways bank, not in the 64 KB — so they survive a reset but not a power cycle, which is what /STARTUP.BAT is for.

2.8 The status bands

Turn on *status bands* (F12, under Terminal) and the console stops being the whole screen: a band at the top and a band at the bottom stay still while the text scrolls between them. Each band is one row high: the bands are on or off, and that is the only setting.

The bands are JIM's, the terminal's, the way tmux's status line is tmux's: the machine does not draw in them, and nothing a program prints or pokes into the console's screen can reach them. **The top band** names what is on the screen at its left and has the clock and the date at its right, in whichever formats the Terminal page is set to. The first name is what is running: K/OS at the prompt, EhBASIC INVADER2.BAS once a program is loaded, EhBASIC PROG.BAS > VI EDITTMP.BAS while *VI has it, and BBC BASIC > VI EDITTMP.BBC from the Tube — a trail of who started whom, with the file each one has open, LOGO's .LGO, BOOK's page and CP/M or

the Linux prompt on the Tube included. While the second screen (below) is up it says *Terminal*. **The bottom band** has, at its right, the network the computer has — LAN for a cable, Wi-Fi and its signal's quality, offline — and, on a laptop, the battery's charge, with an arrow up while it charges and down while it does not — and, with *Battery time* on (F12, under *Terminal*), the time it has left after it, 77%↓ (7:16), or while it charges the time to full; at its left whatever a program last asked JIM to put there, and, while somebody types into the machine from another computer (Chapter 11), the keys they send.

A program asks for that line the way it asks JIM for anything, in the stream it prints: `ESC ] 4510 ; note ; text BEL` (an empty text takes it away again). A program that wants the bands whole can still claim them from VICKY — she lays the screen out — and write into the bands' own memory; JIM keeps out until it hands them back. BANDS in `/SYSTEM/BIN` is the demonstration, and Appendix A has the registers (`$D0B0–$D0BF`). The CPU clock is not in the bands: what it changes is how soon a long calculation ends, which the calculation shows better than a number would. INFO has it. Programs that take the whole screen — the games, the editors — get the whole screen anyway: the bands are part of the console, and the console is what a program leaves behind when it asks for the glass.

2.9 Two screens

JIM can show either of two screens. The first is the machine's: K/OS, and whatever it is running. The second is a terminal of JIM's own, on a session on the Linux beneath the machine — or, through `ssh`, anywhere else — that the emulator keeps open whatever the machine is doing:

TERMINAL

opens it the first time and shows it after that, and so does **Alt+2**; **Alt+1** is the machine again. F12, under *Terminal*, has both. Neither screen stops when the other is up: a program on the machine goes on running, and its graphics and sprites — which are VICKY's, not JIM's — stay on the glass above the terminal's text. The machine is single-tasking and stays so; the terminal is where the multi-tasking is, on the far side.

What the terminal runs is the first line of `/SYSTEM/ETC/TERMINAL.CFG` that is not a comment; with none, a login shell on the Linux beneath. One line makes it a `tmux` on a server, the same session every time, and so as many terminals as you like behind the one screen:

```
k4510-connect you@server tmux new -A -s k4510
```

`k4510-connect` is `mosh` when it can be and `ssh` when it cannot. Over a phone's hotspot `mosh` is the one to have: what you type shows at once instead of after the round trip, and the session goes on through a network that drops or changes under it. The server needs `mosh-server` and its UDP ports 60000–61000 open; without them the line falls back to `ssh` by itself. `mosh` keeps no scrollback (`tmux` does) and does not carry JIM's pictures — for those, write `ssh -t` instead. Section 11.5 has the rest.

`tmux`'s own keys then move between its windows, and one line in that server's `~/.tmux.conf` makes its prefix and `K` come back to `K/OS`, so the machine is one more window in the list:

```
bind K run-shell "printf '\033]4510;kos\007' > #{client_tty}"
```

That is JIM's own sequence, `ESC] 4510 ; kos BEL (; term goes the other way), and any program on either screen may send it. The session follows the machine's MODE: a change of size reaches it as a terminal's resize. When it ends, Enter starts another. A machine with Linux locked off (Chapter 11) has no second screen: it is a way into Linux.`

2.10 The network

A URL is a file name. Anything the shell reads — `TYPE`, `LOAD`, `CP`, `RUN`, and the bare-word rule above — accepts `http://` or `https://` in place of a name, and the machine fetches it:

```
TYPE https://raw.githubusercontent.com/mlongval/k4510/master/README.md
CP http://example.org/game.prg game.prg
RUN http://example.org/game.prg
```

EhBASIC's LOAD and BBC BASIC's LOAD on the Tube take URLs the same way. The idea is the C64's Meatloaf cartridge, where a disk name could be an address on the internet; here it costs the ROM nothing, because the ROM never looks at a name — the host does the fetching. A typed line is 95 characters, and so is the longest URL.

Some URLs are more than a file: they are a place. TNFS is the little file protocol the FujiNet and Meatloaf servers speak, and ftp:// and sftp:// are what the rest of the world does; the machine's current directory can be on any of them:

```
CD tnfs://fujinet.online/  
DIR  
CD CBM  
CD -
```

DIR lists the server, a bare name loads from it (so RUN, and the bareword rule, run programs straight off the internet), CD .. climbs, CD - comes home. The prompt shows where you are. A name and password go in the URL the ordinary way (sftp://me:secret@host/dir), and the FujiNet spelling — N:TNFS://... — is understood too.

Or give the place a name on the disk, and it stays there:

```
MOUNT tnfs://fujinet.online/ /MNT/FUJI  
DIR /MNT/FUJI  
MOUNT                list what is mounted  
UMOUNT /MNT/FUJI
```

Everything that reads a directory — DIR, CD, TYPE, RANGER, a program started by name — then works there as on the machine's own disk. A mounted place is read-only.

A zip file mounts the same way — one on the disk, one inside another mount, or one on the internet (a URL ending in .zip):

```
MOUNT GAMES.ZIP /MNT/GAMES  
CD /MNT/GAMES  
DIR  
UMOUNT /MNT/GAMES
```

Its folders are folders and its files open like any others, so a game

shipped with its data as one zip runs from where it is mounted. The zip itself is never changed: copy a file out of it to change it. The whole zip is read when it is mounted, so a zip changed on the disk afterwards needs mounting again. A zip the machine cannot read safely is refused, not guessed at: one with a name that would climb out of its folder (`../`), an encrypted file, or a zip too big for the old format (zip64).

For programs that want a live connection there is the *N: device* at `$D900` (FujiNet's name for it): four channels, each a URL opened for reading and writing — `tcp://host:port` for a connection, `http://` for a page. TELNET host port is the demonstration, a `telnet.prg` in `/SYSTEM/BIN`: what you type goes out, what arrives is drawn by JIM, the terminal (Chapter 5), so a BBS gets its ANSI colours and CP437 art and the cursor and function keys go out as VT sequences; F12 hangs up (Escape belongs to the far end). It offers the far end the terminal types *xterm-color*, *VT220*, *VT100* and *ANSI*, and a Unix host that takes the first gets UTF-8 as well. The register map is in Chapter 13 and `core/net.h`.

So that there is no guessing, this is the whole list of what the machine speaks:

Scheme	What
<code>http://</code> , <code>https://</code>	fetch a file; a name anywhere a name goes; MOUNT shows one as a folder
<code>tnfs://</code>	a directory on a TNFS server
<code>ftp://</code> , <code>sftp://</code>	a directory on a server; sftp does not check the server's key
<code>tcp://</code>	a raw connection, through the N: device only

An *ssh session* is SSH (Chapter 11).

2.11 CAPSLOCK

CAPSLOCK toggles a caps-lock: while it is on, letters read from the keyboard come up uppercase, so a language whose keywords are uppercase — BBC BASIC, EhBASIC — needs no Shift held. Digits and symbols are untouched, so it is a caps lock and not a shift lock, and Shift gives you the *other* case while it is on. It is also suspended while a program is running, so a program that wants lower case gets it, and the

lock comes back when the program does. CAPSLOCK ON and CAPSLOCK OFF set it explicitly; *CAPSLOCK works from inside a BASIC.

2.12 Scripts and the boot file

EXEC name runs a text file as shell commands, one per line. A line whose first character is # is a comment, and a blank line is ignored. At power-on the machine runs /STARTUP.BAT as such a script, if it exists — straight in, with no pause and nothing to catch. It is the place to put your aliases; /SYSTEM/ETC/STARTUP.SAMPLE is one to copy from. If one ever stops the machine booting, page 33 is the way out.

A .BAT boots the machine and an .RX automates it: when a script needs to make a decision, read a result or talk to two things at once, that is RX's job (Chapter 10) and EXEC does not try to compete.

2.13 The monitor

MON (old-timers may type W0Z) opens the machine monitor at a * prompt; its grammar is Wozmon's, with 28-bit addresses:

```
*FF80.FF8F      examine a range
*0300:A9 41 60   store bytes
*0300R          run from $0300
*X              leave (or Q, or EXIT)
```

Anything else typed at the * prompt is a shell command, run on a clean machine and returned from. FILL from.to value and COPY from.to dest work on memory, by DMA, anywhere in the 256 MB.

The monitor is a program, MONITOR, and it loads at \$E000, in the RAM under the ROM, so that the memory it is there to look at — \$0800 to \$CFFF — is exactly as the last program left it. Addresses below \$10000 read as a program sees them: \$A000 to \$FEFF is that RAM, not the ROM, and the ROM shows only in its always-there page at \$FF00.

SUPERMON in /SYSTEM/BIN is the larger tool, in the tradition of Jim Butterfield's: M memory, D a disassembly that speaks the whole 45GS02, A an assembler, registers, hunting and filling — and @ escapes to the shell without leaving it.

```

K4510 Fantasy Computer - K/OS
CPU: 45GS10 RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM
/HOME] mon
monitor: addr addr.addr addr:b b b addrR (28-bit hex) X leaves
*ff80.ff8f
0000ff80: 4C 00 FF 4C 09 FF 4C 12 FF 4C 1B FF 4C 24 FF 4C
*

```

The monitor examining the system-call jump table. MON FF80.FF8F runs one line without entering the prompt.

2.14 Colours and palettes

COLOR fg bg sets the text colours (palette indices, in hex), and PALETTE the 256 colours behind them. The machine's own is the C64's sixteen, yellow on blue; /SYSTEM/ETC/PALETTES holds others:

PALETTE LOAD GREEN	a phosphor-green tube
PALETTE LOAD AMBER	an amber one
PALETTE RESET	the machine's own back

The palette last loaded is kept, in k4510.cfg: the machine comes back in it at power-on and after a reset, until PALETTE RESET.

AMBER, GREEN and GREY are ramps — black and fifteen shades of one colour — so each carries a COLOR line that sets the text to a bright shade on black. Colour 1 in each is the brightest shade, because the shell draws

its highlights in colour 1 (directories in DIR, the banner) and they must stay brighter than the text. C64 is the machine's own sixteen, as it boots them, and PEPT0 is the same sixteen as Philip Timmermann measured them off a real C64 — duller, warmer, and what one looked like on a television. CLEAR is the sixteen made readable for colour-blind eyes (protan, red-blind, most of all): every colour a program writes text in reads at 4.5:1 or better on a slightly darker blue, red goes pink, so it no longer passes for a muddy brown, and grey sits lighter than it. A palette file is plain text — an index and three hex bytes to a line, # for a comment — and names only the entries it changes; PALETTE SAVE name writes one of yours. PALETTE LOAD in /STARTUP.BAT makes it the machine's. F12 → Video → Palette lists them all and types the PALETTE LOAD for you.

2.15 Housekeeping

INFO is the machine's self-description; TIME the clock; MODE sets the text screen — CLS clears it and CLG clears the bitmap over it, whoever drew it — and both reach a BASIC through the * escape, which is how you tidy up after a demo that left its picture behind. HUSH silences the sound, whichever part of the machine is making it. BANNER clears the screen and prints the power-on banner again, which is a tidy way to end a session or start a screenshot. RESET restarts the machine from the shell, the same cold start the reset chord performs.

IDEA is a screenshot in words: a thought about how something could be better, kept before it goes. It takes no text. Typing IDEA writes a new brainshot to /SYSTEM/BRAINSHOTS and opens VI on it, and the thought goes in there — a line, or a page. The machine adds what it knew at that moment — the time, the directory, what was running, the screen — so an idea found a week later still says what it was about. From a BASIC it is *IDEA, and it loads nothing over the program. The files are plain text; TYPE and VI read them, and so does the person you asked to build the idea (Chapter 11).

It did once take the idea on the line itself, and that is worth knowing because it is the sort of convenience that looks free and is not. The text was typed into the shell's command line, which is 96 bytes in the ROM's own RAM, so anything longer than ninety characters was cut — and cut

at eighty-four from a BASIC, where a star command travels through that same buffer with a wrapper around it. Worse, it was cut in the middle of a word and said nothing about it, which is precisely the wrong behaviour for a thing whose whole purpose is catching a thought before it escapes. Making the line longer would mean taking RAM from the ROM's C stack, which has form for breaking the shell when it runs short. VI holds 256 characters to a line and as many lines as you care to type, so the longer road is the only honest one.

SETUP measures the computer the machine is running on, with the sound and the picture really running, and keeps the fastest clock it holds without a gap — for that computer, so a later boot there pays nothing. It also times how many pixels that computer can draw a frame with half of it to spare (`tools/k4510-vidcap`, run beside the machine) and keeps that as the pixel cap: from the next start, the panel's resolutions above it are not offered. BENCH measures it again whenever you like: frames a second and gaps in the sound at every clock of the menu's ladder, about 25 seconds, the report in `/SYSTEM/LOG/BENCH-NN.TXT`. A clock is right for a host when it holds 60 frames a second with no gaps.

MODE on its own says where you are; MODE *n* moves. The text always starts in the top-left cell: there is no margin. Moving clears the screen — the old text was laid out for the old shape — and the banner is printed again at the next prompt, so the machine says what shape it is now in rather than leaving you at a bare cursor.

MODE	pixels	text
0	640×480	80×30
		8×16 cells; the machine starts here
1	640×240	80×30
2	320×240	40×30

The F12 menu, under Video, has the same two knobs: *Resolution* shows what the machine is actually in — it follows a MODE you type — and choosing another asks the ROM to perform it, because the console's geometry belongs to the ROM and not to the host. VICKY can draw smaller fields than these, but they are for a program that wants the pixels, not for a shell; a program asks for them through the video registers (Chapter 13).

The picture is always painted into 640×480: a 240-line mode has each line drawn twice, and a 320-wide one has its pixels doubled side-

ways. Raster lines and SHEILA's display list count the glass, 0–479, not the mode — which matters the moment a program asks VICKY for a 200-line field, where the first line of the picture is raster line 40.

2.16 When **STARTUP.BAT** is the problem

/STARTUP.BAT runs at power-on, and a bad one runs at every power-on. There is no moment to catch — the machine boots straight into it — so the way out is not a key held at the right instant but a switch that stays where you put it: the F12 menu, under Shell: *Run STARTUP.BAT*. That setting is the host's, kept in k4510.cfg, so it survives a power cycle and no wedged program in the machine can take it away from you. Boot clean, fix the file, switch it back on.

There is a second way, for one boot only:

```
./k4510 --no-startup.bat
```

(--no-startup does the same, as does K4510_NO_STARTUP=1 in the environment). It skips /STARTUP.BAT for that run and touches nothing: the F12 setting and k4510.cfg are left exactly as they were.

2.17 When something goes wrong

Type DUMP ON, make it go wrong again, then type BUG and answer its questions: the report it writes is the issue. Appendix B is the whole of it, on two pages.

Chapter 3

Every Command

This chapter is the whole vocabulary of the prompt, one line each, and it is not written by hand: `doc/guide/mkref.py` reads the shell's own command table out of the ROM's source, the programs out of `/SYSTEM/BIN` and the languages out of `/LANG` at the moment this book is built. A command the machine has and this list does not describe stops the build, and so does a description of a command that has gone — so what is below is what the machine you are reading about answers to. Chapter 2 is how the shell behaves; this is what you can say to it.

Each entry says where the word lives, which matters more than it looks:

ROM

in the resident part of the operating system, always there.

ROM, bank *n*

in one of the sideways banks (Chapter 12), paged in for the moment it runs.

runs MONITOR, runs TYPE

a word the ROM still answers to, which hands its line to a program on the disk. The work moved out of the ROM in September 2026 to make room; the word did not change.

/SYSTEM/BIN

a program. Programs run by name from anywhere, and take their arguments the same way the ROM's words do.

/LANG/name

a language: its folder, with its examples.

ROM, on the Linux

a word the ROM answers to by starting something on the Linux the machine runs on (Chapter 11).

Every word below also works from a BASIC, Forth or the monitor with a * in front, and a bare name that is not here is tried as a program, then as a CP/M program if F12 allows it, then as an alias, then as an RX script — in that order, so an alias never hides a command. Capitals are the convention; the shell does not mind either way.

Files and directories

- CD [dir | .. | - | url]** *ROM, bank 3*
 Change directory; the rest of the line is the name, spaces and all. Alone, to /. A tnfs:// or sftp:// URL goes there; - comes home. Also CHDIR.
- CP [-f] from to** *ROM*
 Copy a file (a URL may be the source). Refuses to overwrite without -f. Not COPY, which is memory.
- DIR [-a] [-l] [dir | pattern]** *ROM*
 List a directory. -a (-all) shows the hidden (dot) names, -l (-long) one to a line; a pattern matches here, * any run and ? any one character. Also LS.
- LOAD name [addr]** *ROM*
 Load a file into memory, at addr or at the address in its header.
- MKDIR dir** *ROM, bank 1*
 Make a directory.
- MOUNT [url path]** *ROM, bank 3*
 Alone, say where the machine's own disk really is – the system in RAM, what you save on which partition – and then list your mounts; otherwise show a tnfs://, http:// or https:// URL at a path of the disk.
- RENAME [-f] old new** *ROM*
 Rename or move a file. Refuses to overwrite without -f. Also REN, MV.
- RM [-f] name** *ROM*
 Move a file to /.TRASH, where DELETE -r gets it back. -f really removes it. Also ERASE, DEL.
- RMDIR dir** *ROM, bank 1*
 Remove an empty directory.
- SAVE name from.to** *ROM, bank 1*
 Write memory from.to to a file.
- UMOUNT path** *ROM, bank 3*
 Take a mount away. Also UNMOUNT.

XD name*ROM, bank 1*

A file as a hex dump. Esc stops it. Also HEX.

Running things

ALIAS [name [text]]*ROM, bank 2*

List, define, or (name alone) remove an alias. Aliases are tried last, so none can hide a real command.

ECHO text*ROM*

Print the text.

EXEC name*ROM*

Run a text file as shell commands, one a line; # starts a comment. /STARTUP. BAT is run this way at power-on.

HELP*runs TYPE*

The command summary: TYPE /SYSTEM/ETC/HELP.

RUN [name | addr]*ROM*

Run a program; a bare name does the same. RUN addr jumps there.

SWAP [-k] command*ROM*

Put the whole 64 KB and the screen away, run the command on a clean machine, and give them back. -k (-keep) keeps the screen the command left.

The screen

BANNER*ROM*

Clear the screen and print the power-on banner again.

CAPSLock [ON | OFF]*ROM, bank 1*

Toggle the caps lock: letters come up uppercase. Suspended while a program runs. Also CAPS.

CLG*ROM, bank 1*

Clear the bitmap over the text, whoever drew it.

CLS*ROM*

Clear the text screen.

COLOR fg [bg] [!]*ROM, bank 1*

The text colours, as palette indices in hex. A pair the palette makes hard to read is refused, with one that reads suggested; ! has it anyway. Also COLOUR.

MODE [-l] [-s N | WxH | n] [-d | -n]*ROM, bank 3*

Alone, say the mode. -l (-list): this panel's integer display resolutions, each with its text grids. -s N (-scale=N), or its size: the one at scale N. -d (-double): twice the rows, in smaller letters; -n (-normal) the larger again. 0 640x480, 1 640x240, 2 320x240; 5, 6, 7 are scale 1, 2, 4.

PALETTE [LOAD name | SAVE name | RESET | i rr gg bb]*ROM, bank 2*

The 256 colours: list them, set one, load a .PAL from /SYSTEM/ETC/PALETTES, save them, or put the machine's own back.

The machine

DUMP [note | ON | OFF]*ROM, bank 1*

Write the machine's whole state to dumps/ on the host. ON writes one every fifteen seconds.

HUSH*ROM, bank 1*

Silence the OPL2 and the sound sequencer.

IDEA*ROM*

A brainshot: opens VI on a new one in /SYSTEM/BRAINSHOTS, with the machine as it was. No text argument: the shell line would cut it. *IDEA from a BASIC.

INFO [-vcmgst]*ROM, bank 1*

The machine's self-description, the clock in force included; with options only those parts: -v (-version, the exact build), -c (-cpu), -m (-memory), -g (-graphics), -s (-sound), -f (-files), -t (-time), -a (-all).

RESET*ROM*

Cold-start the machine, as the reset chord does.

TIME*ROM, bank 1*

The date and the time.

Memory

COPY from.to dest*runs MONITOR*

Copy memory, by DMA. Not files: that is CP.

FILL from.to value	<i>runs MONITOR</i>
Fill memory with a byte, by DMA.	
MON [line]	<i>runs MONITOR</i>
The machine monitor, Wozmon's grammar with 28-bit addresses. With a line, runs it and returns. Also WOZ.	

Languages

BBC	<i>ROM</i>
BBC BASIC on the Tube (Chapter 5). Also BBC.	
EBASIC	<i>/LANG/EHBASIC</i>
Enhanced BASIC with the machine's graphics (Chapter 4).	
FORTH	<i>/LANG/FORTH</i>
Tali Forth 2 (Chapter 6).	
LOGO	<i>/LANG/LOGO</i>
Turtle graphics (Chapter 7).	
RX name	<i>/LANG/RX</i>
Run a REXX script (Chapter 10); a bare HELLO runs HELLO.RX.	

The Linux beneath

!command or !	<i>ROM, on the Linux</i>
Run a command on the Linux beneath, or (alone) open a shell there. Can be locked off.	
CC name	<i>ROM, on the Linux</i>
Compile name.C here with cc65 into name.prg.	
PAS name	<i>ROM, on the Linux</i>
Compile name.PAS here with Mad Pascal into name.prg.	
SSH [user@]host	<i>ROM, on the Linux</i>
An ssh session, through the Linux's ssh. Locked off with !.	
TERMINAL	<i>ROM</i>
Show JIM's second screen: a terminal session on the Linux beneath, or wherever /SYSTEM/ETC/TERMINAL.CFG says (an ssh to a tmux, say), running whatever the machine does. Alt+1 comes back to K/OS, Alt+2 goes there again. Locked off with !@sec:screens@.	

Programs in /SYSTEM/BIN

BANDS /SYSTEM/BIN

A program writing to the bottom status band.

BENCH /SYSTEM/BIN

About 25 s: frames per second and sound gaps at every clock step, to /SYSTEM/LOG/BENCH-NN.TXT.

BOOK [n | word | page] /SYSTEM/BIN

This handbook, on the machine: the contents, chapter n, or the chapter whose title holds the word. Tab chooses a link, Enter follows it, Backspace comes back.

BUG /SYSTEM/BIN

Asks seven questions about a fault and writes the report (Appendix B).

CHROUT /SYSTEM/BIN

How fast the ROM's console prints.

CODEPAGE [437 | K4510] /SYSTEM/BIN

Alone, say which code page the machine speaks. 437 is IBM's, the default; K4510 gives 26 of its Greek and maths places to Western Europe's capitals, oe, the euro and German quotes. Remembered, as F12 -> Terminal -> Code page.

DELETE [-l | -r name | -e | name] /SYSTEM/BIN

The trash: list it (-l, -list), put a file back (-r, -restore), empty it (-e, -empty), or send a file there.

EDIT [-s] [-u] [-v] [name] /SYSTEM/BIN

The editor, in MS-DOS EDIT's manner: menus, the mouse, dialogs; -s (-system) in the console's colours, -u (-upper) BBC BASIC's keywords in capitals at each save, -v (-vi) VI's keys (Chapter 9).

FONTED [name.FNT] | -l name /SYSTEM/BIN

The font, edited where it lives, both sizes, every edit on the screen at once. -l (-load) loads a .FNT and leaves, for STARTUP.BAT.

HEXED name | \$address /SYSTEM/BIN

A hex editor: a file, loaded whole (8 MB at most), or memory by 28-bit address, changed live. Hex on the left, the code page on the right; go to, find, save, undo, the mouse.

KEYTEST /SYSTEM/BIN

Asks for every key and checks what arrives.

KOMMANDER /SYSTEM/BIN

The two-panel file manager.

MARK [MHz] /SYSTEM/BIN

The benchmark, for setting this machine beside others: at every clock from 10 to 60 MHz (or the one nearest the number given) a loop of known length (the work of a 6502 at so many MHz), the Byte Sieve, a copy, and a fixed-point Mandelbrot twice — its multiplies by shift and add, then by the MATH unit. Beside each time is the clock a real 65C02 would need to match it (an Apple IIe is 1 MHz, a BBC Master 2), from the loops' cycles counted on a cycle-exact simulator. Figures are machine time; the last column says whether this host kept up. It first checks that the CPU computes correctly. The report is kept in /SYSTEM/LOG/MARK.TXT.

MONITOR /SYSTEM/BIN

The monitor as a program: MON, WOZ, FILL and COPY run it.

MOUSETEST /SYSTEM/BIN

The mouse registers, live, with a sprite pointer.

NVIM [name] /SYSTEM/BIN

Neovim, on the Linux beneath, set up for the machine: its colours, its languages, F9 to compile and F10 to run (Chapter 11).

PADTEST /SYSTEM/BIN

The held-keys register, live: a gamepad's first test.

PETSCII /SYSTEM/BIN

JIM, the terminal, speaking PETSCII.

PROG [-s] [-v] [name] /SYSTEM/BIN

The programmer's front end, in EDIT's manner: edit a C or Pascal program, compile it with F9, run it with Ctrl+F9, the compiler's messages under the text; -s (-system) the console's colours, -v (-vi) VI's keys (Chapter 9).

RANGER /SYSTEM/BIN

The miller-column file manager.

SAY text /SYSTEM/BIN

Prints its arguments: the smallest program there is.

SETUP /SYSTEM/BIN

Measures this host thoroughly and keeps the clock it settles on, and the pixels it can draw a frame.

SPLIT /SYSTEM/BIN

A split screen held by SHEILA: blitter lines above, four rows of text below, and how many lines a second.

STATUS /SYSTEM/BIN

The whole machine at a glance: its display and clock, how much of the 256 MB holds anything, the memory of the Linux beneath, where the system and your files really are and how much room is left, what is mounted, and the network by name and address.

SUPERMON /SYSTEM/BIN

Jim Butterfield's monitor, grown up: an assembler and a 45GS02 disassembler.

TELNET host [port] /SYSTEM/BIN

A terminal on a TCP connection. F12 hangs up.

TYPE name /SYSTEM/BIN

A file, a screenful at a time; Esc or Q stops. A URL works.

VI [name] /SYSTEM/BIN

The modal editor (Chapter 9).

WALL /SYSTEM/BIN

What was sent to this machine from outside (tools/k4510-remote wall, on another computer): each message waiting, oldest first, answered as it asks — a key for a notice, a digit for one of its choices, a line of text — and the answer goes back. Esc leaves a message for later. When the prompt is idle the sender types WALL for you.

WORD [-s] [name] /SYSTEM/BIN

-s (-system): the console's colours. A reader for Microsoft Word's .DOCX, in EDIT's manner: the document laid out to the window, emphasis in colour, lists and tables; Save As Text@sec:word@.

Chapter 4

EhBASIC

The machine has two BASICs, and this is the one with the machine in it: EhBASIC 2.22 — Lee Davison's Enhanced BASIC — with this machine's additions: graphics statements that ride the blitter, floating point on the MATH unit, and an escape hatch to the shell. It has no sound *keyword*: a program that wants a note pokes the sound sequencer at \$D5E0 or the OPL2 at \$D480 directly, which on a machine with a friendly memory map is not much of a hardship (Chapter 13).

```
RUN EHBASIC
```

It comes up ready: the banner, 46335 Bytes free — some 7 KB more than a C64's 38911 — and a Ready prompt — the traditional Memory size ? question is gone; the machine knows.

4.1 Ten minutes of it

```
RUN "DEMOS.BAS"
```

The menu chains: RUN "name" (or LOAD inside a running program) loads another BASIC program and runs it — that is how one BASIC program hands over to another, exactly as on a C64.

4.2 The machine's own statements

```
GRAPHICS 2          640x480 bitmap over the text
PLOT X,Y,C          LINE X1,Y1,X2,Y2,C
TRI X1,Y1,X2,Y2,X3,Y3,C
PALETTE I,R,G,B     GCLS          GRAPHICS 0
```

```

K4510  EHBASIC DEMOS

1  LINES      SPINNING LINES (BLITTER LINE OP)
2  TRIS       300 RANDOM TRIANGLES
3  SINE       SINE WAVES (MATH UNIT)
4  STARS      A STARFIELD (ANY KEY LEAVES)
5  README     HOW TO USE EHBASIC HERE
I  INVADERS   SPACE INVADERS ON THE TEXT SCREEN
J  INVADER2   SPACE INVADERS WITH HARDWARE SPRITES
G  GRAPH2D    PLOT Y=F(X), PAN AND ZOOM
H  GRAPH3D    A SURFACE IN WIREFRAME, ROTATE IT
B  BENCHMARKS MENU
*  K/OS COMMANDS: *DIR, *CD /LANG/EHBASIC, *TYPE NAME, *INFO ... (@ WORKS TOO)
Q  QUIT TO READY

CHOICE?

```

The demo menu. A key picks an entry; each demo returns here when it ends.

Colours 0–15 belong to the text screen; demos use 16 and up. GRAPHICS 0 puts the text mode and its palette back, whatever the program changed.

4.3 Hardware sprites

The 128 sprites VICKY carries are reachable from BASIC, which is unusual enough to be worth saying plainly: this is not a bitmap being redrawn, it is the video chip carrying the picture for you.

```

SPRDEF n,page,w,h,bpp    give sprite n a shape
SPRITE n,x,y             put it there and show it
SPROFF n                 hide it

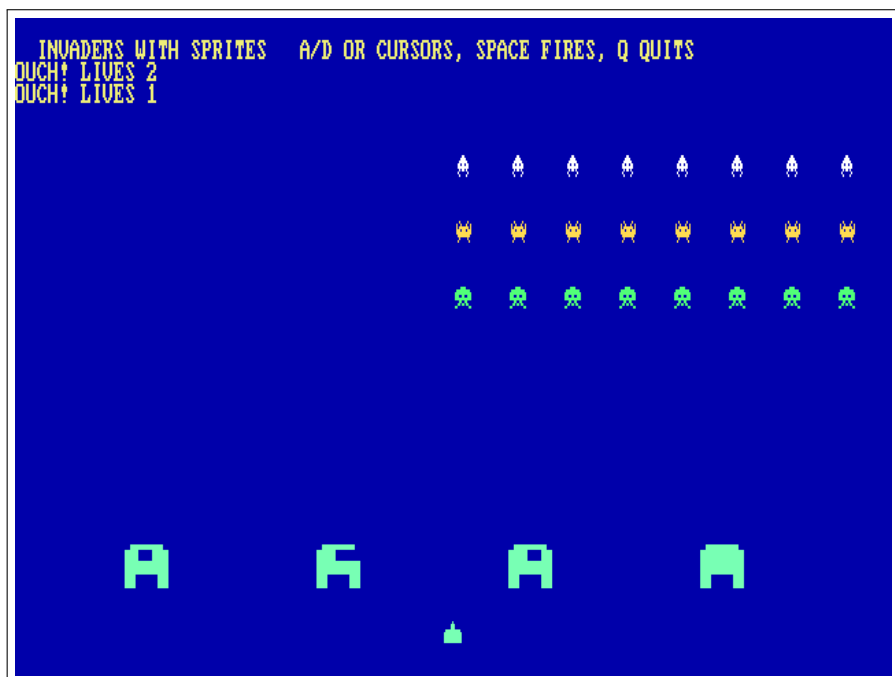
```

SPRDEF says where the pixels are and what shape they make: *page* is the 256-byte page the data starts at, which is how a 16-bit BASIC

number reaches into 256 MB (page*256 is the address); *w* and *h* are 8, 16, 32 or 64; *bpp* is 4 or 8. `SPRITE` positions the sprite and turns it on, `SPROFF` hides it. Colour 0 is transparent, and there is no per-line limit — all 128 may be on the same raster line.

Each statement re-points the chip at the attribute table and re-enables sprites, so a `GRAPHICS 0` or a shell `MODE` in between costs you nothing: the next sprite statement puts them back.

`INVADER2.BAS` in `/LANG/EHBASIC/EX` is the demonstration — Space Invaders with the arcade shapes, poked 4 bits-per-pixel into \$040000 through a bank register, four bases that erode as they are hit, and thirty-one sprites moving. `INVADERS.BAS` beside it is the same game on the text screen, which is a fair way to see what the sprites buy you.



`INVADER2.BAS`: twenty-four invaders, four bases and a cannon, every one of them a hardware sprite. The bases erode because their shape pages are poked, not redrawn.

4.4 The * escape

Any shell command works from BASIC with * in front — *DIR, *CD EHBASIC, *MON, *DUMP a note — and *BYE leaves BASIC for the shell. Escape or Ctrl-C stops a running program (and silences the sound); the program's variables survive for PRINT-style post-mortems.

4.5 Editing the program in VI

*VI and *EDIT, with nothing after them, edit the program that is in memory:

```
10 PRINT "HELLO"  
*VI
```

BASIC writes the program to a temp file, opens the editor on it, and reads it back when you leave — so what you type in the editor is what you LIST afterwards. Give either one a file name and nothing special happens: *VI notes.txt is the ordinary * escape, and the editor edits that file.

The trip out and back is not free. The editors load where the interpreter lives, so the shell command underneath is SWAP, which puts all 64 KB and the screen aside first and restores them afterwards. And because the program is written out and read back, *variables do not survive* — this is an immediate-mode thing, like LOAD. The temp file (EDITTMP.BAS) is left behind, which is occasionally a useful accident.

Why the first line is blank: SAVE starts its output with a newline, so the editor opens on an empty line 1 with the program below it. Harmless — BASIC ignores it on the way back in — but it is why the line count is one more than you expect.

Chapter 5

The Tube

The BBC Micro's most elegant idea was the Tube: a fast port through which a *second processor* — another CPU with its own memory — could take over the computation while the Beeb kept the keyboard, the screen and the discs. The K4510 has a Tube of its own at \$D800, and the first thing fitted to it was Richard Russell's BBC BASIC, running on the host machine with a flat 256 MB of its own. Others have followed: CP/M on a Z80, the host's own shell behind !, and a chess engine.

BBC

You get the > prompt of a BASIC with real power behind it, and it is *fast* — the co-processor is not cycle-matched to 1985:

```
HIMEM=PAGE+250*1024*1024
DIM space 200*1024*1024
```

both just work. Type *QUIT to hand the console back to the shell.

5.1 Graphics and sound

The console cannot show what it cannot see, so the Tube speaks up: MODE, GCOL, PLOT, MOVE, DRAW, CIRCLE and the palette arrive at the machine as escape sequences, and the Tube's gatekeeper (the *Tube ULA*) executes them on the VICKY blitter. SOUND and ENVELOPE-less music go the same way, onto the machine's four-channel sound sequencer and out through the OPL2 — the classic Beeb SOUND chan,amp,pitch,dur, queued per channel like the original. Channel 0, which was the Beeb's noise, is a feedback-heavy FM patch; 1 to 3 are plain two-operator tones. The patch is written again on every note, so a program that has zeroed the chip for its own purposes does not silence the sequencer.

```

CD /LANG/BBCBASIC
BBC
LOAD "EX/KALEID.BBC"
RUN

```



KALEID.BBC: MODE 2 kaleidoscope, drawn by BBC BASIC on the Tube, painted by VICKY.

The /LANG/BBCBASIC/EX directory carries a period programme selection: KALEID, CIRCLES, ROSES, MOUNTAIN, BOUNCE, CLOCK (a live analogue clock face) and TUNE — Frère Jacques as a three-voice round, the sound sequencer's party piece.

5.2 Sprites

VICKY has 128 hardware sprites, and BBC BASIC reaches them the way RISC OS reached its own: VDU 23,27 selects and shapes, PLOT &ED places.

There is no sprite file. A sprite is *captured* from the bitmap after BBC BASIC has drawn it with the words it already knows:

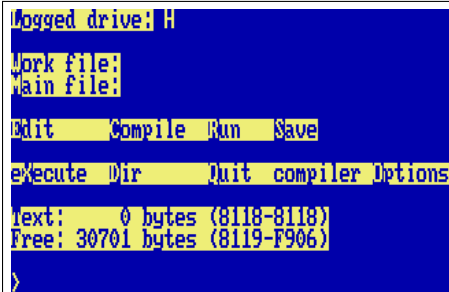
```
MODE 2
GCOL 0,1:CIRCLE FILL 40,40,30
MOVE 8,6:VDU 23,27,1,0,32,32|
CLG
PLOT &ED,600,500
```

VDU 23,27,0,n	select sprite n (0–127) for PLOT
VDU 23,27,1,n,w,h	capture n , $w \times h$ pixels (8/16/32/64), bottom-left at the graphics cursor
VDU 23,27,2,n	hide n
VDU 23,27,3,n,f	flip: bit 0 horizontal, bit 1 vertical
VDU 23,27,4,n,z	depth: drawn after layer z (default 1, over the bitmap)
VDU 23,27,5,n,m	sprite n shows sprite m 's picture
PLOT &ED,x,y	show the selected sprite, bottom-left at x, y

A placed sprite is a register write: moving sixty-four of them every frame costs the interpreter nothing but the PLOTs. `SPRITES.BBC`, `INVADERS.BBC` and `TUNNEL.BBC` beside them are the demonstrations (the last one moves nothing at all — it cycles the palette with VDU 19).

5.3 The terminal: JIM

The console you see when the Tube is running is not the ROM's own terminal but *JIM*, at \$DA00 — the Beeb's third I/O page, given a job here. JIM is a VT100 with the ANSI colours and the VT220 editing sequences (insert and delete character and line, erase character, scroll regions, origin mode, DEC line drawing, cursor-position and identity reports), built into the machine the way a real 8-bit computer got a serious terminal: as a card, not a program. It draws on the same text screen as the ROM console, inside the window the ROM gives it, so the two share one screen and one cursor. Anything that needs a terminal writes its byte stream to \$DA00 and reads its answers from \$DA02; keys pushed through \$DA03 come back as the VT sequences the far end expects (arrows, Home/End, PgUp/PgDn, F1–F12, Delete as \$7F). The register map is in `core/term.h`.



```
logged drive: H
Work file:
Main file:
Edit  Compile Run  Save
execute Dir  Quit compiler Options
Text: 0 bytes (8118-8118)
Free: 30701 bytes (8119-F906)
>
```

Turbo Pascal 3 (H: user 3) on JIM: the menu bar in reverse video, the compiler's own screen handling on a VT100 that is a chip.

For CP/M this settles the question every program asks at install time: tell WordStar's WSCCHANGE, Turbo Pascal's TINST, ZDE and the rest that the terminal is a *VT100* (or *ANSI*: the same sequences). Turbo Pascal 3 on H: user 3 comes up with its menu already right; WordStar 4 on E: wants one pass of WSCCHANGE. BBC BASIC's console edition speaks the same language natively, so COLOUR, CLS and PRINT TAB(x,y) land where they should. Bytes \$80-\$FF are drawn as CP437 glyphs, which is what BBS ANSI art is made of.

Everything through JIM

JIM is not only for the Tube. The machine's own programs draw through it too: the shell's prompt and its colours, Mad Pascal's CRT unit (GotoXY, TextColor), EDIT, PROG and WORD, RANGER, KOMMANDER, HEXED, CALC, FONTED, the TRACKER and the rest send JIM the byte stream

a terminal reads, instead of writing into the screen's memory. Games and the graphics layers still go to VICKY directly; that is what they are for. Two additions let a K4510 program say everything it needs in that stream:

ESC [? 4510 h — the palette's own colours

ESC[38;5;*n*m and ESC[48;5;*n*m with *n* below 16 are then the palette's entry *n*. Without it they mean xterm's colour *n*, and xterm's order reaches only twelve of the machine's sixteen: no orange, brown, mid grey or light grey. ESC[?4510l puts xterm's meaning back.

ESC [11 m — glyphs for control bytes

The bytes \$00-\$1F and \$7F draw their CP437 glyphs (arrows, card suits, the little house) instead of being swallowed, as on the Linux console; the nine that must still act — backspace, tab, the line feeds, carriage return, SO, SI, Escape — are sent as their Unicode characters in UTF-8 if their glyph is wanted. ESC[10m ends it.

A reset (ESC[!p or ESC c) clears both. C programs get all of this from demo/jimcell.h: `jc_cell(x, y, glyph, fg, bg)` sends a cell, moving the cursor and changing the colours only when it has to. Why it matters: a screen made of a stream can be sent anywhere, and it is what lets JIM own the status bands and keep a second screen, a session on the Linux underneath or on a distant machine (Section 2.9).

5.4 Star commands

A line starting with * goes to the machine: *DIR, *CD, any shell command — and BBC BASIC's own file words (LOAD, SAVE) read and write the machine's filesystem directly, because the co-processor lives inside fs/ too. The Tube starts in the shell's current directory, so CD /LANG/BBCBASIC then BBC lets LOAD "EX/KALEID.BBC" work without spelling the whole path; the machine's filesystem is the co-processor's whole world — fs/ is shown as /, the host tree above it hidden, and *CD .. stops at the root.

*VI or *EDIT with nothing after it edits the program in memory, as in the other BASICs: BBC BASIC lists it as text to EDITTMP.BBC in the current directory, the editor opens on it, and when you leave, LOAD "EDITTMP.BBC"

types itself and reads it back. Variables go the way they go with any LOAD. With a name (*VI NOTES.TXT) it is the ordinary star command. *EDIT, with a name or without, starts EDIT -u: each save puts BBC BASIC's keywords in capitals, so lines may be typed in lower case (Chapter 9).

Edges, honestly: POINT(and TINT are not implemented; GCOL modes 1–4 draw plain; VDU 5 text prints as text.

Chapter 6

Forth

The machine's third language is native: no Tube, no co-processor, just 45GS10 machine code loaded at \$8C00, with the dictionary below it. It is Tali Forth 2 — a public-domain, ANS-flavoured Forth written for the 65C02, which this CPU speaks as a strict superset.

```
FORTH
```

```
2 3 + . 5 ok
: cube dup dup * * ; ok
7 cube . 343 ok
```

6.1 The machine at your fingertips

Forth's oldest habit is poking hardware, and this machine is one large, friendly memory map. C@ and C! reach every register of Chapter 13 directly:

```
hex
D000 C@ .      read a VICKY register
D5E0 80 swap C!  silence the sound sequencer by hand
```

6.2 The assembler

Tali brings an interactive 65C02 assembler and a disassembler, kept in because on this machine they are the point:

```
' dup 10 disasm      disassemble a word of the kernel
assembler-wordlist >order
here 2 lda.# 0 sta.z drop
```

```

K4510 Fantasy Computer - R/OS
CPU: 45GS10  RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME1 forth
Tali Forth 2 on the K4510 (native 45GS10)

Tali Forth 2 for the 65c02
Version 1.1 06. Apr 2024
Copyright 2014-2024 Scot W. Stevenson, Sam Colwell, Patrick Surry
Tali Forth 2 comes with absolutely NO WARRANTY
Type 'bye' to exit
2 3 + . 5 ok
: cube dup dup * * ; ok
7 cube . 343 ok
' dup 10 disasm
37748 36419 jsr 1 STACK DEPTH CHECK
37751 dex
37752 dex
37753 2 lda.zx
37755 0 sta.zx
37757 3 lda.zx
ok

```

A Forth session: the banner, arithmetic, a new word, and the kernel disassembling itself.

DISASM is strictly 65C02: at a 45GS10-only opcode it prints a polite ? and moves on.

6.3 Manners

The dictionary has about 31.7 KB free for your words. BYE returns to the shell with the screen and the session exactly as you left them.

No file words yet: Forth source lives in /LANG/FORTH but must be typed. An INCLUDED that reads through the machine's storage device is on the list.

Chapter 7

LOGO

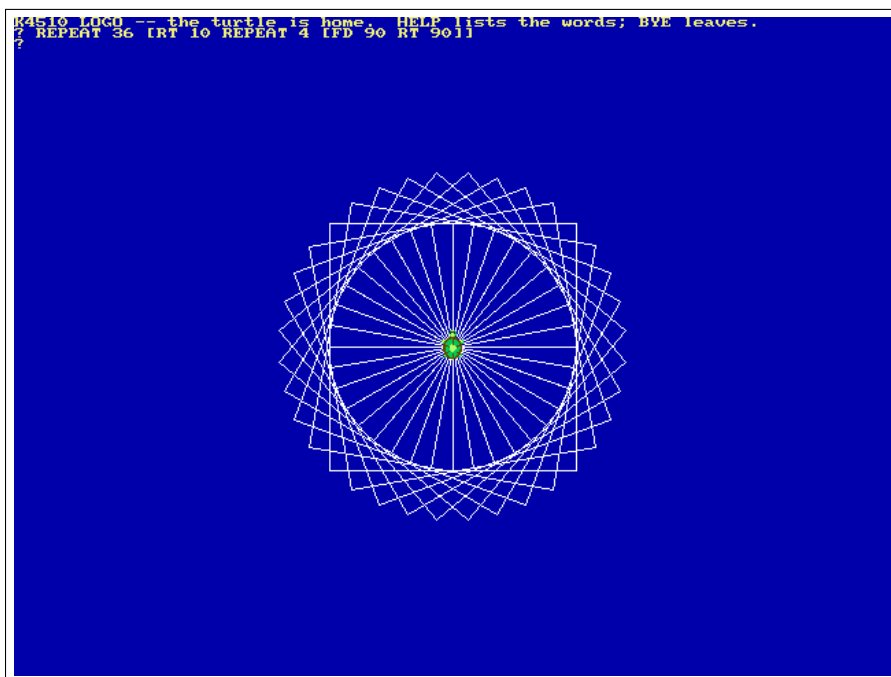
LOGO is the language that taught a generation of children that a computer does what it is told, and it did it with a turtle: a small arrow on the screen that walks forward, turns, and leaves a line behind it. The K4510's is new, written for this machine in September 2026, and native — 45GS10 code, no co-processor.

```
LOGO
FD 100 RT 90
REPEAT 4 [FD 100 RT 90]
```

The picture is VICKY's bitmap *under* the console, which is the classic LOGO screen: the turtle draws below and the prompt lives above, so you can see what you asked for and what you are asking next at once. LOGO draws on the screen it finds — whatever MODE you were in when you typed LOGO, up to the HD 1440×1080 — and leaves it that way when you go. It wants 320×240 at the least: on a smaller screen (the game modes 320×200 and 160×200, which a program may set) it says so and asks whether to move up to 320×240. Answer Y and it moves and runs; answer anything else and it leaves the machine exactly as it was and gives you the prompt back. The turtle itself is a small green turtle, a hardware sprite in sixteen headings — one every 22.5 degrees, drawn once and turned by `tools/mkturtle.py` when the machine is built, so turning it is a matter of pointing the sprite at the right picture.

The turtle is 32 of the machine's pixels whatever the screen, so it looks larger on the coarser ones, where every pixel is drawn twice or four times over, and smaller on the HD screens — as anything else drawn in the machine's pixels does.

It need not be a turtle. `SETSHAPE "BIRD` fetches `/LANG/LOGO/BIRD.SPR` and the bird flies where the turtle walked; `SETSHAPE "TURTLE` brings it back. A shape is sixteen frames of 32×32, one every 22.5 degrees, and `tools/mkturtle.py` draws them — so anyone may add one. Logo's



LOGO: a square repeated round a circle, drawn by the blitter under the console's text.

turtles have taken other shapes since the 1980s: Atari Logo's manual offers "cars, planes, human figures, animals". Every line is the blitter's, and every number is a real one — IEEE floating point, done by the MATH unit, so a heading is an angle and $\text{SQRT } 2$ is what it should be.

7.1 The words

FD n, BK n	forward, back
RT n, LT n	right, left, in degrees
PU, PD	pen up, pen down
HT, ST	hide, show the turtle
SETSHAPE "name	the turtle's shape: TURTLE, BIRD
HOME, CS	to the centre; clear the screen as well
SETXY x y	put it somewhere
SETX, SETY, SETH	one coordinate, or the heading
SETPC n	the pen's colour
FILL	paint the area under the turtle
XCOR, YCOR, HEADING	where it is
REPEAT n [..]	do it n times; REPCOUNT says which time this is
IF c [..]	decide
IFELSE c [..] [..]	decide between two
MAKE "x expr, :x	a variable, and its value
PRINT, SHOW	print a value
RANDOM, SQRT, SIN, COS, INT, ABS	numbers
TO name :a ...END	a procedure; STOP and OUTPUT leave it
LOAD "name	run a .LGO file
EDIT "name	edit one in VI, then run it
HELP, BYE	the words; back to K/OS

7.2 Procedures

A procedure is a new word, and it may call itself:

```
TO TREE :N
  IF :N < 5 [STOP]
  FD :N LT 30 TREE :N * 0.7
  RT 60 TREE :N * 0.7
  LT 30 BK :N
END
TREE 80
```

A procedure's inputs are its own; MAKE writes the nearest variable of that name, or makes a global one. Recursion is real and goes deep — deep

enough for any tree — and a procedure that recurses for ever says so rather than taking the machine with it.

7.3 Files

LOAD "TREE runs TREE.LGO from the directory you are in, or from the examples in /LANG/LOGO/EX: SQUARE, SPIRAL, TREE and SNOW. EDIT "MINE opens MINE.LGO in VI (Chapter 9) and runs it when you leave the editor, which is the way to write anything longer than a line. A .LGO file is plain text, one line as you would type it after another.

Chapter 8

Pascal

The machine has two Pascals, and they share nothing but the language. Keep them apart in your head from the start:

	Turbo Pascal 3	Mad Pascal
What it is	a program of 1985	a cross-compiler of today
Where it runs	on the machine: the Z80, under CP/M	on your desktop
What it makes	a CP/M .COM, for the Z80	a K/OS .prg, for the 45GS10
What it can touch	CP/M's world only	every chip in the machine
How you get it	you supply it; it is Borland's	in the repository, free

Nothing carries over between them — not the source, not the toolchain, not the programs they produce. The first section is one, the second is the other.

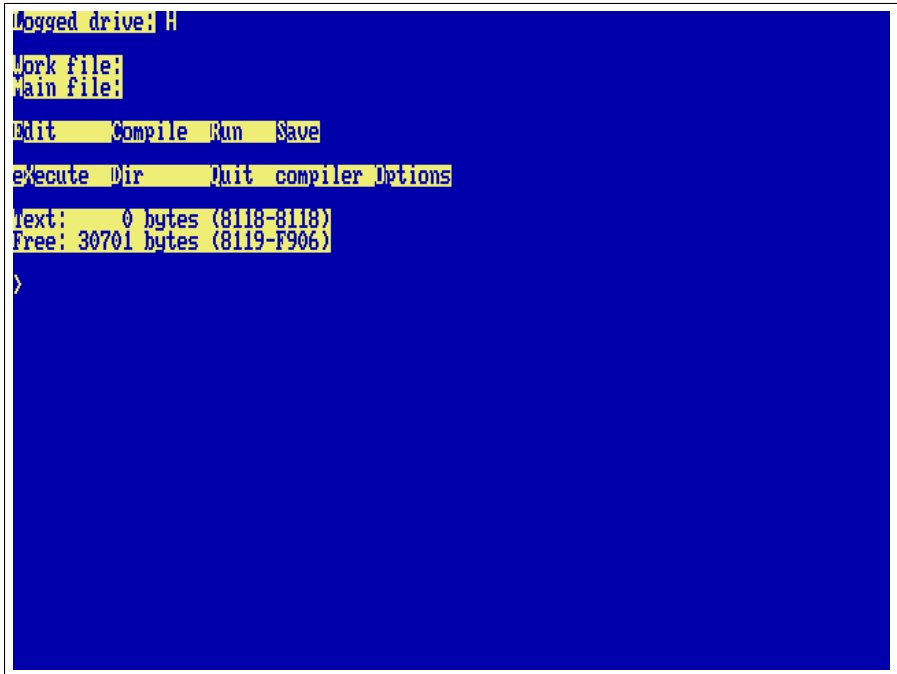
8.1 Turbo Pascal 3, on CP/M

Borland's Turbo Pascal 3 is a CP/M program, so it runs where CP/M runs: on the Tube's Z80, in its own world, exactly as it did on a Kaypro. You write, compile and run without leaving it, and what it produces is a CP/M program that runs under CP/M on the Z80 — it cannot see VICKY, the OPL2 or anything else on the K4510 side of the Tube, any more than a Kaypro could.

It is not shipped, because it is Borland's. Put TURBO.COM and TURBO.MSG into fs/CPM/H/3 — the folder that CP/M's drive P: is a shortcut to — and install it for a "VT100" or "ANSI" terminal, which is what JIM is. Then, from the K/OS prompt:

```
TURBO
```

TURBO is an alias that /SYSTEM/ETC/STARTUP.SAMPLE defines, for CPM K-TURBO: boot CP/M, go to P:, start Turbo, and — because the submit ends in EXIT — come all the way back to the K/OS prompt when you leave it.



Turbo Pascal 3 starting on the Z80, its terminal drawn by JIM.

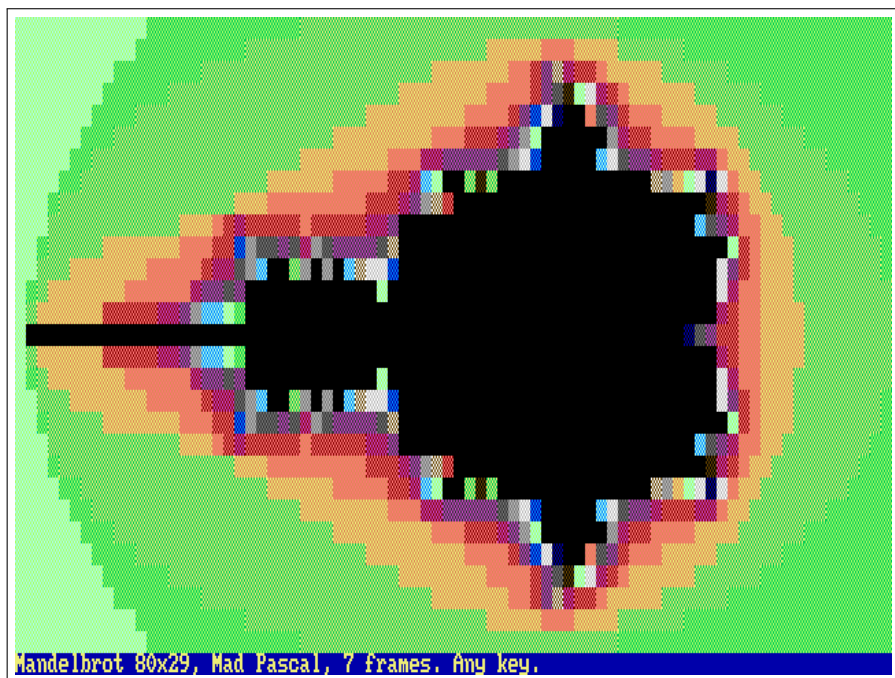
Inside, it is 1985. The editor is WordStar's: Ctrl-E, S, D and X move (the arrow keys are sent as those, so they work too; Home, End, PgUp and PgDn are not, and do not), and Ctrl-K then D leaves the editor for the menu. R runs the program in memory; O then C makes it compile to a .COM you can run from the CP/M prompt by name.

8.2 Mad Pascal, the cross-compiler

Mad Pascal is the other kind entirely — a cross-compiler, like the cc65 this machine's ROM is built with. You write on the desktop, mp compiles to 6502 assembly, MADS assembles it, and the result is a .prg beside the source in /LANG/PASCAL that the shell's RUN loads like any other pro-

gram — and, since 2026-09-08, one the machine can make for itself: PAS HELLO compiles HELLO.PAS in the directory you are standing in (Chapter 11). It is Tomasz Biela's compiler (MIT), a Turbo-Pascal-flavoured language with 8-, 16- and 32-bit integers, fixed and floating point, strings, records, pointers, units and inline assembly, made for the Atari and since taught the C64, the X16 and the Neo6502. The K4510 is its newest target.

PMANDEL



PMANDEL: the Mandelbrot set in text, sixteen colours through JIM, in 250 frames.

That is demo/pas/pmandel.pas: the Mandelbrot set, 78 by 28 cells in the machine's sixteen colours, in a little over four seconds at the desktop's 40.5 MHz — fixed point, 32-bit integers, forty lines of Pascal. PSIEVE is the classic sieve, ten passes timed by the frame counter; HELLO prints, shows the colours, and echoes what you typed after its

name. All of them are already compiled and sitting beside their sources in `/LANG/PASCAL`, so you can run them without installing anything.

Building

On the desktop, with FPC and a Mad-Pascal checkout (the installer patches its target table and rebuilds `mp`). It looks for the checkout in `~/Projects/K4510/toolchain/Mad-Pascal` unless `MP_DIR` says otherwise:

```
make pascal-install    # once; MP_DIR=... to point elsewhere
make pascal            # fs/LANG/PASCAL/*.PAS -> *.prg beside them
```

A new program is a file in `fs/LANG/PASCAL`; `make` finds it, and so does `PAS` on the machine itself. The compiled `.prg` files are committed, so a machine without the toolchain still runs them, and the test suite (`test/pastest.sh`) runs `HELLO` and `PSIEVE` through the ROM.

The target

Everything a Pascal program needs from the machine is in `pascal/` of the repository, laid out as it lives inside a Mad-Pascal checkout: the runtime base (`base/rtl6502_k4510.asm` and `base/k4510/`), where `@putchar` writes to `JIM`, the terminal (Chapter 5); the `SYSTEM` and `CRT` units' machine halves (`lib/*_k4510.inc`); and a `k4510` unit. The consequences for the programmer:

- `Write` and `WriteLn` go through `JIM`, so the `CRT` unit is the real thing: `GotoXY`, `TextColor` and `TextBackground` (the palette's constants: `BLUE`, `YELLOW`, `LIGHT_GREEN`...), `ClrScr`, `ClrEol`, `InsLine/DelLine`, `WhereX/WhereY`, `CursorOn/CursorOff`, `ReadKey` and `KeyPressed` on the keyboard device, `Delay` and `Pause` on the frame counter, and `Sound` on the machine's sound sequencer — which changed its contract when the `OPL2` became the machine's chip: `Sound` takes a channel and a pitch in quarter-semitones and holds the note until `NoSound`. `TextMode(0)` puts the ROM's screen back.
- `ParamCount` and `ParamStr` come from the shell line (`RUN HELLO one two`, or just `HELLO one two`); `ParamStr(0)` is the whole tail.

- uses `k4510` gives every chip as a typed variable at its address — `VICKY[reg]`, `DMA_SRC`, `FS_CMD`, `SYS_FRAMES`, `MATH_F[]`, `TERM_CX` and the rest — plus `FarPeek/FarPoke` into the 256 MB through the 45GS02's flat addressing, `DmaCopy/DmaFill`, `Shell('DIR')`, `LoadFile/SaveFile` through the ROM, `WaitVBlank`, and `TermWrite` for raw escape sequences — each described under *The k4510 unit*, below.
- Memory: code from `$0800`, the program's own; zero page `$22--$3F` for the compiler's registers and `$64--$A3` for its expression stack (the system calls take their arguments in `$F0--$F9`; the ROM has a zero page of its own); `$0300` the string buffer. Programs return to the shell with the ROM's state intact.

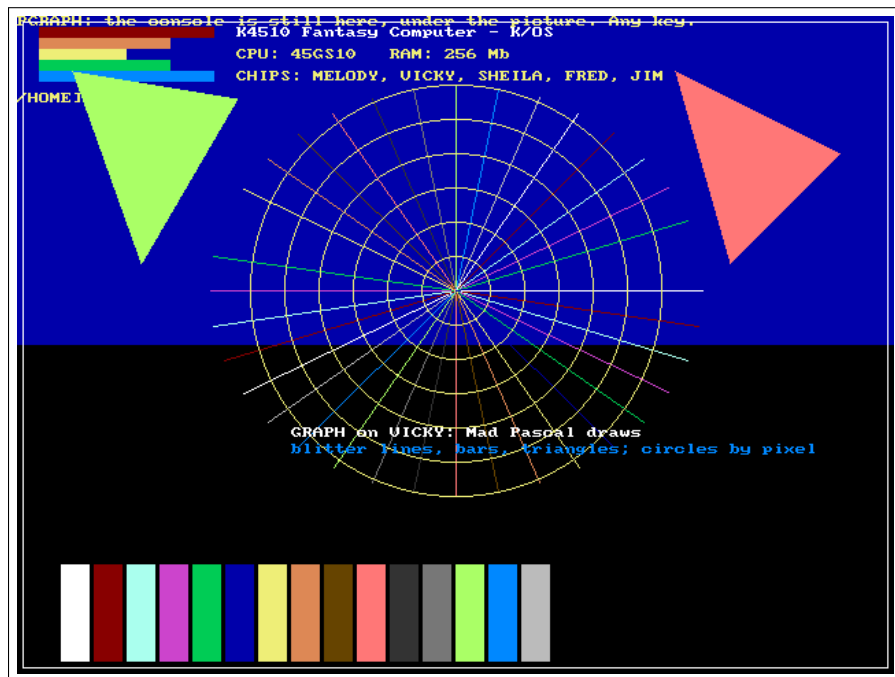
Floating point on the MATH unit

single (IEEE-754, 32-bit) does not run in software here. The runtime's add, subtract, multiply, divide, compare, `Trunc`, `Round` and `Frac` are the MATH unit at `$D700` (Chapter 13): each is a few register moves and one write to FOP, and the unit answers in the next cycle. `PFLOAT` times five thousand rounds of multiply, divide, add and subtract: 5 frames on the unit, 24 with the software library (assemble with `mads -d:SOFTFLOAT=1` to get it back for comparison). The transcendentals are in the `k4510` unit — `MathSqrt`, `MathSin`, `MathCos`, `MathTan`, `MathAtan`, `MathAtan2`, `MathExp`, `MathLn`, `MathPow`, `MathFloor` — one register write each. `SYSTEM`'s own `Sqrt/Sin/Cos/ArcTan/Exp/Ln` on single run on the unit too: the installer patches `system.pas` under `{ifdef k4510}`.

Graphics on VICKY

uses `graph` draws on `VICKY`'s layer 1: an 8-bit bitmap at `$200000` — the one BBC BASIC's ULA uses too — over the console, which stays visible wherever the picture is transparent (colour 0), as in a BBC MODE. `InitGraph(0)` draws at the glass the machine is showing: in one of the panel's integer display resolutions that glass itself (`720×540`, `1440×1080`, `360×270`, up to `1920×1200`), in the classic modes `640×480`; `GetMaxX` and `GetMaxY` say which. `InitGraph(1)` asks for `640×480` whatever the glass, putting the console in MODE 0 for it when it has to; `CloseGraph` puts the console's mode back. `PutPixel` and `GetPixel`

go through the 45GS02's flat addressing (the MEGA65 multiplier does the row arithmetic); Line, LineTo and the K4510 additions DrawLine, FillBar and FillTriangle are the blitter's LINE, FILL and TRIANGLE operations — one register write each; Circle, Ellipse, Rectangle and the rest of Mad Pascal's generic GRAPH build on those; OutTextXY writes with the console's font. PGRAPH is the demonstration.



PGRAPH: bars, a star and triangles by the blitter, circles by pixel, text from the ROM's font — and the console's own line still on top.

The k4510 unit

uses k4510 is the machine itself, for a program that wants more than CRT and GRAPH give it. Most of the unit is the chips' registers as variables at their addresses (below); the rest is ten routines for memory, files and the shell, and ten for the MATH unit. KUNIT in /LANG/PASCAL runs each of them once, and is the source of the examples here.

Far addresses. The unit's addresses are cardinals: physical addresses in the 256 MB, \$000000 to \$FFFFFF. The bottom 64 KB of it is the CPU's own view, so `cardinal(@buf)` is the far address of a Pascal variable, and the DMA routines move between the two. While a program runs, the far memory from \$400000 to \$FCFFFF — some 249 MB — is its own. Below that the machine keeps the fonts (\$10000) and the console's screen (\$30000), and GRAPH draws at \$200000; above it are the ROM, the sideways ROMs and the shell's own buffers.

FarPeek(a: cardinal): byte *far memory*

The byte at far address a, by the 45GS02's flat addressing.

FarPoke(a: cardinal; v: byte) *far memory*

Writes v there.

DmaCopy(src, dst, len: cardinal) *far memory*

Copies len bytes; overlapping is safe. The DMA engine has finished before the call returns.

DmaFill(dst, len: cardinal; v: byte) *far memory*

Fills len bytes with v.

LoadFile(const name: string; dest: cardinal): cardinal *files*

Reads the whole file to far address dest and returns its length; 0 if it could not (no such file — or an empty one).

SaveFile(const name: string; src, len: cardinal): boolean *files*

Writes len bytes from src to the file, replacing it; true if it worked.

Shell(const cmd: string) *the shell*

Runs one line as if typed at the prompt: DIR, CD, COPY, MKDIR, ECHO and the rest. Its output goes to the screen; the result is the byte at \$03FF (Chapter 12), 0 if the command worked — read it straight after: it is the last byte of the runtime's string buffer, and the next string operation may overwrite it.

WaitVBlank *timing*

Waits for the next frame, a sixtieth of a second. SYS_FRAMES is the frame count's low byte, for timing.

TermWrite(const s: string) *the terminal*

Sends s to JIM unchanged, VT100 escape sequences and all.

MathSqrt, MathSin, MathCos, MathTan, MathAtan, MathExp, MathLn, MathFloor (x: single): single *MATH unit*

One write to the unit each; angles in radians, MathLn the natural log.

MathAtan2(y, x: single): single *MATH unit*

The angle of the point (x,y), $-\pi$ to π .

MathPow(x, y: single): single

MATH unit

x^y .

Names and command lines are cut at 95 characters, the length of the shell's own line; a name is the shell's too, relative to the current directory or from /.

```
uses k4510;
const FAR0 = $400000;
var buf: array[0..255] of byte;
    RC: byte absolute $03FF;           { the shell's result }
    len: cardinal;
begin
  DmaFill(FAR0, 256, 0);
  FarPoke(FAR0 + 7, 99);
  DmaCopy(FAR0, cardinal(@buf), 256); { far -> buf: buf[7] = 99 }
  if SaveFile('/HOME/DATA.BIN', FAR0, 256) then
    len := LoadFile('/HOME/DATA.BIN', FAR0 + $1000); { 256 }
    Shell('CD /NO-SUCH-DIR');           { RC <> 0: it failed }
end.
```

The registers. Every chip is a variable at its address, typed to fit, under the names of Appendix A: VICKY[] (the 256 bytes at \$D000), the keyboard (KBD pops the next key, KBDST says whether one is waiting and which modifiers are down), DMA_SRC/DST/LEN/CMD, the file system's FS_CMD and its friends (with the FS_OPEN_READ...command constants), SYS[] and SYS_FRAMES, the MATH unit's MATH_F[0..7] and MULTINA/MULTINB/MULTOUT, TUBE[], NET[], and JIM's TERM, TERM_CX, TERM_CY, TERM_FG, TERM_BG and the rest. Reading or writing one is a single load or store.

A bank window. FarPeek is a call per byte. For a block of far memory used as an ordinary array — indexed, at full speed — point an 8 KB block of the CPU's view at it with a bank register (Chapter 12) and declare the array absolute there. The unit has no names for the bank registers; declare the one you use:

```
var win: array[0..8191] of byte absolute $8000; { block 4 }
```

```

    BANK4: cardinal absolute $D610;           { $D600 + 4*4 }
    BANK4HI: byte absolute $D613;
begin
    BANK4 := $400000;      { open: win[] is $400000-$401FFF }
    win[100] := 1;        { lands in far memory }
    BANK4HI := $FF;       { shut: $8000 is the program's RAM again }
end.

```

Writing the whole cardinal switches the block on as well (its top byte has bit 7 clear); \$FF in that byte switches it off. Block n is $\$2000 \times n$ to $\$2000 \times n + \$1FFF$. Use one the program does not occupy — its code and variables run up from \$0800 (BANKWIN's end at \$137A, so blocks 1 to 4 are free) — and never 0 (zero page and stack) or 5 to 7, which the ROM banks in and out around every system call. BANKWIN is the full demonstration, timed against FarPeek: the window sums 64 KB in 8 frames, FarPeek in 17.

The rules.

- **Code stays in the 64 KB;** data does not have to. A program is at most \$0800–\$CFFF, 50 KB with its variables; everything else goes in far memory.
- **Do not Shell a program.** RUN loads the other program over yours (Pascal programs load at \$0800, C programs at \$6000) and there is nothing to come back to. The shell's own commands are fine.
- **Inline assembly that borrows :bp** must put its low byte back to 0. Mad Pascal indexes any array over 256 bytes through (:bp),y and counts on it; the unit's own routines restore it.

8.3 The library reference

What follows is every routine the K4510 target adds to Mad Pascal, unit by unit, with what it does and a small example. (The k4510 unit, which reaches the machine's memory, files and registers, is described above, under *The k4510 unit.*) Where a routine is Mad Pascal's own and merely

runs on this machine (WriteLn, Inc...) it is not repeated; Mad Pascal's own documentation has those.

Every example in this section is untested. They were written from the units' source, not compiled and run: the Mad Pascal toolchain was not to hand when this section was written. Each one is a few lines that should compile as shown; if one does not, the source in `pascal/mp/lib` is the authority and the example is wrong.

A program names the units it wants on its `uses` line. `crt`, `graph` and `k4510` are always there to name. The `bbc` unit is a program in `/LANG/PASCAL/EX` (`BBC.PAS`); to use it, keep a copy of `BBC.PAS` beside your own program, because `PAS` finds units in the directory it compiles in.

CRT: the screen, the keyboard, the clock and the sound

uses `crt`. Everything is drawn through `JIM`, the terminal (Chapter 5), so the screen is the console's own 80 columns by 30 rows. Columns and rows count from 1, as in Turbo Pascal.

ClrScr Clears the screen and puts the cursor at the top left.

GotoXY(x, y: byte) Moves the cursor to column `x`, row `y`. The next `Write` lands there.

WhereX: byte, WhereY: byte The cursor's column and row.

TextColor(c: byte), TextBackground(c: byte) The colour of the letters, and of the cell behind them, for everything written afterwards. `c` is a palette index 0 to 15. The names are constants: `BLACK`, `WHITE`, `RED`, `CYAN`, `PURPLE`, `GREEN`, `BLUE`, `YELLOW`, `ORANGE`, `BROWN`, `LIGHT_RED`, `DARK_GREY`, `GREY`, `LIGHT_GREEN`, `LIGHT_BLUE` and `LIGHT_GREY` — the Commodore 64's sixteen.

ClrEol Clears from the cursor to the end of the line.

InsLine, DelLine Insert an empty line at the cursor's row, pushing the rest down; delete the cursor's row, pulling the rest up.

CursorOn, CursorOff Show or hide the cursor.

KeyPressed: Boolean True when a key is waiting. It does not take the key.

ReadKey: char The next key, waiting for one if there is none. Arrow and function keys come as the machine's key codes, not as Turbo Pascal's two-character sequences.

Delay(ms: word) Waits about *ms* milliseconds. The clock is the frame counter, so the wait is rounded to frames of 16.7 ms.

TextMode(mode: byte) Puts the ROM's screen and palette back, and resets JIM. *mode* is ignored; `TextMode(0)` is the spelling.

Sound(chan, freq, dist, vol: byte) Starts a note on the sound sequencer, which plays it on the OPL2. *chan* is 0 to 2 for the three tone channels, 3 for noise. *freq* is the pitch in quarter semitones: 53 is middle C and every semitone is 4 more. *dist* is ignored (it is there for Turbo Pascal's signature). *vol* is 0 to 15, the BBC's SOUND amplitude without its minus sign: 15 is loudest. The note holds until NoSound or the next Sound on that channel.

NoSound Silences every channel.

Example (untested): a coloured menu, a key, and a note.

```
program crt demo;
uses crt;
begin
  ClrScr;
  TextColor(YELLOW); TextBackground(BLUE);
  GotoXY(10, 5); Write(' K4510 menu ');
  TextColor(WHITE); TextBackground(BLACK);
  GotoXY(10, 7); Write('1 play a note');
  GotoXY(10, 8); Write('Q quit');
  repeat
```

```

    case UpCase(ReadKey) of
      '1': begin
        Sound(0, 53, 0, 12);  { middle C on tone channel 0 }
        Delay(500);
        NoSound;
      end;
    end;
  until False;  { Q is handled below in a real program }
end.

```

SYSTEM: time, chance and the command line

These are the machine's halves of Mad Pascal's own SYSTEM unit, so they need no uses.

Pause, Pause(n: word) Wait one frame, or n frames (a sixtieth of a second each). Unlike Delay it asks the frame counter directly, so Pause(60) is a second exactly.

Randomize Seeds the generator from the clock.

Random(range: byte): byte, Random(range: smallint): smallint A random number from 0 up to range - 1.

Random: Real, RandomF: Float A random fraction, at least 0 and below 1, as a fixed-point Real or a single.

ParamCount: byte, ParamStr(i: byte): string The words after the program's name on the shell line. ParamStr(1) is the first; ParamStr(0) is the whole tail, spaces and all.

Example (untested): DICE 6 3 rolls three six-sided dice.

```

program dice;
var sides, n, i: byte;
begin
  Randomize;
  sides := 6; n := 1;

```

```
if ParamCount ≥ 1 then sides := StrToInt(ParamStr(1));  
if ParamCount ≥ 2 then n := StrToInt(ParamStr(2));  
for i := 1 to n do Write(Random(sides) + 1, ' ');  
WriteLn;  
end.
```

GRAPH: lines, bars and text on VICKY

uses graph. A picture one byte a pixel, on VICKY's layer 1 over the console, as big as the glass (GetMaxX by GetMaxY). Pixels count from 0, (0,0) is the top left, and colour 0 is transparent, so the console's text shows through wherever nothing is drawn. Colours are palette indexes; 0 to 15 are the same sixteen as CRT's. Anything off the screen is quietly dropped.

InitGraph(0) Points the layer at the bitmap, sized to the glass the machine is showing, and clears it. Call it first, and place things by GetMaxX and GetMaxY rather than by numbers: PGRAPH.PAS draws the same picture at 640×480 and at 1440×1080. InitGraph(1) (GFX_VGA) is 640×480 always, as the unit was until alpha-0.9: in an integer display resolution it puts the console in MODE 0 first, and CloseGraph puts the resolution back — the BBC unit asks for it, because its sprites live at \$260000, just above a 640×480 bitmap. (InitGraph(driver, mode, path) is accepted too, for programs written for other Mad Pascal targets; it does the same as InitGraph(mode).)

CloseGraph Switches the picture off and puts the console's video mode back. A program that does not call it leaves the picture up after it ends.

ClearDevice Clears the whole bitmap to transparent, by the DMA engine. Instant.

SetColor(c: byte) The colour of everything drawn next.

SetBkColor(c: byte) The ground colour VICKY shows where neither the picture nor the text has anything.

PutPixel(x, y), PutPixel(x, y, c) One pixel (x, y are smallint, c a byte), in the current colour or the colour given.

GetPixel(x, y: smallint): byte The colour of a pixel; 0 when it is off the screen.

DrawLine(x0, y0, x1, y1: smallint) A line between two points. The blitter draws it: one register write each, with no loop in Pascal.

LineTo(x, y: smallint), LineRel(dx, dy: smallint) A line from the current position (which starts at 0,0 and is wherever the last line ended) to a point, or by a distance.

FillBar(x0, y0, x1, y1: smallint) A filled rectangle by the blitter. The corners may be in any order, and a bar that hangs off the edge is clipped.

FillTriangle(x0, y0, x1, y1, x2, y2: smallint) A filled triangle by the blitter.

OutTextXY(x, y: smallint; s: string) Draws s into the picture with the console's 8 by 8 font, its top left at x,y, in the current colour.

Mad Pascal's own generic routines — Circle, Ellipse, Rectangle, MoveTo and the rest — are built on these and work as they do on the other targets.

Example (untested): a frame, a star and a caption.

```
program gdemo;
uses crt, graph;
var i: byte;
begin
  InitGraph(0);
  SetColor(BLUE);  FillBar(40, 40, 600, 440);
  SetColor(YELLOW); FillTriangle(320, 80, 220, 300, 420, 300);
  SetColor(WHITE);
  for i := 0 to 15 do DrawLine(320, 240, 40 + i * 35, 440);
  OutTextXY(240, 450, 'HELLO, VICKY');
```

```

    ReadKey;
    CloseGraph;
end.

```

BBC: the Tube ULA's drawing and sound, in Pascal

uses `bbc`. A port of BBC BASIC's graphics and sound words, so that a Pascal program draws and sounds as its .BBC original does. It sits on `GRAPH` and keeps the BBC's own world: coordinates run 1280 by 1024 from the *bottom* left, and are scaled onto the 640 by 480 bitmap; logical colours are the BBC's, in palette entries 16 to 31; sound is the sequencer with the BBC's units. Every routine is the BBC word with a `B` in front. The eleven ports in `/LANG/PASCAL/EX` are all written with it.

BMode(n: byte) `MODE n`. 1 and 5 give four colours, 2 gives sixteen, any other graphics mode two. 3, 6 and 7 put the bitmap away and the console back. It clears the screen and sets the mode's default colours and `GCOL`.

BGcol(c: byte) `GCOL 0,c`: the colour drawn with, `c` below 128; 128 and up sets the background that `BClg` clears to.

BPalette(1, p: byte), BPaletteRGB(1, r, g, b: byte) `VDU 19`: logical colour 1 shows physical colour `p` (0 to 7, the bits are red, green, blue), or the exact red, green and blue given.

BOrigin(x, y: smallint) `ORIGIN x,y`.

BMove(x, y: smallint), BDraw(x, y: smallint) `MOVE` and `DRAW`: put the graphics cursor somewhere without drawing, or draw a line to a point from the cursor.

BPlot(k: byte; x, y: smallint) `PLOT k,x,y`. Of its many kinds this one does 4 and 5 (move and draw), 69 (a point), 85 (a triangle with the two points before), 101 (a rectangle with the point before) and 237 (a sprite).

BClg CLG: the bitmap filled with the background.

BCircle(x, y, r: smallint), BCircleFill(x, y, r: smallint) CIRCLE and CIRCLE FILL, centre and radius in BBC units.

Sprites The machine has 128 hardware sprites; VICKY moves them, so a program draws once and moves for nothing. **BSpriteCapture(n, w, h)** makes sprite *n* from the *w* by *h* pixels (8, 16, 32 or 64) whose bottom left is at the graphics cursor (VDU 23,27,1). **BSpriteSelect(n)** chooses the sprite PLOT 237 will move. **BSpritePlot(x, y)** places the selected sprite with its bottom left at *x,y*. **BSpriteShow(n, m)** makes sprite *n* show sprite *m*'s picture, so one picture serves many sprites. **BSpriteFlip(n, f)** flips it (bit 0 across, bit 1 up and down), **BSpriteHide(n)** hides it, and **BSpritesOff** turns the layer off.

BSound(ch, amp, pitch, dur) SOUND *ch,amp,pitch,dur* (*amp* is a shortint, the rest byte). *ch* is 0 for noise, 1 to 3 for the tones. *amp* is 0 down to -15. *pitch* is in quarter semitones, 53 being middle C. *dur* is in twentieths of a second, and 255 holds the note. The notes are queued: the sequencer keeps 64 a channel, so a whole tune can be sent at once and it plays while the program goes on or ends. **BSoundOff** silences every channel and empties the queues.

BWait(cs: word) WAIT, in centiseconds, rounded to frames.

BRnd(n: word): word, BRandomize RND(*n*): 1 to *n*, from a 32-bit generator — Mad Pascal's own Random is a 16-bit one whose pairs fall on a lattice, which the lines of KALEID showed up. **BRandomize** seeds it from the clock.

BSx(x), BSy(y) A BBC coordinate as a bitmap column or row, for a program that wants to mix BDraw with GRAPH's PutPixel.

Example (untested, except for the sound, which is TUNE's): a mode 2 picture with a held chord.

```
program bbcdemo;
uses crt, k4510, bbc;
```

```

begin
  BMode(2);                      { sixteen colours }
  BGcol(1); BCircleFill(640, 512, 300);
  BGcol(7); BMove(100, 100); BDraw(1180, 924);
  BPaletteRGB(8, 255, 128, 0);    { colour 8 is orange }
  BGcol(8); BPlot(101, 900, 200); { a rectangle from the cursor }
  BSound(1, -12, 53, 255);        { C, held }
  BSound(2, -12, 69, 255);        { an octave above, held }
  ReadKey;
  BSoundOff;
  BMode(7);                      { the console back }
end.

```

Things this Mad Pascal does differently

Found while writing the demos; each cost an hour once.

- Result holds a function's value, and Exit(value) sets it and leaves at once — the BBC's = value. FUNCS.PAS is the example.
- A function's local variables, Result included, are *static*: one copy, shared by every call. So Join(Join(a, b), c) gives the outer call the inner one's Result. Call them one after the other into variables.
- Strings are joined with Concat(a, b), two at a time. + on strings is a syntax error with a literal first and, with variables, compiles to an address sum.
- Pascal ignores case, so a variable called page and a constant called PAGE are the same thing.
- A local array's own name is a pointer to it. To hand its address to a routine in assembly, write adr.buf, not buf.

Chapter 9

The Editors

Two of them, because they answer different questions. EDIT is the one to reach for first: it looks and works like the editor MS-DOS shipped from version 5 on, menus, mouse and all, so there is nothing to learn. VI is for the file too big to think about, and it is modal, so it expects you to have met `vi` before. Under both is the same engine — and PROG (below) is the third front end on it.

All of them take their keys raw from the ROM — an arrow arrives as one byte rather than an escape sequence to unpick, with a bit beside it saying that it *is* an arrow and not the accented letter that shares its code. None needs the Tube: they are programs of this machine.

9.1 EDIT

EDIT `name` opens a file, or starts a new one by that name; EDIT alone starts an untitled one. The screen is MS-DOS 5's: the menu bar along the top, the text in a framed window with the file's name in its top border, a scroll bar down the right and another along the foot, and the status line at the bottom with the line and column. The colours are EDIT's too — grey menus, the text on blue, the status line in cyan. EDIT `-s name` uses the console's own colours instead, and the *Options* menu switches between the two while you work. *Options, Tab Width...* sets how many spaces Tab puts, from 1 to 16 (four unless told): Tab always puts spaces up to the next stop and never a tab character, so a file looks the same in every editor. PROG has the same menu item, and the two (and VI's `:set ts=`) share one setting.

The text lives in far memory, not in the 64 KB: a file may run to 16 384 lines of up to 255 characters, and undo goes back as far as the session does. Files are saved with a newline at the end of each line; a DOS file's carriage returns are read and dropped.

Key	Does
arrows, Home, End, PgUp, PgDn	move; with Ctrl, a word at a time and the ends of the file
Shift with a moving key, Ctrl+A	select; typing, Enter, Backspace and Del replace what is selected
Ctrl+X, Ctrl+C, Ctrl+V	cut, copy, paste — DOS's Shift+Del, Ctrl+Ins and Shift+Ins too. With nothing selected, the line
Ctrl+Z, Ctrl+Y	undo, redo
Insert	insert or overwrite (an underline cursor or a block)
Enter, Tab	a new line that keeps the indent; spaces to the next tab stop — Tab and Shift+Tab indent and outdent a selection
Ctrl+N, Ctrl+O, Ctrl+S, Ctrl+Q	a new file, open, save, leave — asking first about unsaved changes
Ctrl+F, F3, Ctrl+G	find, find again, go to a line
Ctrl+R	renumber a BASIC program: 10, 20, 30..., and every GOTO with it
Ctrl+U	BBC BASIC's keywords in capitals (print becomes PRINT); Ctrl+Z puts them back
F10, or Alt and a letter	the menus: Alt+F is <i>File</i> , Alt+S <i>Search</i> ; then the arrows, Enter, or an entry's letter; Esc closes
F1	the keys

The mouse does what it did in DOS. A click puts the cursor in the text, a drag selects, Shift and a click selects from the cursor, and the wheel scrolls. A click on a menu's name opens it and a click on an entry runs it. The scroll bars take clicks too: the arrows move a line or a column, the bar on either side of its thumb a page, and the thumb can be dragged.

The dialogs — *Open*, *Save As*, *Find*, *Change*, *Go To Line* — are boxes with a shadow, in the DOS way: Tab goes from one field or button to the next, Enter is the first button, Esc cancels, and the mouse clicks any of them. *Open* lists the directory you are in; Enter or a second click on a directory goes into it (and the shell's current directory goes with it, as DOS's did), on a file opens it. *Change* replaces every match in the file at once, and one Ctrl+Z takes all of it back.

VI's keys, for the fingers that have them: *Options*, *VI Keys*, or EDIT -v name. EDIT then starts in VI's normal mode, and everything

in the VI section below about moving and changing holds — it is the same code, not a copy of it: counts, h j k l, w b e, d c y with a motion or doubled, x p u, Ctrl-R; . is the one thing missing. i a o and the rest type until Esc, with EDIT's keys while you do (Shift and an arrow still selects). : / and ? open a line on the status row: :w :q :q! :wq :x, :12 goes to line 12, :%s/old/new/g. The menus, the mouse and the dialogs stay as they were, and so do the Ctrl keys — Ctrl+S saves, Ctrl+Q leaves — except Ctrl-R (redo), Ctrl-U and Ctrl-D (half a page), which are VI's in normal mode. What you yank or delete is the clipboard, so dw then Ctrl+V works, and Ctrl+C then p. The maps come too: :map and :imap on the : line, and VI.RC's at the start — so imap jk <Esc> there (it ships that way) is jk out of insert mode in VI, EDIT and PROG alike (a lone j is typed after a second, as vim does).

9.2 VI

VI name is modal in the old way. In *normal* mode the keys are commands; a handful of them put you into *insert* mode, where what you type goes into the file; and Escape brings you back. The : line at the bottom is the third place, for the ex commands. The cursor says which of the three you are in: a block in normal mode, a bar while inserting, an underline on the : line. Accented letters go in like any other, in insert mode and after r.

Modes

Key	Enters insert...
i	before the cursor
a	after the cursor
I	at the start of the line
A	at the end of the line
o	on a new line below
O	on a new line above
s	replacing the character under the cursor
S	replacing the whole line
C	replacing the rest of the line
<i>cmotion</i>	replacing what the motion covers; <i>cc</i> the whole line
Esc	...and leaves it, back to normal mode
:	opens the command line (from normal mode)

Moving

A count goes in front of nearly anything: 5j is five lines down, 10G is line ten.

Key	Moves
h j k l	left, down, up, right — the arrows do the same
w	to the next word
b	back a word
e	to the end of the word
0	to the start of the line
^	to the first non-blank on the line
\$	to the end of the line
gg	to the first line
G	to the last line, or to line <i>n</i> with a count
PgUp, PgDn	a screenful; Ctrl-D, Ctrl-U half of one
Enter, +, -	the next line, the previous one, at its first non-blank
/text	to the next <i>text</i> — a plain substring, not a pattern
?text	to the previous one
n	the same search again; N the other way

Changing

Key	Does
x	delete the character under the cursor
X	delete the one to its left
<i>dmotion</i>	delete what the motion covers: dw a word, d\$ to the end
dd	delete the line
D	delete to the end of the line
<i>ymotion</i> , yy	copy (yank) the same, without deleting
p	put what was deleted or yanked, after the cursor
P	put it before
rc	replace one character with c
~	flip the case of one character
J	join the next line onto this one
u	undo — unlimited
Ctrl-R	redo — also unlimited

Operators take any motion — d2w deletes two words, y\$ yanks to the end of the line — or double themselves to work on whole lines. A charwise operator **clamps to the line**: dw at the end of a line stops there instead of eating into the next one. That is deliberate, and so is the plain-substring search: /foo finds foo, and . * [mean themselves.

The command line

Command	Does
:w	save
:w <i>name</i>	save under another name
:q	leave — refuses on a changed file
:q!	leave anyway, changes lost
:wq, :x	save and leave
:s/ <i>old</i> / <i>new</i> /	replace the first <i>old</i> on this line (/g: every one)
:%s/ <i>old</i> / <i>new</i> /g	the same on every line of the file
:map <i>keys result</i>	define a key sequence, in normal mode
:imap <i>keys result</i>	the same, in insert mode
:renum [<i>start</i> [<i>step</i>]]	renumber a BASIC program (10 and 10 unless told), and every GOTO with it; u puts it back
:make	save, compile a .C or .PAS, and go to the first error
:run	:make, and if it compiled, run it; a key comes back
:cn, :cp	the next, the previous compiler message
:cc <i>n</i> , :cl	message <i>n</i> ; the whole list
:set wrap	a line longer than the screen is folded onto the rows under it (so it starts); :set nowrap keeps it to one row that scrolls sideways, and :set wrap! changes it over. Folded, j and k still move by a line of the file, and gj and gk by a row of the screen. set nowrap in VI.RC keeps it
:set ts= <i>n</i>	Tab's width: it puts spaces to the next stop, never a tab character (4 unless told; set ts=2 in VI.RC keeps it, for PROG too)

The command word is read in either case, so :Q and :WQ work with the caps lock on — which is how most people arrive in VI from a BASIC.

Compiling from VI

:make is the edit-compile loop without leaving the editor. It saves the file, compiles it with the machine's own CC or PAS (Chapter 11) — which one, the name decides — and puts the cursor on the first error, with the message on the status line:

```
VI HELLO.C  
:make
```

error 1 of 3: ';' expected — and :cn is the next, :cp the one before, :cl all of them on one screen, :cc 3 the third. Mad Pascal gives the column as well as the line, and the cursor goes to it. An error in another file — a unit, or k4510.h — is shown with that file's name and does not move the cursor. A clean compile says what it made, hello.prg: 2140 bytes, and counts the warnings, which :cn walks through the same way.

:run is :make and then the program, as if typed at the prompt. It runs over VI by way of SWAP, so VI is there again when it ends; the program's last screen stays up until you press a key. VI reads the file back afterwards, because a program may use the far memory VI keeps the text in — which is why :run saves first, and why the undo history starts again. From a VI that was itself started by SWAP (from a BASIC's *VI, or RANGER), :run cannot nest and says so: leave VI and type the program's name.

The compilers write what they said to /SYSTEM/LOG/MAKE.ERR, one message a line in one form for every language (FILE:LINE:COL:KIND:TEXT), and that file is all VI reads — so a compiler added later needs a script, not a new VI.

Making jk leave insert mode

The usual mapping is the one most people want first, and it is typed at the : line exactly like this, with the Escape spelt out in angle brackets:

```
:imap jk <Esc>
```

From then on, typing j then k in insert mode leaves it, and the two letters never reach the file. A lone j still does: the editor holds it for one second waiting for the k — vim's own timeoutlen, and long enough for a deliberate “j, k” — and hands it over when nothing comes. <CR> is the other spelt-out key; everything else in the result is literal.

To have it every time, put the same line, without the colon, in /SYSTEM/ETC/VI.RC: VI runs that file once the text is loaded, one ex command per line, a line starting with " being a comment. /SYSTEM/ETC/VI.SAMPLE is

one to copy, the way `STARTUP.SAMPLE` is, and the file is yours — it is not in the repository. If a mapping you have defined seems not to fire, the usual reason is that there is no `VI.RC` on that machine, only the sample.

Where the file lives. Nothing of it is in the 64 KB. Every line is a 256-byte slot in far memory — a length and up to 255 characters — and only the line under the cursor is brought down, fetched when the cursor arrives and written back when it leaves. So the ceiling is far memory rather than the address space: 32000 lines, and `LOAD` and `SAVE` go straight to and from a flat copy out there without the file ever passing through the CPU's view. `Undo` is a journal in the same place, which is why it is unlimited: one level would have cost the same as all of them.

A 1.28 MB file of 20000 lines — twenty times the whole address space — opens, edits at the far end and saves back byte for byte.

9.3 PROG

`PROG` name is the front end for writing a program in C, Pascal or REXX. It looks and works like `EDIT` — the same menus, dialogs, scroll bars, mouse and colours (`PROG -s` for the console's own) — with the open files as tabs along the window's top border and, under the text, a second window with what the compiler said. In the spirit of Turbo Pascal: `F9` compiles, `Ctrl+F9` compiles and runs, and an error puts the cursor on the line it is about. `PROG -v` (or *Options*, *VI Keys*) has `VI`'s keys exactly as `EDIT -v` has them, and four more on the `:` line: `:make` `:run` `:cn` `:cp` are `F9`, `Ctrl+F9`, `F4` and `Shift+F4`; `:q` asks about every changed file, as `Ctrl+Q` does, and `:q!` leaves without asking.

Key	Does
moving, selecting, the clipboard, undo Enter, Tab	as in EDIT (above): the arrows with Ctrl and Shift, Ctrl+A, Ctrl+X/C/V, Ctrl+Z/Y a new line that keeps the indent; spaces to the next tab stop (four, or <i>Options, Tab Width</i> , or set <i>ts=</i> in VI.RC) — with lines selected, Tab and Shift+Tab indent them and take the indent back
Insert	insert or overwrite (an underline cursor or a block)
Ctrl+S or F2, Ctrl+O, Ctrl+Q	save, open (in a tab of its own), leave — asking first about every file with unsaved changes
Ctrl+N, Ctrl+W, F6, Shift+F6	a new file, close this one, the next and the previous file
<i>File</i> → <i>New Project</i>	a folder with a PROJECT.K4P and a first file that compiles as it stands
Ctrl+F, F3, Ctrl+R, Ctrl+G	find, find again, change (every match), go to a line
Shift+Ctrl+F	find in files: every source file in this file's directory, into the message list
F9, Ctrl+F9	save every changed file, compile the .C (CC) or .PAS (PAS); and run it. A .RX has nothing to compile: F9 saves it, Ctrl+F9 runs it, and an error takes you to its line
F4, Shift+F4	the next, the previous message — one about another file opens it
F10, or Alt and a letter the mouse	the menus: Alt+B is <i>Build</i> ; F1 the keys the text, a menu, a tab, a message, the scroll bars; a drag selects, the wheel scrolls

PROG does not renumber BASIC: that is EDIT's Ctrl+R and VI's :renum, and since PROG took EDIT's clothes there is no room left in it for a third copy.

The messages are the ones :make reads in VI, from the same /SYSTEM/LOG/MAKE.ERR: an error in the file on screen takes the cursor to its line (and column, from Mad Pascal); one in another file — a unit, a header — is listed with that file's name. A } typed on a line of its own goes back a level. F12 stays the machine's (its menu; Shift+F12 pauses), so PROG leaves it alone; Ctrl-H is Backspace on this keyboard, which is why change is Ctrl+R.

Up to eight files are open at once, as tabs in the window's top border — the one in front lit, a * on each with unsaved changes; a click on a tab brings it forward. F9 saves every changed file before it compiles, because the compilers read the disk: a unit edited in another tab is what PAS sees. An error in another file — a unit, a header — opens that file (or brings its tab forward) at the line. Find in files (Shift+Ctrl+F, or *Search*) looks through every source file in the directory of the file in front, and what it finds goes into the message list, where F4 walks it like errors. After Ctrl-F9 the other open files are read back from the disk: a program may use the memory they wait in.

Projects

A program in several files is a *project*: a folder, and in it a PROJECT.K4P that names them, one KEY=value to a line:

```
# GAME -- a PROG project
NAME=GAME
LANG=C
SRC=GAME.C SPRITES.C SOUND.C
```

For Pascal it is MAIN=GAME.PAS instead of SRC= — the units come in by uses, from the same folder — and OUT= names the program when NAME in lower case will not do. With a PROJECT.K4P beside the file in front, F9 builds the project, whichever of its files you are in — a header or a unit too — and Ctrl-F9 runs the project's program; the files row starts with its name in brackets. PROG GAME/PROJECT.K4P (or *Open* on it) opens every source in a tab, and *File* → *New project* makes the folder, the project file and a first file that compiles as it stands. CC -p and PAS -p build a project from the prompt the same way.

A C project compiles each source once and keeps what it made (on the Linux side, never in your folder): the next F9 compiles only the files that changed — all of them if a .H beside them did — and says so, game.prg: 4211 bytes (1 compiled, 2 kept).

Where it is going. This is PROG's fourth stage: projects came third, the mouse and selection fourth, and REXX is already in. Next come

the other interpreters: EhBASIC, LOGO and Forth run on the file in front of you, their errors in the same list.

9.4 WORD

WORD name reads a Microsoft Word document — a .DOCX, what Word has saved since 2007 — in the same clothes as EDIT: the menu bar, the frame with the name in it, the scroll bar, the status line with where you are and how many words there are. It reads; it does not write a .DOCX. *File* → *Save As Text* (Ctrl+S) writes what it shows as a plain text file, a paragraph to a line, which EDIT can then take.

The page is laid out to the window. Each paragraph is wrapped at a word, centred or set right where Word had it so, with a blank line between paragraphs; lists keep their bullets, and numbered lists their numbers; a table comes out a row to a line, with its cells between bars. A text cell has a colour and nothing else, so emphasis is colour: **bold** is white, *italic* cyan, both yellow, underlined green, a link light blue; the title and the first headings yellow, the next white, then cyan. Accented letters come through as far as the machine's code page has them — é and ç do; most accented capitals lose the accent.

Key	Does
Up, Down, PgUp, PgDn, Space	a line, a screen
Ctrl+Home, Ctrl+End	the beginning, the end
Ctrl+F, F3	find (in any case), find again
Ctrl+O, Ctrl+S, Ctrl+Q or Esc	open, save as text, leave
F10, Alt and a letter	the menus; F1 the keys; the mouse works the menus, the scroll bar and the wheel

WORD -s name is in the console's own colours, as EDIT -s. *View, Tab Width...* chooses how many spaces a Tab in the document is drawn as (four unless told, 1 to 16; always spaces, never a tab character), and reads the file again so the change shows.

How it reads one. A .DOCX is a zip of XML files, and the machine can already MOUNT a zip (Chapter 2): WORD mounts the document, reads

word/document.xml — and the styles and list definitions beside it — into far memory, unmounts it, and parses the XML itself, on the 45GS10. Pictures, text boxes, footnotes, headers and footers, fonts and sizes are passed over. A Word 97–2003 .DOC is another kind of file altogether and WORD says so rather than guessing; save it as .DOCX in Word first. A .TXT opens too.

9.5 Editing from inside a BASIC

There are two cases, and they look alike, so here is the rule.

To edit the program you are writing, type *EDIT or *VI with nothing after it. BASIC saves the program to a temporary file, runs the editor on it, and loads it back when you leave — so what you type in the editor is what you LIST afterwards. Variables do not survive the round trip, exactly as with LOAD. Section 4.5 has the details for EhBASIC; BBC BASIC does the same with *VI and *EDIT (Chapter 5), and LOGO with EDIT "name (Chapter 7).

To renumber it, which none of those BASICs could do for itself, use :renum in VI or Ctrl-R in EDIT. Both know the language from the file: a .BAS is EhBASIC's, a .BBC is BBC BASIC's, with its ELSE targets and its lower-case variables (a goto there is a name, not a jump). The lines' own numbers change, and so does every target after GOTO, GOSUB, THEN, RESTORE and ON...GOTO; strings, REM and DATA are left alone. A GOTO to a line that does not exist is kept as it is and counted in the message, and a program whose numbers are out of order is refused, not guessed at. LOGO has no line numbers, and says so.

BBC BASIC wants its keywords in capitals: to it print is a variable and only PRINT prints, so a program typed in lower case stops with *Mistake*. EDIT's Ctrl+U (*Edit, Uppercase Keywords*) puts every keyword in the file in capitals — only a whole word that is one: left\$(becomes LEFT\$(and procdraw PROCdraw, but total, count% and name\$ stay the variables they are, and strings, the rest of a REM or DATA and a star command are left as typed. EDIT -u name does it at every save, so you can type in lower case throughout; BBC's *EDIT starts EDIT that way. (A keyword run into a name, as in for i%=1 to 10, is not found: leave the spaces in. And BBC BASIC's own *LOWERCASE ON is the other way round the problem: it

takes the keywords in lower case, at the price of every lower-case variable that happens to be one.)

To edit any other file, put SWAP in front:

```
*SWAP EDIT NOTES.TXT
```

A program loaded from the shell lands on top of whatever is in memory, so a plain *EDIT NOTES.TXT would run the editor over the program you were writing. SWAP (Chapter 2) puts the machine away first — program, variables, screen and all — runs the editor on a clean one, and gives yours back when the editor exits.

So: no file name, no SWAP needed, it is done for you; a file name, SWAP first.

Chapter 10

RX: the Machine's REXX

/STARTUP.BAT is a list of commands, and a list of commands is enough right up to the moment something has to be decided. RX is the answer to that: a REXX for this machine, in the Amiga's tradition rather than the mainframe's.

```
RX HELLO
```

runs HELLO.RX. So does typing HELLO on its own, because the shell's rule for an unknown word — try it as a program on disk — takes a script as readily as a .prg. The scripts that come with the machine are in /LANG/RX.

10.1 Why REXX, and what ARexx was

ARexx on the Amiga was two things at once. It was the REXX language — readable, typeless, and forgiving in the way a scripting language for people who are not programmers has to be. And it was the *port* system: every serious Amiga program published a port, so ten lines of REXX could drive a paint program and a terminal in the same script. The second half is why anybody remembers it.

Both halves are here. The language is `demo/rexx.c`, 37 KB loaded at \$0800 because it wants nearly the whole program area, and it is a real one: strings and whole numbers, `PARSE` with its templates, `DO` in all its forms, `SELECT`, internal functions with `PROCEDURE EXPOSE` and recursion, compound (stem) variables, `SIGNAL`, and about fifty built-in functions.

The thing that makes it glue is REXX's oldest idea: **a clause the interpreter does not recognise is not an error, it is a command**, sent to whatever the current environment is.

```
ADDRESS COMMAND  
'DIR APPS'  
IF RC = 0 THEN SAY 'that worked'
```

10.2 The three environments

ADDRESS	Sends the line to
COMMAND	K/OS's own shell — exactly as if you had typed it. The result comes back in RC
TUBE	the co-processor's shell, with its output brought back in RESULT
<i>name</i>	a program's port (below)

10.3 Ports: the file is the mailbox

The Amiga's ports were message ports between tasks that were all running at once. This machine runs one program at a time, so that arrangement could not be copied — and the substitute is the obvious one: a file.

A script writes its request to /LANG/RX/MAIL.COMD, SWAPs to the program with that file as its one argument, and reads the reply from /LANG/RX/MAIL.RPL: a result code on the first line and the text after it. A program joins the scheme by honouring one argument, which is about twenty lines of work.

CHESS is the worked example, and it plays a whole game this way: NEW, MOVE, GO, LEVEL, BOARD, FEN and STATUS. Its position lives in /APPS/CHESS/PORT.GAM, because a program swapped in starts fresh every time and the file is its memory. CHESS.RX in /LANG/RX plays against the engine and draws the board itself from what comes back.

10.4 What is in /LANG/RX

A language with nothing written in it proves nothing, so:

HELLO.RX	the smallest one
MACHINE.RX	the machine reading its own registers
HOSTINFO.RX	the host asked about itself, over the Tube
WC.RX	a word count
SUMFILE.RX	a column adder
GUESS.RX	a guessing game
WUMPUS.RX	Hunt the Wumpus, in 150 lines
CHESS.RX	a game of chess played through the port

Games are good tests. Wumpus alone found three real faults in the interpreter on the day it was written: EXPOSE of a stem was *assigning* it, which silently threw away every member of the caller's map; LEAVE stopped at the first END it met rather than the loop's, so leaving from inside a SELECT went nowhere; and a plain DO group was counting as a loop. None of them would have shown up in a script that only ran commands in order.

One thing to know about running programs from a script. A program started by the shell loads at \$6000 — through the middle of the interpreter. So RX asks the filesystem whether the first word of a command names a file: a built-in shell command runs directly and its output stays on the screen, and a program is swapped in and out around the interpreter. That is also why RX uses SWAP -k (Chapter 2): the ordinary SWAP would restore the screen and throw away what the program had just printed for the script to read.

Chapter 11

The Linux Underneath

Every emulated machine sits on a real one. On most of them that fact is carefully hidden; here it is a feature, because the real machine has compilers on it and the fantasy one is being used to write programs.

This chapter is about the seam — the one character that crosses it and the words that use it — and then about the Linux itself: what is on it, how it gets onto a stick or beside another system, and how it boots.

11.1 ! — the Linux prompt, from the machine's prompt

A line beginning with ! is not the machine's:

```
!ls -l
!git status
!nvim notes.txt
```

runs that command on the Linux the machine is standing on, with its output drawn on the machine's screen. A bare ! with nothing after it gives you an interactive shell there; leave it the way you leave any shell.

It is carried on the Tube (Chapter 5), which is the part of this machine already built to have another processor's console on the glass: the host's shell is started on the same pty the co-processors use, JIM renders it, and the ROM's own key loop feeds it. So the colours stay the machine's, full-screen programs work — nvim, htop and tmux run in the window — and ls --color lands in the machine's palette. The shell is told its terminal is **xterm-color**, and it speaks UTF-8: JIM draws what it can in the machine's character set and a near likeness of the rest, and an accented letter you type arrives as one.

JIM also draws *pictures*. It speaks the Kitty terminal's graphics protocol, so a program that can show an image in a terminal that way shows it here, among the text, scrolling with it:

```
!chafa -f kitty --size 30x14 cover.png
!kitten icat photo.png
```

The picture goes onto VICKY's top layer, dithered onto 216 of the palette's colours (entries 40 to 255; the console's sixteen are left alone), and is gone when the screen is cleared. Raw RGB and PNG are understood, sent in the escape itself or named as a file; animation and pictures *under* the text are not. BOOK uses the same door for the pictures in its pages, and so can any program of the machine's own: what it names is a file on the machine's disk.

SSH [user@]host is the same thing for a computer across the network: an ssh session, drawn by JIM, through the Linux's own ssh.

The third door is a *login* on the Linux beneath — its own tty, its own session — where ! gives a shell on the Tube. It is F12 → Host → *Telnet into the host*, and it asks for no password: the telnet it speaks to listens on the loopback and nowhere else, so the only way to knock on it is from the machine standing on that very Linux — which already opens an unauthenticated shell there with !. One line in /STARTUP.BAT gives it a name, which is how the shipped /SYSTEM/ETC/STARTUP.SAMPLE has it:

```
ALIAS HOST TELNET 127.0.0.1 23
```

linux = locked in the menu file shuts all of them together.

11.2 NVIM — Neovim, dressed for the machine

The machine's own editor is VI. The Linux's is Neovim, and NVIM [name] brings it onto the glass the way ! would, but set up for this machine (tools/nvim/init.lua):

- It is in the machine's colours — JIM's sixteen, which are the palette's, so a PALETTE you load recolours it too — and it draws nothing the code page has not got.
- It knows the machine's languages by their extensions in capitals: .C is C (not the C++ Neovim takes *.C for), .PAS Pascal, .RX REXX, .BAS the machine's BASIC, .BBC BBC BASIC, .LG0 LOGO. The BASIC's and

LOGO's keywords are read from the interpreters' own sources (`tools/mknvim.py`), so a keyword added there is coloured here.

- `:make` (or F9) compiles with the machine's compilers — the same CC and PAS, a `PROJECT.K4P` beside the file included — and reads `/SYSTEM/LOG/MAKE.ERR`, the file VI's `:make` reads, into Neovim's error list: it puts you on the first error, and `:cn`, `:cp` and `:cl` go through them as in VI.
- `:Run` (or F10) builds it and the machine runs it: Neovim steps aside, the program runs on the machine as VI's `:run` runs it — REXX, BASIC and LOGO in their interpreters — and a key brings Neovim back, on the line you were on. A REXX program that stops on an error brings you back to that line.

Everything else is Neovim as it comes.

Locking the door. On a machine a child uses, the way into Linux may be a way into trouble. `linux = locked` in the menu file (Section 1.7) turns `!`, SSH and the menu's telnet row away with a message, and `consoles = locked` makes the Linux consoles unreachable from the keyboard. PAS and CC still compile.

What this means for the container. The sandboxed flavour (`linux/podman.sh`) is a wall on purpose: it is handed the display, the sound, the game controllers and one folder, which the machine sees as `/MNT/SHARE`, and nothing else. `!` inside it is real, and reaches the container — which is a small Debian with the compilers on it, and not your computer.

11.3 PAS and CC: compiling on the machine

The compilers live on the Linux side, so the machine reaches them the same way you would — but without you having to remember four options:

```
VI HELLO.PAS
PAS HELLO
HELLO
```

PAS name compiles name.PAS with Mad Pascal, and leaves name.prg beside it; the name is looked for in the directory you are standing in, or give a path, PAS /LANG/PASCAL/PMANDEL.CC name does the same for name.C with cc65, using exactly the rule the machine’s own Makefile uses for its programs. A Pascal unit beside the program is found whatever case its name was saved in (Mad Pascal on Linux looks for myunit.pas; the machine writes MYUNIT.PAS). Intermediates go to a scratch directory and never appear on your disk. The compiler’s errors appear on the screen, one to a line as FILE:LINE: Error: ..., and in /SYSTEM/LOG/MAKE.ERR, which is what VI’s :make reads (Chapter 9); the exit status comes back as the result code, so a script can test it — as any ! command’s does.

That is the loop the machine owns: *edit the source on the machine, compile it from the machine’s prompt, run it on the machine*. The compiler itself still runs on the Linux beside it, which is why “self-hosted” has an “almost” in front of it — but the part that matters when you are writing a program is all on this side of the seam.

11.4 What is on the other side

On a desktop, the other side is your own computer and has whatever you have put on it. On the K4510’s own Linux it is a Debian built for the purpose, and the list is short because everything on it is there for a reason:

cc65, 64tass, nasm	the machine’s own toolchain — it can rebuild itself
FPC + Mad Pascal	what PAS runs
git, neovim	so that work done here is work
curl, ssh	what ftp://, sftp:// and SSH go through
telnetd	bound to the machine itself and nothing else
NetworkManager, Tailscale	the network: F12 → Host

The consoles. Ctrl+Alt+F1 is the machine; Ctrl+Alt+F2 to F6 are Linux text consoles, 80 by 25 in the IBM PC’s own font; Ctrl+Alt+F1

comes back. (On a laptop whose F-keys are media keys, add Fn.) Quitting the emulator (Shift+Esc, or F12 → Quit) drops you to a Linux shell on the machine's own console rather than to nothing, and F12 has a *Shut down the computer* row because there is no desktop to go back to. *Save and power off*, beside it, is the machine's own hibernation: the whole machine goes to a file, the computer turns off, and the next power-on comes back to the program and the screen you left, about as fast as an ordinary start. What a state never holds stays behind: a program on the Tube, the second screen's session, a telnet connection. The power button performs a clean shutdown. Closing the lid does what F12 → Host → *Lid closed* says: *keep running*, to begin with, or *suspend*. (Quit to the Linux shell and nothing is keeping the machine awake: there the lid suspends.) The consoles speak UTF-8 and follow the keyboard layout chosen in F12.

The battery. A laptop that keeps its battery between limits — the Dell charges to 80% and starts again under 75% — lives longer, but leaves a fifth of the battery at home. F12 → Host → *Charge to 100% once* lifts the limits until the battery is full, or until it is unplugged after charging, and then puts them back. The limits are the laptop's own, so they hold through a reboot; so does the request.

The network. F12 → Host shows the computer's name and address, and *Wi-Fi / network setup* opens NetworkManager's own screen on a spare console to join a network; the machine is back when you leave it. A network joined once is remembered.

Telnet, both directions. TELNET 127.0.0.1 23 from the machine's prompt — or F12 → Host → *Telnet into the host* — logs in to the Linux underneath through the front door rather than through !, which is useful when you want a session that survives what the machine is doing. The daemon listens on the machine itself only, so there is no network path to it at all. Outbound, the network is real: TELNET reaches a BBS, and URLs work as Chapter 2 describes.

11.5 mosh: another computer, over a poor network

The K4510 Linux has `mosh` beside `ssh`, and a small command that chooses between the two: `k4510-connect`. `ssh` sends every key to the

other computer and shows it only when the echo comes back, so on a phone's hotspot each keystroke waits out the round trip, and a network that drops for a moment can end the session. mosh logs in with ssh, then carries the session over UDP: what you type shows at once (underlined until the other side agrees), and the session goes on through a dropped link, a change from the hotspot to home Wi-Fi, or a closed lid.

```
k4510-connect you@server                a shell there
k4510-connect you@server tmux new -A -s k4510  the same tmux every time
```

k4510-connect tries mosh first. If mosh is not on this side, or cannot start a session on the other — no mosh-server there, or its UDP ports blocked — it says so and runs `ssh -t` instead. A mosh session that ran and ended keeps its own result; only a mosh that fails within its first fifteen seconds falls back. Set `K4510_REMOTE=ssh` to skip mosh altogether. The same line can go in `/SYSTEM/ETC/TERMINAL.CFG`, so that `Alt+2` opens it (Chapter 2).

The other computer needs mosh installed (it brings mosh-server) and UDP ports 60000–61000 open to you — on the tailnet, and on the home network if you connect there by its local address. Logging in is still ssh's: a password, or a key if you give this machine one.

What mosh does not do. It keeps the screen, not a scrollbar — tmux on the other side keeps that. It does not carry JIM's pictures (the Kitty graphics protocol) or JIM's own sequences, such as a note for the bottom band: for those, use `ssh -t`. There is no port or agent forwarding. If this side goes away without leaving — a power cut, Save and power off — the mosh-server on the other side waits for a client that will not come back; the next connection starts a new one, and tmux's `-A` puts you back in the old session. To leave a mosh that has lost its server, press `Ctrl+^` then `..`

11.6 On a stick

```
sudo ./linux/build-live.sh
```

makes the image, and `dd` puts it on a USB stick. At boot a menu offers

the keyboard layout, and then the whole system is copied into RAM, so the stick is not being read while you work. A persistence partition on the same stick keeps what should survive a restart: the machine's home directory — `k4510.cfg`, the menu file, your saved states and the machine's disk with your files on it — the networks you have joined, and Tailscale's identity. Leave the stick in.

The laptop's own drive is not there. The stick's kernel command line blacklists the storage drivers absolutely, so the laptop's disk is not mounted, not visible, and not writable by anything on the stick. That is not a policy setting you could turn off by accident; the machine cannot see it.

11.7 Beside another system

A computer that already runs Linux can have the machine as a second choice at power-on. Make an empty partition of 2 GB or more on its internal disk and label it `K4510`; boot the computer's usual Linux with the stick plugged in; and from the stick run

```
sudo ./install-k4510.sh
```

It formats that partition the first time (and never again), takes 4 GB off its end for a second partition, `K4510LIVE`, and copies the system onto *that*; arranges for your settings and files to be kept on `K4510`; and adds two entries to the computer's boot menu. There are two partitions because the machine loads itself into memory when it starts, and what it loads is the whole of the partition the system is on: kept apart, your files stay on the disk and only the system is copied. (A `K4510` partition under 8 GB is left as one, and says so; 8 GB or more is the size to make.) The two entries:

K4510 Fantasy Computer

the machine, quietly: the machine's logo while Linux starts and nothing else, then the prompt.

K4510 (text boot)

the same system, showing Linux's messages as it starts — the one to choose when something needs looking at.

It refuses a partition on a USB disk, and one on the stick itself, and it touches nothing of the other system but its boot menu. Run it again from a newer stick to update: the system is replaced and your settings and files are kept. The stick is only needed for the install.

The boot menu. The computer boots whichever of the two systems was used last, so the other system's own updates, which restart it, come back to it. With the menu set to stay hidden, as it is on the computer this book was checked on, the computer starts at once; Esc as it starts shows the menu. That is the other system's GRUB setting (GRUB_TIMEOUT_STYLE=hidden, GRUB_SAVEDefault=true in /etc/default/grub), and the installer leaves it as it finds it.

The whole system is read into RAM at boot here too — everything on the K4510 partition, which is why nothing large should be kept on it beside the system itself.

11.8 From another computer

The machine's Linux answers ssh (as the user k4510), which is how the menu file is edited once the machine is locked, and how a screenshot is taken from across the room:

```
ssh k4510@machine k4510-shot
```

takes one as PrtSc would and prints the name it was saved under, in shots/; k4510-shot --full takes the whole display. And the machine can be typed into from there, as if at its keyboard:

```
ssh k4510@machine "k4510-type 'B00K\n'"  
ssh k4510@machine k4510-type --key down down enter
```

\n is Enter and ~ a half-second pause; keys without a letter go by name (up, pgdn, esc, f1...). Typing and a screenshot at a time is the machine driven from another room — F12's menu included, with --key f12. F12 → Input → *Key pipe* can turn it off, or — as it starts — show every key so typed at the foot of the screen. k4510-screen is the third tool beside them: the machine's text screen, as text, so another computer can read what the machine says without looking at a picture.

On the computer doing the driving, `tools/k4510-remote` puts the three together behind one command — `status`, `type`, `key`, `screen`, `expect` (wait until a text appears), `shot`, `run` (a script of those, one a line), `logs` and `reboot` — and `k4510-remote tui` is all of it from a menu, with the machine's screen live above it and a pass-through mode in which what you type goes straight to the machine (`Ctrl-]` leaves).

Part II

Programmer's Guide

Chapter 12

Memory

12.1 The 64 KB view and the 256 MB behind it

The 45GS10's program counter is 16 bits: code executes inside a 64 KB window. The 256 MB of physical memory is reached three ways:

- **Flat pointers:** LDA [ptr],Z and the Q forms read and write any byte of the 256 MB directly. Data never needs banking.
- **Bank registers** (\$D600, one per 8 KB block): write a 28-bit base and that block of the window shows that memory. Bytes 0–2 set the base; byte 3 switches (bit 7 = off).
- **DMA** (\$D200): copy, fill, swap, line, triangle — instant, physical addresses.

12.2 The CPU view, unmapped

\$0000-\$01FF	user	zero page and stack: the program's own
\$0200-\$07FF	system	the ROM's data, its base page (page 6) and 6502 stack (page 7), and two bytes worth knowing (below)
\$0800-\$CFFF	user	50 KB for programs, ROM-free at launch
\$A000-\$BFFF	ROM	<i>the sideways window; RAM underneath</i>
\$C000-\$CFFF	ROM	<i>RAM underneath, bankable</i>
\$D000-\$DFFF	I/O	always I/O, whatever is banked; \$DB00-\$DEFF is K/OS's workspace
\$E000-\$FEFF	ROM	<i>RAM underneath, bankable</i>
\$FF00-\$FFFF	stub	always ROM: system-call stub and vectors

Three bytes of system state are fixed, so that programs can rely on them:

\$022E — a script is running

non-zero while EXEC (and so /STARTUP.BAT) is feeding the shell. A program that would wait for a key — TYPE’s “more”, say — should not, because there is nobody there to press it.

\$022F — the console’s mode

the shell’s MODE, bits 0–2 (bit 7: MODE 0’s 80×60). A program that changes the video mode reads it to put it back: the Pascal graph unit does, since its 640×480 picture needs MODE 0.

\$03FF — the result

the shell’s result code: 0 for success. A program sets it to say it failed, which is what an RX script reads as RC.

12.2.1 Low memory

The first two kilobytes, where the machine and its programs meet:

\$0000-\$00FF	program	the zero page, all of it; system calls take their arguments in \$F0-\$F9
\$0100-\$01FF	program	the 6502 stack, all of it: a program started at the prompt finds it empty
\$0200-\$022D	K/OS	the ROM’s initialised variables
\$022E-\$022F	K/OS	a script is running; the console’s mode (above)
\$0230-\$02CF	EhBASIC	lent to the interpreter: part of its code
\$02D8-\$02F5	K/OS	the launch trampoline: banks the RAM under the ROM in, calls the program, banks it out
\$0300-\$043F	program	EhBASIC’s vectors and input buffer; \$03FF the result (above)
\$0440-\$05FF	K/OS	the ROM’s variables
\$0600-\$06FF	K/OS	K/OS’s own base page: its zero page at \$0602-\$062F, the caller’s \$F0-\$F9 copied to \$0630 during LOAD, SAVE and ARGS; the rest free
\$0700-\$07FF	K/OS	K/OS’s own 6502 stack

The 45GS10 can put the zero page and the stack on any page (its B register and the stack pointer's high byte), so the ROM runs on pages 6 and 7 and the system-call stub switches both on the way in and back on the way out. Its C stack is in the I/O page, \$DB00 to \$DEFF, where no device is: a kilobyte of RAM, growing down from \$DF00. INFO -m shows how much of it has been used — a fence in the machine watches every access to it, and one below \$DB00 would write a dump and say so.

12.2.2 Who uses the 256 MB

The 256 MB behind the window, by 28-bit address. Everything not in the list is free for programs — in the main \$0040000 to \$FFFFFFF, about 220 MB.

\$0000000-\$00FFFF	the CPU's 64 KB, unmapped: the RAM under the ROM and under the I/O page included (the workspace is \$000DB00). SWAP saves and restores all of it
\$0010000-\$0013FFF	the fonts: 8×8 and 8×16 in use, then the CP437 pair and the K4510 page's pair
\$0030000	the console's text cells, 4 bytes each, up to 180×67
\$003C000	the status bands' own cells, drawn by JIM (12 KB)
\$003F000	one blank text row in the current colours
\$E000000-\$EC00000	the editors' (VI, EDIT, PROG): document slots, the file as loaded, the unnamed register, RENUM's table, MAKE.ERR
\$F000000	the editors' undo
\$FD00000	SWAP's copy of the caller's 64 KB
\$FD10000	SWAP's copy of the screen
\$FD40000	the second screen's text cells (Section 2.9)
\$FDFF000	the shell's palette, kept while a program runs
\$FE00000	EXEC's script, loaded whole
\$FE10000	a .PAL being loaded
\$FF00000-\$FF1FFFF	sideways ROM banks 1 to 16, 8 KB each (1 to 3 are used)
\$FFFA000-\$FFFFFFF	the ROM image, 24 KB, with a hole where the I/O page covers it

12.3 Programs

A .prg begins with two addresses, where it loads and where it starts, and the shell honours both. The C programs of /SYSTEM/BIN load at \$0800, as Mad Pascal's do (their C stack is the top of \$0800-\$CFFF; the big ones — VI, EDIT, PROG, WORD — have layouts of their own that also use the RAM under the ROM), MS BASIC at \$7000, EhBASIC in the RAM under the ROM (it leaves \$0800-\$BCFF to BASIC), and MONITOR at \$E000, in the RAM under the ROM — so that the memory a monitor is there to look at, \$0800 to \$CFFF, is left exactly as it was.

Waiting. A program that waits for a key or for the next frame should say so: a write of anything to \$D545, WAIT, puts the CPU to sleep until the next interrupt — the frame's, sixty a second — or a key. Nothing is missed, the loop simply looks again when the CPU wakes, and an idle machine stops costing the computer beneath it most of a core. The ROM's own key wait, EhBASIC's line input and wait_vblank() in demo/k4510.h all use it:

```
while (!(k = rom_getin())) K_WAIT();
```

When the screen changes. F12 can change the resolution or the canvas while a program runs; the ROM lays the console out again on the program's next key read, and the screen it had drawn is gone. A program that can draw itself again says so with a write of 1 to \$D546, RESIZE. Its next rom_getin() after such a change then answers \$8F with bit 7 of \$D546 set (a typed \$8F has it clear); the program writes 1 again, reads the size from JIM (\$DA05 columns, \$DA06 rows) and draws the whole screen. A program that has not asked gets Escape, as before. EDIT, PROG, WORD and VI ask (demo/dosui.h does it for the first three).

12.4 System calls

The page \$FF00 is always the ROM, whatever is banked, and its jump table is the whole of the interface a program needs:

\$FF80	CHROUT	print the character in A
\$FF83	CHRIN	wait for a key; it comes back in A
\$FF86	GETIN	a key in A, or 0 if none is waiting
\$FF89	LOAD	name at (\$F0), to \$F2--\$F5; status in A, size in \$F6--\$F9
\$FF8C	SAVE	name at (\$F0), from \$F2--\$F5, length \$F6--\$F9
\$FF8F	SHELL	run the line A/X points at, as if it were typed
\$FF92	VIDEO	put the ROM's video mode and palette back after a program drew
\$FF95	ARGS	what followed the program's name: (\$F0), length in A

A program has the zero page and the stack page to itself. K/OS runs on a base page and a 6502 stack of its own — the 45GS10 can put both anywhere: they are pages \$06 and \$07 — and keeps its C stack in a kilobyte of RAM in the I/O page, \$DB00-\$DEFF, where no device is. The stub switches both on the way in and back on the way out, and for LOAD, SAVE and ARGS copies the caller's \$F0--\$F9 across; nothing else of the program's is touched.

Anything handed to a system call must lie below \$A000: during the call the ROM is banked in over \$A000--\$FFFF, and a line kept up there would read as the ROM's bytes. SHELL copies the line into the shell's own buffer before running it, so the program's copy is left alone.

What CHROUT does with control characters. Everything goes to JIM, the terminal, which is a faithful VT100 — with three exceptions, because CHROUT is the machine's own console and keeps the promises 8-bit consoles made:

- **A new line** is CR (\$0D), LF (\$0A), or CR followed by LF: each is *one* new line, so a program written for any of the three conventions, and a text file from DOS or the web, comes out single-spaced.
- **Backspace** (\$08) rubs out the character before the cursor; a terminal's backspace only moves.
- **Form feed** (\$0C) clears the console and leaves the status bands alone.

A program that wants the terminal's own meanings — a bare CR that returns to the start of the line, for a progress counter — writes its bytes to JIM at \$DA00 instead, as TELNET does.

12.5 RAM under the ROM

`rom_out()` (or three pokes to the bank registers) banks blocks 5–7 onto the RAM beneath the ROM: a program then owns \$0800–\$CFFF + \$E000–\$FEFF = 57 KB. Every system call still works, because the calls go through the \$FF00 stub page, which banks the ROM in and out around them.

12.6 Sideways ROM

The operating system is bigger than its half of the 64 KB — so, like the BBC Micro before it, the machine grew *sideways*. The ROM file is a 24 KB base image plus appended 8 KB banks paged into the \$A000–\$BFFF window: bank 0 is the base image itself, banks 1 and 2 hold the colder commands, and bank 3 has the line editor, its history and the directory commands. Chapter 3 says which bank every command lives in. The shell pages the right bank in around each call, and up to sixteen banks — 128 KB of operating system — fit in the scheme.

The ROM is full, and it is kept that way on purpose: a command whose work can be a program on the disk becomes one — TYPE and the monitor did in September 2026 — because a program costs nothing to load (the disk is the host's, and on the K4510's own Linux it is in RAM) and can be replaced without touching the ROM.

The calls go one way, and this is the rule to hold on to if you write code of your own for a bank: code in a bank may call resident code as freely as it likes, but resident code must never call into a bank. At that address a different bank holds something else entirely, and the dispatch is the only thing that knows which bank is in. A routine that everything uses belongs in the base image, whatever it costs there.

12.7 RAM under the I/O page

A MAP of block 6 hides \$D000-\$DFFF and exposes RAM: with the ROM also banked away a program owns a contiguous field from \$0800 to \$FEFF — 61.75 KB of the 64. The trick that makes it safe is that MAP is an *instruction*, not a register: the program that hid the I/O can always ask for it back, so there is no way to lock yourself out. (The far-call gate is unreachable while block 6 is mapped; unmap before far calls.) One kilobyte of that RAM is not free: physical \$DB00-\$DEFF is K/OS's workspace, and holds the frames of the shell that started the program. A program that maps block 6 straight under itself must leave those bytes alone, or map the block onto RAM somewhere else.

12.8 Programs bigger than the window

The K4SG executable format loads segments to any physical address and the far-call gate (\$DF00) calls between them: a plain JSR into slot n banks descriptor n 's code in, and the callee's RTS banks it back out.

Chapter 13

The I/O Page

One page, \$D000-\$DFFF, always visible. The authoritative register-level reference is the machine's own headers, and Appendix A is generated from them at build time — the way the Neo6502 handbook generates its keyword reference from the firmware source. This chapter is the map of what lives where:

\$D000	VICKY	video: layers, palette, blitter, sprites, SHEILA
\$D100	input	keyboard FIFO and its status, the held keys, the mouse, break-pending
\$D200	DMA	copy / fill / swap / line / triangle
\$D300	storage	the host filesystem: open, read, dir, chdir...
\$D480	OPL2	the YM3812: address, data, status — the AdLib's arrangement
\$D500	system	clock, RTC, frame counter, DUMP, the settings
\$D600	banks	the ROM reads, the sound sequencer at \$D5E0
\$D700	FRED	the eight bank registers, masks at \$D620
\$D800	Tube	the MATH unit: IEEE floats, transcendentals, math lists, mul/div
\$D800	Tube	the co-processor: status, byte in, byte out, start/stop
\$D900	N:	the network: four channels, tcp://, http(s):// and tnfs:// (see core/net.h)
\$DA00	JIM	the terminal: a VT100/ANSI in hardware, drawing on the console's screen (see core/term.h)
\$DB00	workspace	not a device: 1 KB of RAM to \$DEFF, K/OS's C stack (Chapter 12)
\$DF00	far gate	call slots, descriptor table pointer, depth

13.1 A split screen: SHEILA

The Apple II put four lines of text under its graphics; C64 programmers did the same with a raster interrupt timed to the line. On this machine

the CPU does nothing at all: SHEILA, VICKY's display-list coprocessor (the Amiga's copper is its ancestor), runs a short list from memory at every frame, in step with the raster, and a register it writes takes effect on the line about to be drawn. A split is six instructions:

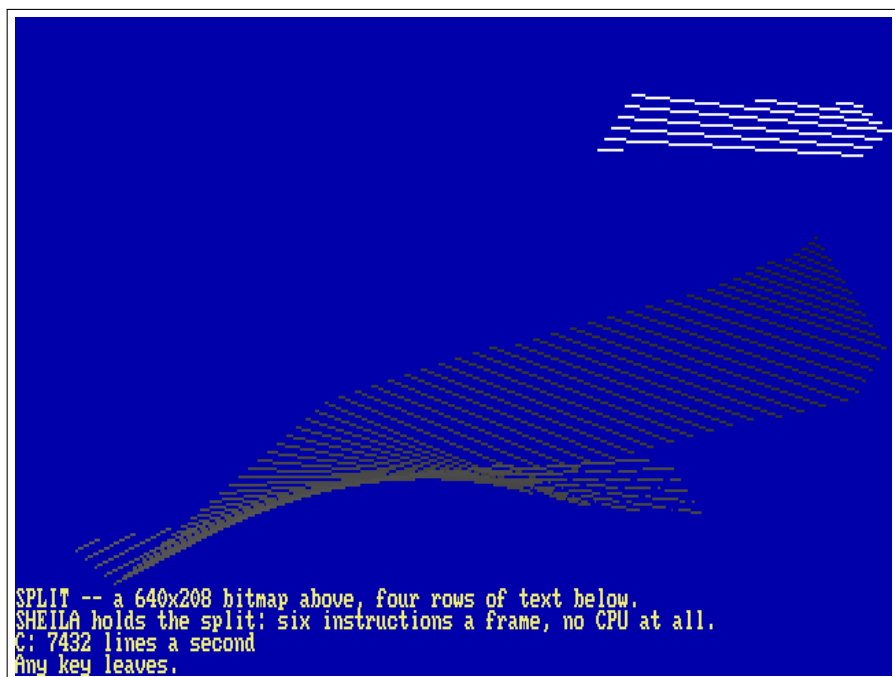
```
MOVE $10,$00    line 0: layer 0, the console's text, off
MOVE $20,$19    layer 1, a 640-wide 8 bpp bitmap, on
WAIT 416        26 text rows of 16 lines: the split
MOVE $20,$00    the bitmap off
MOVE $10,$07    the text on: rows 26-29 are text
END             and again from the top, next frame
```

With the status bands up, the split has three parts rather than two: the text layer stays on at line 0 for the top band, the bitmap takes over at the band's end, and the text comes back for the last four rows and the bottom band — eight instructions. SPLIT reads where those lines fall from the console itself (\$DA06, its rows, and \$DA08, the rows above it), so it is right with the bands up or down. Each instruction is four bytes — the operation (1 WAIT, 2 MOVE, 0 END), then its operands — and the list is anywhere in memory, a 28-bit address in \$D060–\$D063, started by 1 in \$D064. A MOVE is exactly a write by the CPU, so any VICKY register can change mid-frame: the mode, the scroll, a layer, the palette. Two rules: line numbers count the glass, 0–479, whatever the mode (in a 240-line mode each line is drawn twice, so split on an even one); and **nothing turns SHEILA off after a program**, so a program that turns it on turns it off when it leaves.

SPLIT is the demonstration, and it has two twins that draw the same ribbon of lines — endpoints bouncing, the line sixteen behind erased — as fast as they can, and say how fast in the text band: /LANG/EHBASIC/EX/SPLIT.BAS, with EhBASIC's own LINE. At the 40.5 MHz a host starts at:

Language	lines/s	how it draws one
C (SPLIT)	7400	fifteen register writes
EhBASIC	330	LINE, one blitter operation

The blitter draws a line in no time at all, so the numbers are the languages: what a BASIC spends is its own arithmetic, its arrays and its line lookups, and three hundred lines a second is a good deal more than



SPLIT: blitter lines above, the console's last four rows below, and SHEILA holding the line between them.

a game of 1985 asked for. The split itself costs every one of them nothing. A faster clock (F12, or SETUP) scales all three.

Chapter A

The Registers

Everything the machine's devices do, they do through the I/O page. This appendix is the register-level reference, and it is not written by hand: `doc/guide/mkregs.py` lifts it out of the machine's own headers at build time, so what you read here is what the emulator was compiled against. When a device gains a register, this appendix gains it in the next build — which is the only way a reference of this kind stays true.

Nothing in it is reworded: every word below is a word from a header. What the generator decides is only which column a word belongs in — the address on the left, the register's name after it, the comment's own sentence following. Where a comment's own spacing was doing the work of a table, that table is kept exactly as it was written, in monospace, because there it is the spacing that carries the meaning.

The voice is still the voice of working code: terse, occasionally opinionated, and using the machine's own shorthand — LE for little-endian, R: and W: for a register that reads and writes differently, \$ for hex. Addresses are given as an offset inside the device's block where the block's base is obvious, and in full where it is not.

Chapter 13 is the map of which device lives where; this is what is inside each one.

A.1 The I/O page

Generated from `core/io.h`.

The OPL2, wired the AdLib's way, so every AdLib register list means what it

The OPL2, wired the AdLib's way, so every AdLib register list means what it says here. The machine's one sound chip.

\$D480	<i>W ADDR</i> the register to write next R STATUS bit7 IRQ, bit6 timer 1 expired, bit5 timer 2 expired
\$D481	<i>W DATA</i> write it; R the last value written to that register
\$D482	<i>R ID</i>
\$02	= an OPL2 is fitted

The status bands and the clock's format, for the guest at \$D52D-\$D52F

The status bands and the clock's format, for the guest at \$D52D-\$D52F. \$D521 was the natural home and is FULL: all eight bits are spoken for (CP/M-by-name, margin, STARTUP.BAT, the bands, the mode request, and three of the video mode). So these are their own bytes, and there is room after them for whatever the menu grows next.

\$D800 status: bit 0 a co-processor is up, bit 2 the host shell is fitted, bit 3 a UCI chess

\$D800 status: bit 0 a co-processor is up, bit 2 the host shell is fitted, bit 3 a UCI chess engine is fitted (K4510_UCI, or stockfish where it usually lives), bit 7 a byte waits. \$D803 program: 1 BBC BASIC, 3 CP/M, 4 the host shell ('i), 5 the chess engine, 2 stop. \$D804-7 the command string's address for program 4 (empty = an interactive shell), \$D808/9 its rows/columns.

\$D53A R BATTERY the host's battery, for the status band: charge in % in bits 0-6

\$D53A R BATTERY the host's battery, for the status band: charge in % in bits 0-6

(0-100), bit 7 set on AC power or charging, \$FF = no battery (a desktop, the browser build). The frontend reads it from the host every ten seconds; K/OS draws "nn%" and an arrow at the right of the bottom band.

Bank registers and the far gate

BANK registers: \$D600 + 4n, n = 0..7, one per 8 KB block of the CPU view.

bytes 0-2 physical base bits 0-23 (little-endian); they only set the base

byte 3 bits 24-27 of the base, and bit 7 = OFF. Writing byte 3 switches the block: bit 7 clear = on, set = off. (So STQ works, and a byte-wise save/restore never switches a block on by accident.) Reads give the base; byte 3 reads \$FF while the block is off.

\$D620 read: bit n = block n banked

\$D621 read: MAP mask (bit n = MAPPED)

A banked block resolves $\text{phys} = \text{base} + (\text{cpu} \& \$1\text{FFF})$. MAP rewrites all eight blocks, so the ROM's "MAP off" at program exit clears the banks too. The I/O page \$D000-\$DFFF and the stub page \$FF00-\$FFFF stay visible whatever is banked, so banking blocks 6 and 7 reveals the RAM under the ROM only. FAR gate: JSR \$DF00 + 4n calls descriptor n of the table at FARTAB.

\$DF00-\$DF7F 32 call slots

\$DFF0 return gate (RTS lands here)

\$DF80-\$DF83 **FARTAB** 28-bit pointer to the descriptor table (read/write)

\$DF84 read: nesting depth

\$DF85 read: last error (1 overflow, 2 underflow, 3 bad slot); write clears descriptor, 8 bytes: base[4] block flags entry[2]; flags bit0 leave banked on return, bit1 do not bank (long jump to resident code). A/X/Y/Z pass through both ways: cc65 __fastcall__ works across it.

The MATH unit

MATH unit. Results are ready the cycle after the write that triggers them.

\$D700-\$D71F **F0..F7** eight IEEE-754 single registers, little-endian, read/write

\$D720 **FOP** write = execute; low 5 bits op, see below

\$D721 **FARG** (dst < 4) | src, register numbers 0..7, write before FOP

\$D722 **FFLAGS** from the last op: bit0 result zero, bit1 negative, bit2 NaN/inf

\$D724-\$D727 **FI** int32 for ITOF / FTOI

ops: 0 MOV Fd=Fs 1 ADD 2 SUB 3 MUL 4 DIV (Fd = Fd op Fs)
 5 Sqrt 6 SIN 7 COS 8 TAN 9 ATAN 10 EXP 11 LOG 14 ABS 15 NEG 16 FLOOR 17 ROUND (Fd = f(Fs))
 10 ATAN2 (Fd = atan2(Fd,Fs)) 13 POW (Fd = Fd^Fs) 21 FMOD (Fd = fmod(Fd,Fs))
 18 CMP flags from Fd - Fs, registers unchanged
 19 ITOF Fd = (float)FI 20 FTOI FI = (int32)Fs, truncated

	Math list – a program for the unit in RAM, run with one write:
\$D728–\$D72B	MLPTR 28-bit pointer to the list
\$D72C	MLRUN write = run from MLPTR until END or a STOP fires
\$D72D	MLSTAT 0 reached END, 1 a STOP fired, \$FF runaway (65536 ops)
\$D72E, \$D72F	MLCNT 16-bit counter for DJNZ, read/write list ops, 2 bytes each (op, arg) unless noted; ops 0..21 are the FOP ops with arg = (dst«4) src, and in addition:
\$80	END
\$81	STOPNEG stop if last flags negative
\$82	STOPPOS if not negative
\$83	STOPZERO
\$84	STOPNZ
\$85	JUMP arg (signed, in ops from the next op)
\$86	DJNZ arg MLCNT–, jump if nonzero
\$87	STOPFIGE arg stop if FI >= arg (unsigned byte compare on the low byte, FI clamped)
\$88	LDF arg=dst«4, then 4 bytes: IEEE single immediate into Fdst (6-byte op)
\$89	LDI , then 4 bytes: int32 immediate into FI (6-byte op)
\$8A	LDMS arg=dst«4, then 4 bytes: 28-bit address of a Microsoft- format float (exponent excess-128, mantissa1 with sign in bit 7, mantissa2, mantissa3 – EhBASIC's packed variable format); converted into Fdst (6-byte op) MEGA65-compatible integer unit (same addresses as the MEGA65):
\$D770–\$D773	MULTINA
\$D774–\$D777	MULTINB (unsigned 32-bit, LE)
\$D778–\$D77F	MULTOUT = A * B, 64-bit
\$D76C–\$D76F	DIVOUT integer part of A / B
\$D768–\$D76B	fractional part (32.32) recomputed on every write to an input byte; B = 0 gives all-ones.

System registers, and the SIDs

SYS registers (read-only unless noted):

\$80, 81 CPU clock, kHz, LE (40500)

\$02,03	physical RAM, MB, LE (256)
\$04	read: latch the host clock into \$05-\$0C and return 0
\$05	sec
\$06	min
\$07	hour
\$08	day
\$09	month
\$0A,0B	year LE
\$0C	weekday (0=Sun)
\$0D,0E,0F	frames since reset, 24-bit LE (vblank count)
\$10-\$1F	version string, NUL-terminated
\$20	ROM base page (e.g. \$A0 for a 24 KB ROM)
\$F0	write: DUMP – the host writes dumps/dump-NNN.txt (machine state, screen, PC history, keys, the shell log); read: the number of the last dump
\$F1	write: append a byte to the shell log (the ROM logs command lines and DUMP notes)
\$F2	write 1/0: automatic dump every 900 frames (15 s) on/off (on at reset on the desktop, off on the Pi; DUMP or DUMP ON also arms the per-instruction PC recorder); read: the setting
\$23	read: the CPU clock setting in force (0 = 40.5 MHz, 1 = 30, 2 = 20, 3 = 15, 4 = 10); write: ask for one – the host applies it next frame (BENCH sweeps them)
\$24,25	audio gaps LE: callbacks that found the ring empty since last cleared; any write clears

The keyboard

Keyboard: a FIFO of key-down events. Printable keys arrive as ASCII (\$20-\$7E, already shifted/dead-keyed by the host layout on the desktop); control keys as ASCII controls; everything else as \$80+ codes.

bit7 event available; bit0 shift, bit1 ctrl, bit2 alt held;

bit7 event available; bit0 shift, bit1 ctrl, bit2 alt held; bit6 the byte LAST READ from KBD was a key code (KEY_*), not a character; bit5 the byte WAITING is

a key code. The two share \$80-\$9F – KEY_LEFT and é are both \$82 – and these bits are how a line editor tells them apart.

— WATCH (\$D530-\$D535): who is trampling this byte? —————

\$30-\$33 28-bit physical address to watch

\$34 write 1 to arm; write 0 to disarm; reads back the armed state

\$35 hits since last arm (saturates at 255)

A CPU write to the watched byte – normal, MAPped, banked or a flat 32-bit store – fires a DUMP (dumps/dump-NNN.txt, tagged “watch”) and disarms, so one bug produces one dump. DMA and the host itself do not trip it: the question WATCH answers is “which instruction wrote this”, and the dump’s PC history is the answer.

The stack fence

— FENCE (\$D550-\$D55C): how deep does the ROM’s C stack go? ————

\$50 write: the base-page address of the C stack pointer – arms (0 disarms) and starts the measure; read: it

\$51 the base page that pointer lives in (the B the ROM runs with)

\$52,53 the floor, LE: the lowest address the stack may reach

\$54,55 read: the deepest address reached since armed; write \$54: start again

\$56 read: trips – accesses below the floor (saturates at 255)

\$57,58 read: the lowest 6502 S seen on the ROM’s own stack page

\$59,5A read: the pointer as it was when armed (the stack’s top)

\$5B,5C read: the lowest S on page 1 under a program (K4510_FENCE_DEEP only)

The ROM arms it at reset (crt0.s); INFO -m shows it. The first trip writes a DUMP tagged “stack fence”. core/mem.h has the rest.

WAIT: sleeping until something happens

— WAIT (\$D545): write anything, and the CPU sleeps until the next interrupt (the frame’s, sixty a second) or a key arrives; reads \$FF. The way to wait for a

key or a frame without the host spending its time on the loop. Time goes on while it sleeps.

— RESIZE (\$D546): the screen changed under a running program (Doc, 2026-10-07: F12's Canvas switched with EDIT open, and EDIT went on drawing for the old size). A program writes 1 to say it can lay itself out again; then, when the F12 menu changes the video mode or canvas while it waits for a key, GETIN answers KEY_RESIZE (\$8F) instead of the Escape it gives everyone else, and bit7 here is set – a typed \$8F, CP437's A-ring, comes with it clear. The program writes 1 again (which clears bit7), reads JIM's size and redraws. Read: bit0 the program's wish, bit7 a resize waiting. The wish is the program's: a program it runs starts without it, and it ends with it.

— PALNAME (\$D547): the palette the machine last loaded, kept by the host for the next power-on (Doc, 2026-10-07: “the palette chosen in F12 is not saved”). Write: PALETTE LOAD's file path a character at a time, then 0 – a 0 alone (PALETTE RESET) is the VIC-II sixteen; \$FF starts the read again. Read: the kept path a character at a time, 0 at its end. The ROM loads it at every reset, after the VIC-II sixteen. The frontend fills io_palname from k4510.cfg (video.palette) and saves it when io_palname_new is set.

— VIDCAP (\$D548-\$D54A): SETUP asks the host how many pixels it can draw a frame (2026-10-07). Write 1 to \$D548: the frontend runs tools/k4510-vidcap beside the machine (a few seconds; the machine goes on running). Read \$D548: 0 not asked, 1 measuring, 2 done, 3 it could not. \$D549/A: the cap it found, in thousands of pixels, which the frontend has written to k4510.cfg (video.cap) for the next start.

The Tube

The Tube (\$D800): Acorn's answer, refitted. The HOST runs Richard Russell's BBC BASIC interpreter (the vendored BBCTTY console edition, tube/bbcbasic) on a pty; the machine talks to it byte-wise:

\$D800 *R* **status** bit0 alive, bit7 a byte waits in \$D801
 \$D801 *R* next byte from the co-processor (pops)
 \$D802 *W* a byte to the co-processor (its keyboard)
 \$D803 *W* 1 start (spawn), 2 stop (kill)

The co-processor has its own flat 256 MB; PAGE/HIMEM live there, far beyond the 64 KB view. The co-processor is a process on the host (BBC BASIC, RunCPM's Z80 as program 3); where none can be started, status reads 0. The console it talks to is JIM, the terminal at \$DA00 (core/term.h).

A.2 VICKY, SHEILA and the sprites

Generated from core/vicky.h.

VICKY

VICKY – the K4510 video chip.

Register block at IO_VICKY (\$D000), 256 bytes, byte-addressed. Every pointer is a 28-bit physical address into main RAM; there is no video memory. Rendering is per scanline into an 8-bit indexed framebuffer the frontend supplies; the frontend applies vicky_palette_rgb().

\$00 **CTRL** bit0 display enable; the rest pick the mode. Without bit5 the glass is 640x480 and the raster lines 0..479 – a smaller mode is drawn into it, doubled, and centred. bit5 (hd-modes branch, 2026-09-14): the HD family, drawn at its own size, the raster lines 0..h-1:

1|32 1440x1080 1|32|2|4 720x540

1|32|2|4|16 360x270 (a 1080-line panel shows them 1x, 2x, 4x)

bit1 columns halved (320), bit2 lines halved (240), bit3 a 200-line field (40 blank lines top and bottom), bit4 columns quartered (160; with bit1).

1 640x480 1|4 640x240

1|2 320x240 1|2|8 320x200

1|2|8|16 160x200 1|4|8 640x200

\$01 **BGCOL** background palette index (where nothing is drawn)

\$02	RASTER read: current line (low 8); write: raster-compare low
\$03	read: line high bits; write: compare high
\$04	IRQSTAT bit0 vblank, bit1 raster==compare, bit2 SHEILA IRQ op, bit3 sprite collision. Write 1s to acknowledge.
\$05	IRQMASK same bits; IRQ line = IRQSTAT & IRQMASK
\$06	PALIDX palette index for the write port
\$07	PALR
\$08	PALG
\$09	PALB – writing B commits the entry and increments PALIDX
\$0A-\$0D	SPRTAB 28-bit pointer to the sprite attribute table (128 x 16 B)
\$0E	SPRCTL bit0 sprites enable
\$0F	reserved
\$60-\$63	SHEILA 28-bit pointer to SHEILA's list
\$64	SHEILACTL bit0 enable; the list restarts every frame at line 0
\$70-\$73	BLTSRC 28-bit
\$74-\$77	BLTDST 28-bit
\$78,79	BLTW width px
\$7A,7B	BLTH height px
\$7C,7D	BLTSS source stride (bytes)
\$7E,7F	BLTDS dest stride
\$80	BLTOP 0 copy, 1 keyed copy (src 0 skipped), 2 fill (value = BLTSRC byte 0), 3 AND, 4 OR, 5 XOR, 6 LINE: from (LX0,LY0) to (LX1,LY1), colour = BLTSRC byte 0, into the BLTDST surface of stride BLTDS, clipped to 0..BLTW-1 x 0..BLTH-1 7 TRIANGLE: filled (LX0,LY0)-(LX1,LY1)-(LX2,LY2), same colour, surface and clip as LINE
\$81	BLTFLG bit0 H-flip, bit1 V-flip
\$82	BLTCMD write anything: go. Instant. Reads 0.
\$84,85	LX0
\$86,87	LY0
\$88,89	LX1
\$8A,8B	LY1
\$8C,8D	LX2
\$8E,8F	LY2 (signed 16-bit)

	Blits are 8 bpp (one byte per pixel) in this version.
\$90-\$9F	COLSS read: sprite-sprite collision bits, one bit per sprite (sprite n hit another sprite this frame). All 16 cleared on read of \$90.
\$A0-\$AF	COLSL read: sprite-layer collision bits (sprite n over a non-transparent layer pixel). All 16 cleared on read of \$A0.
\$C0-\$DF	THE GLASS (2026-10-07, docs/design-video-foundations.md). The host finds the panel and tells VICKY (vicky_set_panel); she works out the canvas – the panel, or its largest 4:3 – and the integer display resolutions (IDRs) it divides into.
\$C0,C1	PANELW R the panel's pixels
\$C2,C3	PANELH R
\$C4,C5	CANVW R the canvas
\$C6,C7	CANVH R
\$C8	IDRN R how many IDRs it offers, largest first
\$C9	IDRIX RW which one \$CA-\$CF describe (0 = the largest)
\$CA	IDRS R its scale: panel pixels a machine pixel, each way
\$CB	IDRF R bit0 HD text possible there; bit1 at three times (24-wide glyphs)
\$CC,CD	IDRW R its width
\$CE,CF	IDRH R
\$D0	GLASSCTL RW what CTRL bit5's glass is. bits0-1: 0 CTRL's own bits (1, 2, 4 – the canvas /1, /2, /4: MODE 5-7); 1 the IDR at scale IDRSEL; 2 software, SWW x SWH. bits4-5, how software shows: 0 the largest whole scale, 1 fit (sharp-bilinear: the whole multiple, then smoothing); 2, stretched to 4:3, is reserved and fits for now. bit6 reserved (scanlines).
\$D1	IDRSEL RW the scale wanted: one not offered becomes the next larger scale that is, else the largest offered
\$D2,D3	SWW RW a software resolution, 160-1920 wide } clamped to the
\$D4,D5	SWH RW 100-1200 high } limits and panel
\$D6,D7	GLASSW R the glass CTRL and the above give now
\$D8,D9	GLASSH R (GLASSCTL 1 or 2 implies CTRL bit5, here and for TXT)
\$DA	SCALE R its whole scale on the panel (0: not an IDR)
\$DB	TXTCELL W a text cell, as LCTRL's field: 0 8x8, 1 8x16, 2 16x16, 3 16x32 (text32)
\$DC	TXTCOLS R whole cells of it on that glass (255 at most)
\$DD	TXTROWS R

\$DE	TXTPAD R spare lines above them: half the spare, rounded down
\$DF	TXTHPAD R spare pixel columns to their left, likewise A text32 layer paints the spare pixels round a whole grid in the nearest cell's background; K/OS scrolls layer 0 by -TXTHPAD, -TXTPAD to centre its console.
\$B0-\$B7	LAYOUT the status bands and the console between them. VICKY is the one owner of where they are (2026-10-01, Doc: "A then B"; docs/notes/status-bars.md). Writes say what is wanted, reads of \$B5-\$B7 say what is in force:
\$B0	BANDTOP a program's top band, rows (used while BANDCTL bit1)
\$B1	BANDBOT a program's bottom band, rows
\$B2	BANDCTL bit0 the user's bands are on – the F12 switch; the host sets it and a guest write cannot change it bit1 the bands are the PROGRAM's: its heights, and K/OS
	draws nothing in them. It works with bit0 off. Set it,
	then call VIDEO (\$FF92); a program MUST clear it and call VIDEO again before it exits (BANDS.PRG is the example).
\$B3	TCOLS the text grid, columns } written by whoever sets the
\$B4	TROWS the text grid, rows } mode (K/OS's VIDEO); 0 = none
\$B5	CONOY read: the top band in force = the console's first row
\$B6	CONROWS read: the console's rows
\$B7	CONBOT read: the bottom band in force
\$B8-\$BB	BANDMAP 28-bit: the bands' OWN text32 cells (option B, 2026-10-01): the top band's rows, then the bottom's, TCOLS cells a row. While layer 0 shows CONMAP, its band rows are drawn from here and not from the map, so nothing that writes the console's map can reach a band. 0 = off.
\$BC-\$BF	CONMAP 28-bit: the console's text32 map, as K/OS declares it. A program that points layer 0 at a map of its own gets its own rows, bands or no bands. Writing \$BF latches the layout the bands are drawn to: K/OS writes it after it has laid the console out, so a band switched on in F12 covers nothing until K/OS has moved the console for it. The rules: bands only on a grid of 40x30 or more; the user's are one row each; a program's that would leave the console under VICKY_BAND_MIN_ROWS, or take more than VICKY_BAND_MAX_ROWS between them, fall back to one each. JIM's window (\$DA05-\$DA08) is clamped inside the console.

Sprite attribute entry, 16 bytes, in main RAM:

+0,1	X (signed 16)
+2,3	Y (signed 16)
+4..7	DATA 28-bit pointer
+8	CTRL bit0 enable, bit1 8 bpp (else 4), bit2 H-flip, bit3 V-flip, bits4-5 Z: drawn after layer Z (0..3)
+9	SIZE bits0-1 width 8/16/32/64, bits2-3 height 8/16/32/64
+10	PALOFS (4 bpp: index = PALOFS«4 pixel)
+11..15	reserved

Pixel 0 is transparent. No per-line limit. 128 sprites.

SHEILA – the display-list coprocessor (Doc named it, 2026-08-22; the Amiga's copper is the ancestor). 4-byte instructions in main RAM, executed at the start of each scanline until a WAIT blocks. Register writes take effect for the line about to be drawn.

00	END stop until next frame
01	WAIT lo hi wait for line >= (hi«8 lo)
02	MOVE reg val write val to VICKY register reg
03	SKIP lo hi skip next instruction if line >= value
04	JUMP a0 a1 a2 continue at 24-bit address
05	IRQ set IRQSTAT bit2

At most 256 instructions per line are executed (runaway guard).

Layers 0..3 at \$10 + n*\$10, 16 bytes each:

+0	LCTRL bit0 enable, bits1-2 mode (0 bitmap, 1 tile, 2 text8, 3 text32), bits3-4 bpp (0=1, 1=2, 2=4, 3=8), bits5-6 cell size (tile: 8/16/32/64 px square; text: 0 = 8x8, 1 = 8x16)
+1	LPALOFS palette offset for <8 bpp: index = (value « depth) pixel
+2,3	SCROLLX 16-bit, pixels
+4,5	SCROLLY 16-bit, pixels
+6,7	STRIDE bitmap: bytes per row. tile/text: map entries per row.
+8..+B	DATA 28-bit pointer: pixels (bitmap) or glyph/tile set
+C..+F	MAP 28-bit pointer: the map

Map formats:

tile	2 bytes/cell: bits 0-9 tile index, 10 H-flip, 11 V-flip, 12-15 palette offset (used for <8 bpp). Tile pixel data at DATA + index * (size*size*bpp/8), rows MSB-first packed.
------	--

text8 1 byte/cell: glyph index. 1 bpp 8xH glyphs at DATA + g*H.
 Colours from LPALOFS: index = (LPALOFS«1) | pixel.

text32 4 bytes/cell: glyph lo, glyph hi (16-bit index), fg, bg – byte-wide
 palette indices per cell. bit7 of glyph hi = reverse.

Layer 0 is bottom. Pixel index 0 is transparent in every layer; BGCOL is the ground (text32 bg is never transparent). Changed 2026-08-22 from “opaque in the lowest layer” so SHEILA backgrounds show under text.

The layout in force (\$B5-\$B7): top band, console rows, bottom band. rows is

The layout in force (\$B5-\$B7): top band, console rows, bottom band. rows is 0 while no text grid has been declared (\$B4 = 0).

A.3 The network

Generated from core/net.h.

The N: device

The network, three ways – each borrowed from the machine that did it first.

The Meatloaf rule (the C64’s Meatloaf cartridge): a URL is a file name. A name beginning http://, https:// or tnfs:// that reaches the filesystem device (\$D300) for reading – LOAD, TYPE, CP, RUN, EhBASIC’s LOAD, BBC BASIC’s LOAD on the Tube – is fetched and served like a file. No ROM change: the ROM passes names through untouched and the host fetches.

TNFS (the Spectranet’s file protocol, the one FujiNet and Meatloaf servers speak; UDP, port 16384): a tnfs://host[:port]/path is also a DIRECTORY. CD tnfs://host/path puts the machine’s current directory on the server: DIR lists it, a bare name loads from it (so the REXX rule runs programs off the internet), CD .. climbs it, CD - comes home. The server is read-only from here.

The N: device (FujiNet’s network device, as the Atari and Apple saw it), at \$D900, for programs that want a live connection: four channels, each a URL opened for reading and writing.

\$D900 W command

\$D901 R status (0 ok, 1 not found / refused, 2 i/o error, 3 bad command,
 4 closed by the peer, 5 name too long, 6 not fitted)

\$D902 *RW* channel 0-3
\$D904-\$D907 **NAMEPTR** 28-bit -> URL, NUL-terminated
\$D908-\$D90B **ADDR** 28-bit
\$D90C-\$D90F **LEN** bytes requested; updated to bytes done
\$D910-\$D913 **SIZE** after OPEN: the content length, \$FFFFFFF if unknown; after STATUS: bytes waiting
 commands: 1 OPEN (tcp://host:port a connection; http://, https://, tnfs:// a file, fetched whole, then READ)
 2 READ (up to LEN bytes to ADDR, what has arrived so far; LEN = done, 0 = nothing yet)
 3 WRITE (LEN bytes from ADDR; tcp only)
 4 CLOSE
 5 STATUS (SIZE = bytes waiting; status 4 once the peer has closed and the bytes are gone)
 6 GET (the Meatloaf rule for programs: fetch the whole URL into ADDR, at most LEN; LEN = done, SIZE = total)

Reads never block the machine: a program polls, as it would a UART. Underneath: core/net_plat.h – sockets and curl on the desktop, Circle's stack on the Pi (no TLS there: https answers 6).

A.4 JIM, the terminal

Generated from core/term.h.

JIM

JIM, the terminal (\$DA00) – the Beeb's third page, given a job: a VT100 with the ANSI colour and VT220 editing additions, in hardware – the way a real 8-bit machine got a serious terminal: a card, not a program. It draws on the VICKY text32 screen the ROM console uses, inside the geometry the ROM gives it, so the console and the terminal share one screen and one cursor. Anything that needs a terminal writes its byte stream here: the ROM for the Tube (CP/M programs set up for VT100/ANSI, BBC BASIC's console edition) and TELNET for the BBSes (ANSI-BBS: CP437 glyphs, 16 colours).

\$DA00 *W DATA* a byte of the stream
\$DA01 *R STATUS* bit7 a reply byte waits; bit0 the stream moved the cursor since CX/CY were written

- \$DA02 *R* **REPLY** the next reply byte (pops): answers to ESC[6n / ESC[c, and translated keys
- \$DA03 *W* **KEY** a K4510 key code (io.h): its terminal bytes go to REPLY (arrows ESC[A.. or ESC OA.. in application mode, Home/End, PgUp/PgDn/Ins ESC[n~, Del \$7F, F1-F4 ESC OP.., F5-F12 ESC[15~..., everything else through unchanged)
- \$DA04 *W* **CTRL** 1 reset (modes, attributes, cursor home; the screen kept) 2 clear the screen and home
- \$DA05-\$DA0D *RW* **COLS ROWS OX OY CX CY FG BG STRIDE** the window: origin (OX,OY) cells, STRIDE cells per row. The window is kept inside the console VICKY lays out (\$D0B5/\$D0B6): moved, not shrunk, if it would cross a status band (2026-10-01).
- \$DA0E *RW* **FLAGS** bit0 cursor shown (blinking) bit1 read: application cursor keys (DECCKM) bit2 PETSCII mode bit3 the status bands are the program's – a DOOR onto VICKY's BANDCTL bit1 (\$D0B2), which is where the claim lives since 2026-10-01; see core/vicky.h.
- \$DA0F *RW* **BANDTOP** a door onto VICKY's BANDTOP (\$D0B0)
- \$DA16 *RW* **BANDBOT** a door onto VICKY's BANDBOT (\$D0B1). The three doors are kept for programs written before VICKY owned the layout; new ones use \$D0B0-\$D0B2.
- \$DA17 *RW* **CODEPAGE** 0 strict CP437 (power-on), 1 the K4510 page: JIM's table and the fonts follow
- \$DA18 *RW* **SCREEN** which screen JIM shows: 0 K/OS, 1 the terminal (a session on the Linux beneath or beyond, /SYSTEM/ETC/TERMINAL.CFG; the TERMINAL command writes 1). Alt+1 / Alt+2, F12 > Screen and ESC] 4510 ; kos BEL from the session switch too (2026-10-05)
- \$DA10-\$DA13 *RW* **BASE** 28-bit address of the text32 map (reset: \$030000)
- \$DA14,\$DA15 *RW* **DEFFG DEFBG** the colours SGR 0 / 39 / 49 return to

Sequences: the VT100 set (cursor, ED/EL, DECSTBM, DECSC/DECRC, IND/RI/NEL, tabs, DECAWM/DECOM/DECCKM, DEC line drawing via ESC(0 and SO/SI, DSR, DA, DECALN, RIS), ANSI SGR 0/1/4/5/7/22/24/27/30-37/39/40-47/49/90-97/100-107 and 38;5;n / 48;5;n for n < 16, VT220 ICH/DCH/IL/DL/ECH/SU/SD/CHA/VPA, IRM, ESC[?25 cursor, ESC[s/u, DECSTR, DECSCUSR (ESC [n SP q: 0-2 block, 3-4 underline, 5-6 bar – the shape VI changes with its mode). Bytes \$80-\$FF are glyphs (CP437). UTF-8: ESC % G on, ESC % @ off (CTRL 1 leaves it). On, a UTF-8 sequence draws as its CP437 glyph (or a near one, or '?'), and a byte that continues no sequence is CP437 as

before. The ‘j shell and TELNET turn it on for their sessions. Two K4510 additions, for a program that draws its whole screen through JIM (EDIT, PROG, WORD; demo/jimscr.h), 2026-10-05: ESC[?4510h makes 38;5;n and 48;5;n with $n < 16$ the palette’s own entry n (SGR’s ANSI order reaches only twelve of the sixteen), ESC[?4510l puts xterm’s meaning back; SGR 11 draws the bytes \$00-\$1F and \$7F as their glyphs (all but BS HT LF VT FF CR SO SI ESC: the Linux console’s display-control flag), SGR 10 stops. A reset clears both. And the terminals’ synchronized update, 2026-10-06: ESC[?2026h holds the glass at the last whole picture while a program redraws, ESC[?2026l shows the new one at once (half a second at most; a reset ends it).

A.5 Memory and the far view

Generated from core/mem.h.

The ROM window and the stub page

The ROM image lives in the top 64 KB of physical memory and is seen in the unmapped CPU view from `mem_rom_base` up; the physical RAM at \$A000-\$FFFF is “RAM under the ROM”, revealed by banking blocks 5/7 onto \$A000/\$E000 (K-05). The page \$FF00-\$FFFF always reads the ROM, whatever is banked: the system-call stub and the vectors live there.

K/OS’s workspace

K/OS’s workspace – 1 KB of RAM in the I/O page, \$DB00-\$DEFF (2026-10-06), where no device is. Visible wherever the I/O is – so whenever the ROM runs, whatever a program has banked. The ROM keeps its C stack there (its base page was there too at first, and cost the host 6-7%: it is at \$0600). The bytes are the RAM under the I/O page at the same addresses, physical \$00DB00-\$00DEFF: a buffer on the ROM’s C stack has the same address for the CPU as for DMA and the devices, which take physical ones (the first try put the workspace elsewhere, and every file name the shell built on its stack was read by the file device as empty), and SWAP’s 64 KB image includes it, as it included the old stack at \$0600.

Chapter B

Filing an Issue

This machine has one author, and an issue you file will, in all likelihood, be handed straight to an AI coding session to work on, as written. That is worth knowing, because it changes what a good report is: not a description of the fault, but the machine's own account of it. The machine can write most of that account itself, and the whole of this appendix is how to let it.

B.1 Before you try again

None of these is politeness. Each one removes a day.

Turn DUMP ON before you reproduce it. Then every fifteen seconds, and again the instant you type DUMP once it has gone wrong, the machine writes its whole state — registers, the screen as you see it, the last 4096 instructions, your keystrokes, and the log of every command you have typed — to `dumps/dump-NNN.txt` on the host. A dump taken while it is wrong is worth more than any description either of us could write, and DUMP ON means you cannot miss the moment.

Try it once with `k4510 --no-startup.bat`. A `/STARTUP.BAT` that has gone wrong looks exactly like a machine that has gone wrong, and telling them apart has cost this project a day before. Ten seconds of yours settles it.

Check this book. If the machine disagrees with a page here, that is still a fault worth filing — it is this book's, and it is fixed the same way. Say which page.

B.2 BUG writes the report

Then let the machine do the writing:

```
/] BUG
```

BUG interviews you — seven questions, one at a time, each answered in a line or two — and writes the finished report to

```
/SYSTEM/LOG/BUGREPORTS/BUG-YYYYMMDD-HHMMSS.TXT
```

making the directory if it is not there. Five lines it fills in itself, and they are the five most often got wrong by hand: which machine (the K4510 on its own Linux, or in a window on somebody's desktop), the exact build (INFO -v says the same — a trailing + means the emulator was built from an edited tree, which is exactly the report that will not reproduce), the screen you are in, the last dump, and when.

The questions are the ones a coding session would otherwise have to ask you: where you were (the shell, a BASIC, CP/M...), what you typed *exactly*, what happened, what you expected instead, and *why* you expected it — the line most reports leave out. The K4510 is a fantasy machine (page 142): there is no silicon it can be measured against, so “wrong” only means something next to what you were expecting and where that came from. It is also what separates a bug from a design decision from an error in this book, which are three different repairs.

BUG is a program rather than a command, so it reaches you everywhere the * escape does: *BUG from either BASIC, from the monitor, from CP/M.

DUMP first, then BUG. If the report's Dump line says there is none, the more valuable half is missing. Type DUMP while it is still wrong and run BUG again — the two travel together, and this is the one thing that is easy to do in the wrong order.

B.3 Where it goes

<https://github.com/mlongval/k4510/issues>

From the host, the report is in fs/SYSTEM/LOG/BUGREPORTS/ and the dump in dumps/: open an issue, attach both, and you are done — the report is the whole text of the issue. The form there asks BUG's questions again for anyone who has no report file: a fault in this book, a machine that

will not boot, a telephone. If you have no account there, the same two files in an email are worth exactly as much.

Feature requests and disagreements are welcome in the same shape — “what I expected” does the work whether or not anything is broken.

One thing to check before you attach a dump. It carries the shell log, which is every command line you typed in that session, and file names from your own disk along with them. It is a plain text file: open it and have a look before it goes anywhere public.

Chapter C

The Sound, and What It Took

Every other chapter of this book describes the machine as it is. This one describes how it got that way, because the sound took longer to get right than anything else in it and the route was instructive. Nothing here is needed to use the machine. It is here because a fantasy computer is a series of decisions, and the decisions about sound were the ones that most often turned out to be about something else.

It has one thesis, and it is worth stating before the evidence rather than after: **the sound was hard because keeping a real-time audio ring fed from a machine that runs in bursts is hard, not because emulating a SID is expensive.** Every fault the sound reported turned out to be somewhere else, and the chip emulation — the part everyone suspects — has never once been on the critical path.

If you are reading it to find out what happened to the SID chips: they are gone, and the last two sections say why. The reason is not the one you would guess from the six weeks described here, which is most of what makes it worth writing down.

C.1 Four chips, because it is a fantasy

The C64 had one SID. The plan for this machine was four, at \$D400 and every \$20 after it, for no better reason than that a machine which never existed may have whatever its author thinks it should have and four is a chord. Dag Lem's reSID — the emulation that has been the reference for thirty years — was vendored unchanged from VICE 3.3, wrapped in core/sid.cc, and mixed to 16-bit mono. The frontend fed a ring buffer a scanline at a time and an audio callback drained it at 48 kHz.

That took an afternoon and it worked. Everything after this section is the six weeks of it not quite working.

C.2 A SID is not a CPU

The first mistake was in the first version and is the kind only a fantasy machine can make: the SIDs were clocked at the CPU's clock.

On a C64 the two are the same crystal, so the question never comes up.

Here the CPU runs at 40.5 MHz and the SID is a 1 MHz part, and clocking reSID at 40.5 million cycles a second cost **21 milliseconds a frame** — more than the whole frame — and put every pitch out by a factor of forty-one. The fix was one line and the lesson outlived it: *the SID's clock is not the CPU's*, and every later change that touched the clock had to be told so again. The most recent time was in August 2026, in a benchmark that swept the CPU clock and forgot to tell reSID, and rendered every note at the wrong rate.

C.3 “Choppy choppy choppy” — which was not the sound

The machine's first outing on real hardware, a Raspberry Pi 3B+ on a television, produced the report that named the problem for the rest of the project: *the SID sounds on the RPi3 are very very bad, choppy choppy choppy*.

It was not an audio bug. The Pi was running the machine at about 17 ms against a 16.7 ms frame, and two recent changes had spent that slack — a debug recorder storing every instruction fetch, and a rule that made the I/O page win on every memory access, which put an extra comparison on every read and write in the machine. Neither was visible on a desktop. Both were fatal on the Pi, and what they broke was the sound, because the sound is the first thing that breaks when a frame runs long.

That is the pattern the rest of this appendix is about. **The sound is the machine's alarm bell.** Almost every time it rang, the fault was somewhere else.

C.4 Silence is not free

With a clean power supply and honest measurements, the Pi at 20 MHz sat 1.6 ms over its frame budget. Of the 18.3 ms, **3.4 was the SIDs** — all four of them, three of them silent.

reSID costs the same for silence as for sound: a chip that has been written to is a chip that must be clocked, whether or not anything is audible. So a chip is now clocked and mixed only once the machine has written to it since reset, which at the prompt is three chips' worth of frame given back. Note what that 3.4 ms was — four chips *clocked*, not four chips playing — and that it is the largest figure the sound has ever cost anywhere in this project.

That fix has a tail which is still in the machine: the cost is latched by the *first write*, not by whether anything is sounding. A program that pokes a SID register during setup and then goes quiet pays the full price for the rest of the session. On a desktop that is half a millisecond of sixteen and nobody notices. On a Pi with no slack it is worth knowing.

C.5 The ring that started empty

On the Pi the audio callback is served by the same core that runs the machine, once a frame, a block at a time. A frame makes 800 samples; the block was 1024; the ring started empty. So the ring's level lived between nothing and one frame, and a callback regularly found 800 real samples and 224 of silence — a gap, on a steady beat, at a frame rate the machine was comfortably holding.

Smaller blocks and a lead of three of them — 32 ms, which a music player does not feel — and the level never touches the floor.

It made things worse. At 15 MHz the gap count went from 40–48 in two seconds to 52–62.

C.6 Chips that play on without the machine

The lead failed because the model behind it was wrong, and the numbers said so plainly. At 15 MHz the Pi ran at 58–60 frames a second, not a locked 60. Audio was made only by the machine's frames, 800 samples

each. At 58 frames that is 46,400 samples a second against the 48,000 the device consumes. *Any* shortfall drains the ring and then gaps for ever after; a lead only postpones the first one. Even rounding the cycles per scanline down from 520.83 to 520 is a 0.16 % deficit on its own.

So the machine stopped tying its sound to its frames. Real hardware does not stop its sound chips because the processor stalled — a SID keeps oscillating whatever the 6502 is doing — and now nor does this one: after a frame, if the ring is below its target, the SIDs are clocked on *without the CPU* until it is full. The SID clock does not move, so pitch does not move; a slow frame sustains a note a fraction longer rather than cutting it.

This is the most important decision in the whole story, and it is the one that came back.

C.7 The meter that read zero

With the chips decoupled, and with the Pi's callback served every 128 lines rather than once a frame, the gap counter read **zero**, on three boots, at the default clock. By every instrument the project had, the sound was clean.

Doc: *both say 0 drops but I still can hear them.*

He was right and the meter was wrong. The gap counter counts callbacks that found the ring *empty* — it counts silence. And the decoupling exists precisely to ensure the ring is never empty. So across the entire band where a host is merely losing rather than drowning, the counter reads zero and there is nothing wrong with the sound except that a growing fraction of it was not made by the program. The SIDs were playing on without the machine. That is what he was hearing.

It is now counted. Beside the gaps there is a second number: how many samples the machine did not make and the chips supplied. On one host, at rows both instruments had called clean:

Clock	Frames	Gaps	Filled in
30 MHz	53 fps	0	36,464
20 MHz	60 fps	0	16,818
10 MHz	60 fps	0	1,609

of 96,000 samples in the two seconds measured. At 30 MHz *more than a third of the sound was not the program's*, and every meter said clean.

Clean now means three things at once, because any two of them can be true while the sound is wrong: sixty frames a second, no gaps, and under two per cent filled in.

The moral, three times over. A performance regression that presented as an audio bug. An audio fix that made the gap count worse. A gap counter that read zero because an earlier fix had guaranteed it would. Each time, the instrument was measuring something adjacent to the fault — and each time, the person listening to the machine was right before the numbers were.

C.8 A machine keeping somebody else's time

The newest finding, and the one that reframes the rest: on the desktop the machine was not keeping its own time at all.

The Pi has always paced itself against the wall clock, because its graphics layer blocks until the display flips and a frame that overran by a millisecond would wait for the next one and halve the frame rate. The desktop did the opposite — it left the pacing to the display's vertical sync and did none of its own, so the machine ran at whatever the compositor handed it. On one host, a 60 Hz display, that was **51.8 frames a second**, with the act of putting the picture on the glass taking 16.9 ms of a 19.3 ms frame.

A machine at 51.8 frames a second makes 51.8 frames of sound where the device wants 60. The chips filled in the missing seventh, exactly as designed, and it was audible — and the new counter measured it at 17 %. The machine now keeps its own 60 Hz on the desktop as it always has on the Pi, and presents when it is ready:

	Whole frame	Onto the glass
before	19.294 ms (51.8 fps)	16.867 ms
after	16.668 ms (60.0 fps)	0.260 ms

C.9 Where it stood, when it was decided

By the end of it the machine had four reSID chips in it and ran as many of them as you asked for, and the question of whether to keep them was open. It had been raised as a performance question, after a Raspberry Pi that would hold only 15 MHz. On that ground it was settled, and settled *against* replacement. **reSID was not the cost.** One chip sounding measured **0.039 ms of a 16.67 ms frame** on a desktop — a quarter of one per cent — and four chips written since reset came to about half a millisecond. The largest the sound ever cost was the 3.4 ms of §C.4, on the Pi, before silent chips stopped being clocked, and that was a book-keeping fault rather than a fidelity one. Nothing measurable was to be bought by removing them.

Where the question was real was identity. The SID is a part this machine inherited rather than chose: it was in here because the C64 had one, in the same way the * prefix is in here because Acorn had it. A machine that never existed is entitled to a voice its author picked on purpose — and the original specification for this one read *4 SIDs/OPL2*, so a synthesiser of the other tradition would not be a betrayal of the plan but the rest of it. Against that: four SIDs were what the machine had sounded like since its first afternoon, they cost almost nothing, and the entire body of C64 music played on them, which is not a small thing to hand back.

C.10 What was decided

On 1 September 2026 the SIDs were turned *off* — not deleted. The OPL2 became the machine's default chip on every host, the menu rows that offered SIDs went away, and one line in `k4510.cfg` gave them back. That is the disposition this project calls OFF, and it exists so that a decision of this size can be tried before it is taken.

On 5 September it was taken: *“nuke anything having to do with SIDs”*. Out went reSID, the four-chip mix with its phase carry, the SID clock select, the settings and their migration, the SID demos and the player, the tune archive, and every register write to \$D400 in the ROM, in EhBASIC and in Mad Pascal's runtime. The machine has one sound chip now and

it is the OPL2, at \$D480, nine FM voices wired the AdLib's way.

Three things fell out of it that are worth recording:

- **The sound sequencer had to learn FM.** It was the SIDs' last real user — BBC BASIC's SOUND, Mad Pascal's Sound and the invaders all reach it — so its four channels became OPL2 voices, with the Beeb's noise channel as a feedback-heavy patch and the other three as plain two-operator tones. The patch is rewritten on every note, because a program may have zeroed the chip for its own purposes and the sequencer must still sound after it.
- **The render seam got a name.** `core/audio.c` is a page of C that owes nothing to any chip: cycles in, samples out at the device's rate, the clock advanced. Everything this appendix is about — the ring, the lead, the filling-in — lives there now, and a future chip plugs into it rather than into the machine.
- **Save states changed format,** and old ones refuse to load rather than pretending.

C.11 And the moral, once more

Read back, the six weeks argue for a conclusion the last two sections then decline to draw. Every fault the sound reported was somewhere else; the chip emulation was never on the critical path; and when the machine was measured properly, the SIDs cost a quarter of one per cent of a frame. The performance case for removing them was nil, and this appendix says so at length.

They were removed anyway, and rightly, because the question was never performance. It was what the machine is. That is the difference between a fantasy computer and a product: a product's parts are decided by measurement, and a fantasy computer's are decided by its author — having first taken the trouble to find out what the measurement actually said, so that the decision is an honest one rather than a rationalised one.

Everything above this section is kept exactly as it was written, because the route matters more than the destination, and a book that qui-

etly rewrote its own history would be worth less than one that admits what it tried.

Chapter D

The Code Page

Every character the K4510 shows is one byte, and the code page says which character each of the 256 bytes is. The machine's is IBM's code page 437 — the PC's, the one with the box drawing — as every PC program and every bulletin board expects it. That is what it starts in, and what this book's pages on the machine are written in.

There is a second page, the *K4510 page*, for when it is wanted. It is CP437 with 26 of its places given to letters Western Europe needs and CP437 never had: the accented capitals of French, Italian and Spanish, æ and £, the euro, the section and pilcrow signs, German's quotes, and ø. The places they took were CP437's Greek and mathematical symbols and a few marks nobody used. The box drawing, the shades and blocks, and every accented letter CP437 already had — the lower-case letters French, Spanish and German are written with — are in the same places in both.

CODEPAGE K4510 chooses it and CODEPAGE 437 goes back; CODEPAGE alone says which is in use. F12 → Terminal → *Code page* is the same choice, and the machine remembers it. The screen changes at once: the page's font takes the place of the other, and the characters already on the screen are drawn again from it.

Like the register reference, this appendix is not written by hand: doc/guide/mkcodepage.py makes it from core/codepage.h, which holds both pages. JIM turning UTF-8 into bytes and back, the keyboard, the fonts and k4510-remote all read that file, so none of them can disagree with this page, or with each other.

D.1 The K4510 page

Generated from core/codepage.h. The row is a byte's high hex digit, the column its low one: \$82 is row 8, column 2. The 26 places that are not CP437's are in bold. sp is the space and nb the no-break space; \$00

draws nothing.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☹	♥	♦	♣	•	◼	◯	◻	♂	♀	♪	♫	*	
1	▶	◀	↕	!!	¶	§	■	⬆	⬇	⬇	↗	↖	↔	↔	▲	▼
2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	◊
8	Ç	ü	é	â	ä	à	â	ç	ê	ë	è	ï	î	ï	Ä	Å
9	É	æ	Æ	ô	ö	ò	û	ÿ	Ö	Ü	φ	£	¥	€	§	
A	á	í	ó	ú	ñ	Ñ	ª	º	¿	¶	¼	½	¾	¿	«	»
B	▨	▩	▪		†	‡	§	¶	§		¶	¶	¶	¶	¶	¶
C	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞
D	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞	⌞
E	À	Â	Ê	Ê	Ê	µ	Î	Î	Ô	Ù	Û	Û	Û	Á	Í	Ó
F	Ú	±	Ì	Ò	Œ	Œ	÷	ø	°	ø	·	„	“	²	■	NB

The 26 places

Byte	The K4510 page		CP437 had
\$9E	€	euro sign	ℙ
\$9F	§	section sign	ƒ
\$A9	¶	pilcrow sign	⸗
\$E0	À	latin capital letter a with grave	α
\$E2	Â	latin capital letter a with circumflex	Γ
\$E3	È	latin capital letter e with grave	π
\$E4	Ê	latin capital letter e with circumflex	Σ
\$E5	Ê	latin capital letter e with diaeresis	σ
\$E7	Î	latin capital letter i with circumflex	τ
\$E8	Ï	latin capital letter i with diaeresis	Φ
\$E9	Ô	latin capital letter o with circumflex	Θ
\$EA	Ù	latin capital letter u with grave	Ω
\$EB	Û	latin capital letter u with circumflex	δ
\$EC	ÿ	latin capital letter y with diaeresis	∞
\$ED	Á	latin capital letter a with acute	ψ
\$EE	Í	latin capital letter i with acute	ε
\$EF	Ó	latin capital letter o with acute	η
\$F0	Ú	latin capital letter u with acute	≡
\$F2	Ì	latin capital letter i with grave	≥
\$F3	Ò	latin capital letter o with grave	≤
\$F4	œ	latin small ligature oe	[
\$F5	Œ	latin capital ligature oe	]
\$F7	ø	latin small letter o with stroke	≈
\$F9	Ø	latin capital letter o with stroke	•
\$FB	„	double low-9 quotation mark	√
\$FC	“	left double quotation mark	ⁿ

D.2 The languages each writes

CP437 has every lower-case letter of French, Spanish, German, Italian and the Nordic languages, and É Ç Ñ Ä Ö Ü Å Æ among the capitals — enough for most written French, which leaves the other capitals unaccented. With the K4510 page these are complete in both cases: French, Italian, Spanish, German, Danish, Norwegian, Swedish, Finnish, Dutch, Catalan, Galician, Basque, Irish, Scottish Gaelic, Afrikaans and Albanian.

Portuguese is four letters short in either (ã õ Ã Õ).

Text arriving as UTF-8 — from !, from TELNET, over ssh — is turned into the page in use on the way in; a character it has no place for becomes its nearest, and the curly quotes their plain forms.

D.3 The first 32, and \$7F

Bytes \$01–\$1F and \$7F draw CP437’s little pictures in both pages: the faces, the card suits, the arrows, the notes. As text they are also the control codes — \$07 rings, \$1B is Escape, \$18 to \$1A move the cursor — and the console obeys them rather than drawing them. So a program that wants a picture on the screen puts the byte into screen memory itself, as the games do.

D.4 Typing them

Every character a keyboard layout has a key for reaches the machine, if the page in use has it. The dead keys compose the rest: ` then e is è in either page; ` then A is À in the K4510 page, and a plain A in CP437, which has no place for it. FONTED redraws any of the 256, in both sizes of the font, and shows the Unicode value of the character it is editing.

D.5 A BBS

TELNET draws a CP437 session — a bulletin board, an old system, a far end that never negotiates — with strict CP437 whatever page the machine is in, so ANSI art is always exactly as it was drawn. A Unix host, which takes the XTERM-COLOR terminal type and talks UTF-8, is drawn in the page in use, and the machine’s font comes back when TELNET hangs up.

Disclaimer

The K4510 is a hobby project: a computer that does not exist, built for the pleasure of building it. It is offered to you as a gift, and it comes with exactly the guarantees a gift comes with — none.

In the words of the licence it is distributed under (page 149), and meaning every one of them:

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

IN NO EVENT WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM.

In plainer words: nobody promises this machine works, that it will keep working, or that anything it writes to your disk is safe. Keep backups. Do not fly aeroplanes with it, run hospitals on it, or trust it with money. It is a toy, and it is allowed to be wrong.

The same goes for this book: where it and the machine disagree, the machine is right. Nothing here is endorsed by or affiliated with Acorn, Commodore, Microsoft, Digital Research, the MEGA65 project, the Raspberry Pi Foundation, or anyone else gratefully named in these pages. The mistakes are the author's alone.

Constructive comments can be left at the repository's issues page:

<https://github.com/mlongval/k4510/issues>

All complaints, criticisms and negativity can be addressed to
[/dev/null](#)

Thank You

The K4510 is a machine that never existed. Almost everything in it that *does* exist was written by somebody else first, and given away. This chapter is the thanks; the Licences chapter (page 149) is the paper-work.

The heart of the machine

Gábor Lénárt (LGB) wrote the 45GS02 CPU core in Xemu, the MEGA65 emulator. It runs here unchanged — not a line altered, not a bug fixed — and every instruction this book describes is his code executing. There would be no K4510 without it.

Jarek Burczyński and **Tatsuyuki Satoh** wrote `fmopl`, the OPL2 emulation in MAME that this machine's YM3812 is — by way of VICE, vendored unaltered. Nine voices of FM that any AdLib register list from 1987 can drive is their work, not ours.

The VICE team, whose decades of emulation scholarship — documentation, test programs, arguments settled in code — this project consulted at nearly every turn.

The tongues

Lee Davison (1966–2013) wrote EhBASIC, the machine's first BASIC and still the one that comes up in ROM. He is not here to be asked about the graphics and sound words bolted onto it; the machine carries his name in its README and its startup banner instead.

Richard T. Russell wrote BBC BASIC, and has kept writing it for forty years. The console edition of his BBC BASIC for SDL 2.0 (BBCTTY) is what runs on the Tube co-processor in Chapter 5, under the zlib licence

he publishes it with. “BBC BASIC” is the name of his interpreter; this book uses it only to say what is running, and claims nothing in it.

Marcelo Dantas (“Mockba the Borg”) wrote RunCPM, which is the whole of the machine’s CP/M: a complete CP/M 2.2 with its own CCP, vendored here unmodified and simply handed a Z80’s worth of address space.

Scot W. Stevenson, **Sam Colwell** and **Patrick Surry** wrote Tali Forth 2 and put it in the public domain — a Forth written to be read, which is why Chapter 6 can be honest about how it works.

Tomasz Biela (“tebe”) wrote Mad Pascal and MADS; the K4510 target in pascal/ is a guest in his compiler. **Wojciech Bociański** (“bocianu”), whose Neo6502 target showed how such a guest should behave.

Steve Wozniak wrote Wozmon in 1976 in 256 bytes, and the monitor in this machine is still recognisably it.

Jim Butterfield (1936–2007) wrote Supermon, and gave it away as he gave away everything: the machine language monitor a generation of Commodore programmers learned on, published in *Compute!* for anyone to type in. Supermon+64 V1.2 is SUPERMON here, ported and not rewritten. **J. B. Langston** restored and commented the source (jblang/supermon64), which is what made porting it a day’s work rather than a month’s; he asks only for the attribution, and it is given gladly.

Letters on the screen

Fonts are the part of a computer you look at longest, and clean ones with a clear pedigree are hard to come by.

Ville-Matias Heikkilä (“Viznut”) — unscii, placed in the public domain. It is the machine’s one screen font: unscii-8 in the 240-line modes, unscii-16 at 640×480, and both in the F12 menu and the side panel.

The Linux kernel contributors — the 8x8 console font (font_8x8.c) that got the text mode on its feet, and the 8x16 VGA font the K4510x consoles wear.

Kenney makes game art and puts it in the public domain, at a scale and a standard that has quietly furnished a decade of small games. Two

games stand on it: SKYFIRE flies Kenney’s Pixel Shmup ships, and FLUFFY is drawn with **Chloe Wolfe**’s Game Boy platformer set, also CC0. In both cases the art came first and the game was written around it, which is the reverse of the usual order and a much better way to spend an evening.

Ian Schofield wrote Tek40xx, a Tektronix 4010/4014 storage tube on SDL2 that is also a telnet client. It rides along on the machine’s Linux as the second terminal (Chapter 11), built from upstream with one patch of ours.

The bare metal, which the machine no longer stands on

This section is kept because the debt is not cancelled by the code coming out. From August to September 2026 the K4510 ran on a Raspberry Pi 3B+ with no operating system underneath it, and that edition was retired on 2026-09-07 (page 4). Everything below is why it was possible at all.

Randy Rossi wrote BMC64: VICE on a Raspberry Pi with no operating system under it, 50 Hz smooth scrolling and input latency measured in single frames. His initials were in that edition’s name — the **BMC-K4510** was the K4510 on the bare-metal platform he proved out — and it is the direct reason the port existed at all: the route to the Pi was always meant to be BMC64’s `emux_api` seam, and the bare-metal case it proved is what made it thinkable. He also built the **VIC-II Kawari**, a modern drop-in VIC-II with video modes the original never had: the demonstration that you may extend an 8-bit machine’s video chip and still be working inside the tradition rather than outside it. VICKY is a more reckless cousin of that idea. **minch** (`aminch`) has taken BMC64 over from him and carries it onto the newer Pis; the project is in good hands.

Rene Stange wrote Circle, the bare-metal C++ environment that was the entire reason a Raspberry Pi 3B+ could boot straight into this machine with no operating system under it. **Xalior** wrote `circle-libsdl2`, which let the desktop emulator move to the Pi essentially as written.

The workshop

The machine is built with **cc65** (compiler, assembler, linker and runtime for the whole system ROM), Marco Baye's **ACME** (Wozmon and the demos), Zsolt Soós's **64tass** (Tali Forth and Supermon), **NASM**, **GCC** and **GNU Make**; it shows itself to you through **SDL2** — Sam Lantinga's library and its contributors', which is the window, the keyboard and the sound on the desktop, and through Xalior's port, on the Pi as well. This book is set in **XeLaTeX** with **Clear Sans** and **Iosevka**, and every screenshot in it was captured from the machine actually running — a picture that cannot be produced fails the build.

Heritage

Some debts are not code.

Acorn Computers — the Tube, the sideways-ROM model, the * command prefix, and the idea that a second processor should be a normal thing to own. The Beeb's ghost is all over this machine.

Commodore — the other half of its soul: PETSCII, the SID, and the line from the PET to the C65 that stopped one machine short of this one.

The MEGA65 project — for keeping the 45GS02 and the C65 dream alive in real silicon, and for open-roms.

Kim Lemon and the Lemoners — Lemon64 has been the C64's front door since 1998: the games database, the reviews and scans, the music, and a forum that actually answers. Twenty-eight years of one person's dedication to a machine's community, and a good half of the small facts this project needed came from there.

The Meatloaf project (idolpx) — the C64 cartridge that made a URL a file name, which is the whole of this machine's rule for the network: name it, and it is fetched. **The FujiNet project** — the network device the Atari got first, whose N: device this machine imitates and whose TNFS servers it speaks to; `CD tnfs://fujinet.online/` is a directory on theirs.

Paul Scott Robson wrote the Neo6502's firmware — the operating system and BASIC that make an Olimex Neo6502 a computer rather than

a board — and the handbook that documents it. That book is the one this one is modelled on, down to the page size, and the rule that every example must be run and photographed before it may be printed is his. The other model is the **MEGA65 User's Guide**.

The High Voltage SID Collection and the composers in it, who kept thirty years of a machine's music alive and free to listen to. The K4510 played it for one summer, through a SIDPLAY that retired with the chips. None of that music was ever distributed with this machine; it is theirs.

The project's orchestrator is Michael Longval — Doc — who decided what the machine is, what it borrows and what it refuses, what gets built next and what gets thrown away, and read every page of this book against the machine.

And the machine's other author. Most of the K4510's own code — VICKY, SHEILA, the DMA engine, the ROM, K/OS, the Tube ULA, the editors, RX — was written in conversation with Claude (Anthropic's Claude Code), over several months of long sessions with a build log to prove it. The design decisions are the author's; a great deal of the typing was not.

And whoever is missing

This list was assembled by hand and is certainly incomplete. If your work is in this machine and your name is not on this page, that is an error and not a judgement — open an issue at <https://github.com/mlongval/k4510> and it will be fixed in the next printing.

Licences

The Thank You chapter (page 144) is the thanks; this is the record. The authoritative copy is `LICENSES.md` in the repository — if the two ever disagree, that file wins.

The project itself

The K4510 as a whole is distributed under the **GNU General Public License, version 2 or (at your option) any later version**. Everything written for this project — the machine, VICKY, SHEILA, the DMA engine, the MATH unit, the system ROM and K/OS, JIM, the Tube ULA, the network layer, the editors, the demos, RX and the Pascal target — is Copyright © 2026 Michael Longval and is released under those terms. The choice is not really a choice: the CPU core and the OPL2 emulation are GPL-2.0-or-later, and a work containing them must be too.

Code inside the machine

- **Xemu 65xx/45GS02 CPU core** — Gábor Lénárt. `core/xemu/`: `cpu65.c`, `cpu65.h`, `cpu65_mega65_timings.h`, `cpu65ce02_disasm_tables.c`, used unchanged. *GPL-2.0-or-later*.
- **fmopl (YM3812/OPL2)** — Jarek Burczyński and Tatsuyuki Satoh, from MAME by way of VICE. `core/opl2/`, vendored unaltered. *GPL-2.0-or-later*.
- **reSID** — Dag Lem, as shipped in VICE 3.3, was `core/resid/` until 2026-09-05 and is no longer in the tree (Appendix C). *GPL-2.0-or-later*; recorded here because a vendored component that quietly disappears is worse than one recorded as gone.
- **BBC BASIC for SDL 2.0, console edition (BBCTTY)** — Richard T. Russell. `tube/`, vendored and altered (see `tube/ALTERED.md`). *zlib licence*.

“BBC BASIC” is the name of Mr Russell’s interpreter, which he publishes under his own arrangement with the BBC. This project has not sought, and does not hold, any licence to the name: it is used here only to identify the interpreter, which is why this book says “Richard Russell’s BBC BASIC” and not “the K4510’s”.

- **EhBASIC 2.22** — Lee Davison, in the ca65 form from jefftranter/6502. `basic/basic.asm`. *Free for non-commercial use only*; derivatives must carry the words “Derived from EhBASIC” — see `basic/README-EhBASIC.txt`. It ships as a separate program (`fs/LANG/EHBASIC/ehbasic.prg`) and is not linked with the GPL code.
- **RunCPM** — Marcelo Dantas. `cpm/src/`, vendored unmodified (`cpm/VENDORED-FROM.txt`). *MIT*.
- **Tali Forth 2** — Scot W. Stevenson, Sam Colwell, Patrick Surry. `forth/tali/`, vendored unmodified. *Public domain*.
- **Wozmon** — Steve Wozniak’s 1976 monitor, reimplemented here for the 45GS10. Apple published the original listing; this machine’s version is the project’s own code under the project licence.
- **cc65 runtime** — the ROM and every `.prg` are linked against `none.lib`. *cc65’s zlib-style licence*.
- **Supermon+64 1.2** — Jim Butterfield, restored and commented by J. B. Langston (`jblang/supermon64`), ported to the 45GS10. Butterfield published it for anyone to use; Langston asks for attribution.

Fonts

- **unscii-8 and unscii-16** — Ville-Matias Heikkilä. `data/fonts/unscii/`: the upstream `.hex` files and the `font8-unscii.bin` and `font16-unscii.bin` made from them. The machine’s one screen font. *Public domain*.
- **Clear Sans** (Intel, *Apache-2.0*) and **Iosevka** (Belleve Invis, *SIL OFL 1.1*) — the two faces this book is set in. They are not part of the machine.

No Commodore ROMs here. Until 2026-08-23 the text font was derived from a Commodore 64 character ROM. That file and every derivative were removed from the repository *and from its history* before it was made public. Since 2026-09-14 the machine has one font, unscii, and reads no other: a chargen.bin in /SYSTEM/ETC is ignored.

The Linux the machine boots on

The live image is a Debian, assembled by `linux/build-live.sh` from Debian's own archive, so every package in it carries its own terms and Debian's copyright files travel with it. One thing is built from source into it:

- **Tek40xx** — Ian Schofield's Tektronix 4010/4014 terminal, built from upstream with one patch of ours. *GPL-3.0*. It is a separate program on the Linux side, not part of the emulator.

The Tektronix plot files in `linux/tek40xx/plt/` are Tek40xx's own gnuplot examples plus a page generated here. The vintage cassette plots that circulate with *other* Tektronix emulators are marked personal-use-only by their authors and are deliberately *not* vendored; the README says how to fetch them for yourself.

The Raspberry Pi build was here until 2026-09-07 and needed two components that never lived in this repository: **Circle** (Rene Stange, *GPL-3.0*) and **circle-libsdl2** (Xalior, *zlib*, with a *GPL-3.0* linker script). A `kernel8.img` built from those was *GPL-3.0* as a whole, which *GPL-2.0-or-later* code permits. The tag `alpha-0.5` is the last release that shipped one.

Build tools

Not distributed with the machine, but required to build it: GCC and GNU Make (*GPL*), cc65 (*zlib-style*), 64tass (*GPL-2.0*), NASM (*BSD-2-Clause*),

SDL2 (*zlib*), Python 3 (*PSF*), and for this book XeLaTeX (*LPPL*). Mad Pascal and MADS (Tomasz Biela, *MIT*) are needed only if you compile Pascal; the K4510 target in `pascal/` is installed into your own Mad Pascal checkout.

What is deliberately not shipped

- **The CP/M system disk.** RunCPM's `DISK/A0.zip` — Digital Research's ASM, MAC, DDT, STAT, SUBMIT and third-party tools — installs into `fs/CPM/A/0/` for your use but is not committed: its files have mixed provenance and this repository is public. The same goes for WordStar, Turbo Pascal 3 and MBASIC, which the `K-*.SUB` launchers know how to start but which you must supply yourself.
- **Your `STARTUP.BAT`** — yours, not the repository's (copy `/SYSTEM/ETC/STARTUP.SAMPLE` to start one).

If you redistribute the machine

Ship the sources you built from, keep `LICENSES.md` and this chapter with them, and remember two things that are easy to forget: EhBASIC is non-commercial-only, and “BBC BASIC” is the name of Richard Russell's interpreter rather than of anything made here — describe it as his, as this book does, and give whatever you build on it a name of its own.